

**M A S A R Y K
U N I V E R S I T Y**

FACULTY OF INFORMATICS

**Practical Threshold Cryptography
on Resource-Constrained
Security-Certified Hardware**

Doctoral Thesis

ANTONÍN DUFKA

Advisor: doc. RNDr. Petr Švenda, Ph.D.

Brno, 2024

Declaration

Thereby I declare that this thesis is my original work, which I have created on my own. All sources and literature used in writing the thesis, as well as any quoted material, are properly cited, including full reference to its source.

Antonín Dufka

Advisor: doc. RNDr. Petr Švenda, Ph.D.

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Petr, for his exceptional guidance, patience, and countless insightful discussions on research and beyond. His mentorship has been instrumental both to my doctoral studies and to my broader personal development.

I am profoundly thankful to all members of CRoCS. It has been a true privilege to be part of such an inspiring group. Our discussions on research ideas and collaboration have been invaluable for my work. I share my gratitude to Monet+ for their financial support and many discussions on practical applications of my research.

Finally, I wish to extend my heartfelt thanks to my family, whose boundless support allowed me to properly focus on all stages of my studies. To Kristýna, thank you for your constant encouragement throughout the ups and downs of this long and rewarding journey.

The research presented in this thesis was partially supported by the following research projects:

- Cyber-security Excellence Hub in Estonia and South Moravia (CHESS, 101087529) funded by the European Union.
- Digital evidence in criminal proceedings (VJ01010084) funded by the Ministry of the Interior of the Czech Republic.
- Tools for AI-enhanced Security Verification of Cryptographic Devices (VJ02010010) funded by the Ministry of the Interior of the Czech Republic.
- Examining and improving security of elliptic curve cryptography (GA20-03426) funded by the Czech Science Foundation.
- Applied research at FI (2021-2024).

Abstract

Decentralization of cryptographic computations is a promising method of decreasing the risk of secret key compromise. However, the support of threshold protocols, realizing decentralization, is limited in security-certified hardware. In this dissertation thesis, we aim to broaden the understanding of security-certified hardware's capabilities concerning the execution of threshold protocols and propose techniques to make such computation more practical.

In the first part, we analyze the current state of the two largest ecosystems of security-certified devices: Trusted Platform Modules and JavaCard-based smartcards. We studied the capabilities, performance, and security of real devices. Our findings revealed new vulnerabilities in certain elliptic-curve algorithm implementations and broadened the scope of prior ones. The analysis of threshold protocol compatibility revealed that the current interfaces are very limited, yet a promising direction for the computation of elliptic-curve-based threshold protocols on smartcards was discovered.

In the second part, we followed along the discovered direction. Building on prior work, we constructed new approaches for accelerating computation using JavaCard API. We proposed optimization techniques that leverage the asymmetric setting of a resource-constrained device connected to a computationally powerful host. Lastly, we designed new protocols interoperable with state-of-the-art approaches, enabling the selection of suitable protocols in heterogeneous environments. Combining all these approaches facilitated practical threshold computation of major elliptic-curve-based signature schemes on smartcards.

Keywords

threshold cryptography, security-certified hardware, multi-party computation, threshold signatures, digital signatures, smartcards

Contents

1	Introduction	1
1.1	Research goals	3
1.2	Contributions	3
I	Analysis of secure hardware ecosystems	7
2	Trusted Platform Modules	11
2.1	Introduction	11
2.2	Related work	13
2.3	Common Criteria and FIPS certification	14
2.4	Data collection from real devices	16
2.5	Analysis of ECC implementations	20
2.5.1	ECC secret's randomness	21
2.5.2	ECC operation timing	23
2.6	Analysis of RSA implementations	25
2.6.1	Detected changes in key generation	26
2.6.2	Detection of off-chip EK generation	27
2.7	Algorithm support and performance	33
2.7.1	Variability of performance	36
2.8	Conclusion	38
3	JavaCard-based smartcards	39
3.1	Introduction	39
3.2	Related work	41
3.3	Common Criteria and FIPS certification	41
3.4	Data collection from real devices	45
3.5	Analysis of collected data	48
3.5.1	JavaCard API and package support	48
3.5.2	Information from CPLC data	51
3.5.3	Public-key algorithm performance	52
3.6	Analysis of JavaCard open-source projects	53
3.6.1	Open-source activity in time	54
3.6.2	Usage of JavaCard API	56
3.6.3	Applet compatibility and portability	60
3.7	Conclusion	62

II	Threshold cryptography on secure hardware	63
4	Enabling threshold signature computation	67
4.1	Introduction	67
4.2	Related work	69
4.3	Secret sharing and threshold protocols	70
4.4	Required operations	71
4.4.1	Modular arithmetic	71
4.4.2	Elliptic curves	73
4.4.3	Hash functions	75
4.5	Extensions of JCMathLib	75
4.5.1	Achieved performance	78
4.6	Application scenarios	82
4.7	Conclusion	84
5	Threshold Schnorr signature protocols	85
5.1	Introduction	85
5.2	Background	87
5.2.1	Schnorr signatures	87
5.2.2	Current threshold signature protocols	89
5.3	Efficient implementation of FROST	90
5.3.1	Required operations	90
5.3.2	Implementation on smartcards	92
5.3.3	Performance evaluation	93
5.4	Interoperability of threshold Schnorr protocols	95
5.4.1	Nonce exchange	96
5.4.2	Nonce commitment	97
5.4.3	Nonce delinearization	98
5.4.4	Deterministic nonce	100
5.4.5	Summary	101
5.5	New signature protocol SHINE	102
5.5.1	Setup	103
5.5.2	Nonce caching with encryption	103
5.5.3	Signing protocol	106
5.5.4	Security analysis	106
5.5.5	Performance evaluation	110
5.6	Conclusion	112

6	Two-party ECDSA computation approaches	115
6.1	Introduction	115
6.2	Background	117
6.2.1	Additive and multiplicative secret sharing . . .	117
6.2.2	Preprocessing model	117
6.2.3	Paillier's cryptosystem	119
6.2.4	Proof of discrete logarithm knowledge	119
6.3	Three approaches for two-party ECDSA on smartcards	120
6.3.1	Lindell's two-party protocol	120
6.3.2	Two-party protocol with multiplication triples .	123
6.3.3	Presignatures	125
6.4	Implementations on smartcards	126
6.4.1	Paillier's decryption	126
6.4.2	Preprocessing with constrained memory	127
6.5	Performance evaluation	129
6.6	Conclusion	130
7	Distributing computation of an e-cash system	133
7.1	Introduction	133
7.2	Background	136
7.2.1	Digital signature	136
7.2.2	Proof of discrete logarithm equality	137
7.3	Electronic cash concepts	138
7.3.1	Blind protocol	139
7.3.2	Mint model	142
7.4	Distributing mint functionality	143
7.4.1	Distributing BDHKE	143
7.4.2	Distributing DS	145
7.4.3	DOS mitigation	146
7.4.4	Security properties	147
7.5	Implementation on smartcards	151
7.5.1	Single-party case	151
7.5.2	Extension to multi-party setting	152
7.6	Performance evaluation	153
7.7	Conclusion	156
8	Conclusion	157

Appendix	160
A Achieved results	161
A.1 Academic publications included in the thesis	161
A.2 Academic publications not included in the thesis	167
A.3 Teaching and theses supervision	169
Bibliography	171
Authored publications	171
External references	172

1 Introduction

Protection of secrets used in computations is a challenging task underpinning all cryptography. A secret and a device on which it is being processed are exposed to several threats. Malicious software running on the device may attempt to access the secret in the device's memory by exploiting improper process isolation. An attacker with physical access to the device may read out the secret directly from the device's permanent storage or RAM [1]. Side-channel emissions during algorithm execution may lead to observable dependencies on the secret, enabling the inference of information that may eventually lead to the secret's reconstruction [2].

Secure hardware elements have been designed as a dedicated system component capable of increasing the protection of secrets. It is built with its own computing capabilities and storage and provides only a limited interface, minimizing the logical attack surface. It also incorporates various physical anti-tampering measures; its manufacturing process has to undergo strict security evaluation. However, despite all this effort and precaution, key extraction attacks on security-certified devices have been repeatedly demonstrated [A1, 3, 4, 5, 6].

Threshold cryptography [7, 8] is an orthogonal secret protection approach based on decentralization. A secret is split using secure secret sharing [9] and distributed across multiple independent devices. The devices jointly perform operations with the secret shares using a threshold protocol without reconstructing the original secret in a single place. Assuming a protocol's security, it effectively eliminates the single point of failure. Furthermore, it allows for redundancy in the issued shares, maintaining availability even during the inaccessibility of some of them. The growing importance of threshold cryptography is further attested by NIST's recently initiated efforts to standardize threshold schemes [10].

Combining these two protection approaches yields complementary benefits by retaining increased hardware protections while performing distributed computation. Since secure hardware typically have limited resources and threshold protocols require costly algebraic operations, there is a need for a feasibility assessment. However, little research has

been undertaken on the execution of threshold protocols on current resource-constrained security-certified devices.

This doctoral thesis aims to address this gap and study the intricacies that arise from the combination. It surveys the capabilities of the two largest ecosystems of security-certified devices, both using public documents and empirical examination. Building on the surveys' findings, it proposes multiple layers of optimizations, demonstrating that state-of-the-art elliptic-curve-based threshold signature protocols can be computed on current smartcards with practical performance.

The first part of the thesis analyzes the properties of two major classes of secure hardware devices – Trusted Platform Modules and cryptographic smartcards. The analyses approach the problem from two main angles: 1) a study of public information available in security certification documents issued under the Common Criteria (CC) [11] and FIPS 140 [12] schemes, and 2) a large-scale data collection of capabilities, performance, and cryptographic properties of real devices. Although the analyses revealed the capability of performing modular arithmetic and elliptic-curve operations for both device classes, the TPMs' interface inflexibility prohibits the computation of threshold protocols beyond simple two-round constructions [13, 14].

The second part builds on this partially positive but nuanced answer and focuses on cryptographic smartcards. It devises new techniques for repurposing high-level API to compute low-level operations. These techniques were employed to extend a generic library [15] for performing computations on JavaCard-based smartcards. This extension enabled efficient smartcard computation of FROST [16, 17], a state-of-the-art threshold Schnorr signature protocol, and served as a foundation for three approaches to the computation of two-party ECDSA with varying trade-offs between performance and trust. In combination with computation offloading to the host device, the techniques enabled distributed verification of a BLS signature variant on smartcards in suitable settings using a newly proposed protocol. A novel concept of interoperability among different threshold protocols was proposed and studied on threshold Schnorr signature schemes, providing an option for increasing security and efficiency in heterogeneous computation environments.

1.1 Research goals

The research in this thesis is guided by two overarching goals.

Research goal I

Analysis of capabilities, performance and security of the two largest ecosystems of security-certified devices with focus on the compatibility with threshold protocols.

The first goal aims to provide insights into the state of two ecosystems of secure tamper-proof devices (Trusted Platform Modules and JavaCard-based smartcards) and broaden understanding of their capabilities for the realization of threshold protocols.

Research goal II

Design of new approaches for achieving practical computation of major elliptic-curve-based threshold signature schemes on current JavaCard-based smartcards.

The second goal builds on findings from the first goal. It expands the understanding of the realizability of threshold protocols on currently available devices by proposing multiple layers of optimizations designed for the asymmetric setting of a resource-constrained device interacting with a much more computationally powerful host device.

1.2 Contributions

This thesis is divided into two parts corresponding to the stated goals, split into two and four chapters, respectively. Each chapter is backed by one or two publications providing the following contributions.

Part I: Analysis of secure hardware ecosystems

1. In Chapter 2, we study the properties and security of Trusted Platform Modules using certification documents and real-world testing. We developed a tool for collecting capabilities, performance, and cryptographic material and used it to obtain

samples from devices with 78 unique firmware versions. Our analysis revealed unreported leakages in signature algorithms. The most severe discovered leakage allows for extracting the private key of certain Intel's TPM versions using only nine signatures. The suitability of current TPMs for the computation of advanced cryptographic protocols was evaluated. Commands, algorithms, and parameters crucial for constructing TPM-compatible protocols, including anonymous attestation [18], asymmetric decryption [18], and basic multi-signature protocols [13, 14], are commonly supported. Nonetheless, the current interface defined in the TPM specification provides few options for implementing threshold protocols beyond these relatively simple two-round constructions.

2. In Chapter 3, we survey the capabilities and usage trends in the ecosystem of JavaCard-based smartcards using tools we developed. We collected data from security certification documents, real smartcards, and open-source projects. Among other insights, our analysis revealed slow adoption of new JavaCard features by open-source implementations, with almost 70% of constants defined in the specification being unused in open-source code. Data we collected from real smartcards confirmed the lag of algorithm support. However, it has also shown that recent smartcard models started supporting algorithms introduced in JavaCard API 3.0.5 [19] with good performance, enabling practical implementation of more complex elliptic-curve-based protocols.

Part II: Threshold cryptography on secure hardware devices

3. In Chapter 4, we design new approaches for repurposing high-level cryptographic API provided by the JavaCard platform to accelerate operations used in threshold protocols. The approaches include both algorithmic improvements and improvements due to the utilization of more recent JavaCard algorithms, allowing significant performance increases compared to previous works. We integrate these improvements into an open-source library for cryptographic computation on JavaCard-

based smartcards [15]. The library's performance and the extent of its support are evaluated on six smartcard models.

4. In Chapter 5, we study state-of-the-art threshold Schnorr signature protocols, focusing on their suitability for smartcard computation. First, we provide a standard-compliant implementation of the FROST protocol [16, 17] with smartcard-specific optimizations. The protocol is efficiently computable on the latest smartcards but notably slower on older models without good elliptic-curve algorithm support. Next, we propose the notion of threshold protocol interoperability, aiming to increase efficiency and resiliency in heterogeneous environments. We analyze options for interoperability across threshold Schnorr signature protocols and propose an untrusted mediator to facilitate it among certain protocol classes. Finally, based on our findings, we design a new protocol optimized for resource-constrained devices and interoperable with prevalent threshold Schnorr signature protocols. We evaluate the protocol's performance and analyze its security by reducing it to the one-more discrete logarithm problem in the random oracle model.
5. In Chapter 6, we study protocols for computing two-party ECDSA signatures and propose three approaches for their computation on current smartcards with different trade-offs. The first approach utilizes Lindell's two-party protocol [20], accelerated with modular exponentiation co-processor but still requiring a few seconds of computation. The remaining two approaches utilize the preprocessing model with an optional trusted preprocessing party. We implement all three approaches for the JavaCard platform while considering the hardware constraints and evaluate their performance on a smartcard.
6. In Chapter 7, we propose a protocol for distributing a blind BLS signature variant used in e-cash in a way suitable for computation on smartcards. We analyze its security by constructing a reduction to the one-more gap Diffie-Hellman problem in the random oracle model. We implement the protocol, including a novel approach to secure offloading of the used hash-to-curve function, and evaluate its performance on a smartcard.

PART I

**ANALYSIS OF SECURE HARDWARE
ECOSYSTEMS**

Overview

This part of the thesis addresses the first research goal – the analysis of capabilities, performance, and security of the two largest ecosystems of security-certified devices. Chapter 2 focuses on Trusted Platform Modules, and Chapter 3 focuses on cryptographic smartcards based on the JavaCard platform.

Each of the chapters begins with an analysis of the ecosystem’s state from the perspective of Common Criteria [11] and FIPS 140 [12] certification documents, yielding insights into trends in supported platform versions and cryptographic algorithms. Then, we shift focus to real devices and design tools for the collection of their characteristics, which were used for community-sourced data collections. The collected data is analyzed, revealing new deficiencies in implementations of elliptic-curve-based algorithms (Section 2.5) on TPMs and extending prior research findings [4]. Similar vulnerabilities in smartcard implementations were published earlier [5]. These findings highlight that while current certification procedures increase trust in a device’s security, they provide no guarantees. Consequently, additional precautions, such as distributed computation using threshold cryptography, should be considered in high-security environments.

Next, we summarized the scope of supported cryptographic operations provided by the interfaces and their performance. Our findings suggest that although the underlying hardware is capable of computing the relevant cryptographic operations (modular arithmetic and operations with points on elliptic curves) with good performance, the interfaces the platforms provide are very constrained and do not facilitate direct usage of the operations relevant for constructions of threshold protocols. This is particularly limiting in the case of TPMs, where only relatively simple two-round protocols are realizable. Smartcards are more flexible due to their general programmability, and the interface they provide may be exploited to partially accelerate relevant operations.

2 Trusted Platform Modules

This chapter is based on our publication presented at the CHES conference [A1].

- [A1] P. Švenda, A. Dufka, M. Brož, R. Lacko, T. Jaroš, D. Zaťovič, and J. Pospíšil, “TPMScan: A wide-scale study of security-relevant properties of TPM 2.0 chips,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2024, no. 2, pp. 714–734, 2024. DOI: 10.46586/tches.v2024.i2.714–734.
-

2.1 Introduction

The Trusted Platform Module (TPM) is a widely adopted secure hardware component integrated into most modern computer systems. It is defined by an ISO standard [21] and regularly revised by the Trusted Computing Group [22]. TPMs are designed to increase system security by providing means for executing hardware-isolated cryptographic operations with protected key material. Additionally, its tight integration with devices’ hardware facilitates the implementation of unique features like secure boot and remote attestation [23].

Five different types of TPMs are deployed today [24], all complying with some revision of the TPM 2.0 specification [22]. Discrete TPMs (dTPMs) are implemented as standalone hardware modules, placed in a socket, with some level of anti-tampering measures. Firmware TPMs (fTPMs) are implemented in the CPU’s firmware, usually running on one of the CPU cores. Integrated TPMs (iTPMs) are implemented as dedicated hardware integrated into the CPU package – they are the middle ground between dTPMs and fTPMs. The remaining two types, TPM simulators and virtualized TPMs, are not studied in this work.

Despite the wide prevalence of TPMs and their frequent evaluation by thorough and expensive security certification under the Common Criteria scheme aimed typically at the EAL4+ level, serious security vulnerabilities enabling private key extraction have been discovered

and reported [3, 4]. The impact of these vulnerabilities could be mitigated by distributing the computation, e.g., via threshold computation with the host device. However, the extent to which such computation is feasible with existing TPM interfaces and the actual support of relevant features in TPMs needs further investigation.

In this chapter, we aim to study the TPM ecosystem with two primary goals. The first goal aims to assess the security and the level of vulnerability mitigation in existing TPMs by broadening the scope of previous TPM analyses. The second goal aims to evaluate the capabilities and performance of currently deployed TPMs for the computation of threshold cryptography protocols.

We studied information contained in public sources, including security certificates, CVE records, and security advisories, yielding insights about vulnerability mitigation and trends in TPM deployment. However, our most valuable information source was the data collection from real devices consisting of 78 unique TPM firmware samples, performed using tools we developed. Its analysis not only confirmed previously discovered vulnerabilities in additional algorithms and firmware versions, but it also led to the discovery of a new severe vulnerability in certain Intel firmware, allowing private key extraction using just nine signatures. The analysis of devices' capabilities revealed the support for the execution of simpler two-round protocols on most devices and the increasing adoption of newer algorithms and larger key sizes, but it also highlighted the limitations of the current API.

The data collection and processing tools with a database of all collected results are available under a permissive license¹. We sincerely thank all people who contributed to the collection. The new vulnerabilities were responsibly disclosed to the corresponding vendors.

The rest of this chapter is organized as follows. The development of the TPM platform and related work is discussed in Section 2.2. Insight into CC and FIPS certification artifacts of TPM products is provided in Section 2.3. Section 2.4 describes the TPM data collection approach and the resulting dataset. Sections 2.5 and 2.6 analyze cryptographic data collected from ECC and RSA implementations, respectively. Algorithm support and performance are surveyed in Section 2.7. The conclusions follow in Section 2.8.

1. https://crocs.fi.muni.cz/papers/tpm_ches2024.

2.2 Related work

The first specification describing Trusted Platform Modules was published in 2001 by the Trusted Computing Platform Alliance (TCPA) [25]. The Trusted Computing Group (TCG), formed from the TCPA in 2003, standardized the successor of this specification in 2009 as the TPM 1.2 [21]. TPM 1.2 specification offered a one-size-fits-all solution with a single hierarchy, single root key, a limited set of cryptographic algorithms (only SHA1, RSA, HMAC, and MGF1 being mandatory), and very limited options for advanced applications. This version targeted computers and servers and was unsuitable for more constrained environments like mobile computing, IoT, or industrial applications. For this reason, TCG released the TPM 2.0 specification [26], with platform-specific requirements defined for different use cases.

With the TPM 2.0 specification, TPMs started being usable also for a wider range of applications. Chen and Urien [27] have summarized the interface provided by the TPM 2.0 specification that can be used for implementing advanced protocols with TPMs. This interface can be used to construct protocols that perform operations with the secret key stored securely within the TPM but rely on the host for privacy and other protocol properties. A number of remote attestation protocols with better privacy properties have been proposed [18, 28, 29]. Chen and Urien [27] have also shown how four ECC-based asymmetric decryption algorithms can be implemented with TPMs: ECIES, PSEC, ACE, and FACE. Multi-party signature protocols have been implemented on TPMs for BLS signatures [13] and even for a restricted variant of Schnorr signatures [14].

The TPM's wide adoption and critical use case as a root of trust created a need for formal certification. The TCG maintains a certification program for TPMs based on Common Criteria [30, 31]. Still, many devices from the certified list [32], including CC EAL4+ certified ones, were found vulnerable to various exploits, including flaws in the construction of primes for RSA keys (the ROCA attack), leading to a significant loss of entropy [3] or allowing for inferring their origin [33]. Other works [5, 34] exploited noisy leakage of bit-length of ECDSA nonces to extract the ECC private key. The challenges that the CC certification faced were expressed already around 2004 by Hearn [35] and eight years later by Murdoch et al. [36], who discuss why both CC

and FIPS 140 fell short of their promise, pointing to the lack of transparency. Beckert et al. [37] highlight that formal verification of only the specification, but not the implementation, is required to achieve higher EALs. Many of the schemes' problems are also mentioned in a user study of 29 users trained in the NIST cryptographic validation program [38]. A study by Kaluvuri et al. [39] performs a data-driven analysis of EAL4+ ICs and smartcard CC certificates. At the time of their study, no TPM 2.0 device was certified.

Many other attacks on TPMs have been published. The attacks are most commonly targeted at dTPMs and fTPMs. The dTPMs are commonly vulnerable to physical attacks on their exposed data bus, such as sniffing or MITM attacks. Inserting an intercepting device between the socket and dTPM allows an attacker to hijack communication packets, forge the TPM's RNG output, and attack PCR extensions used in attestation, secure boot, and sealing [40]. The fTPMs and iTPMs are implemented either in the CPU's firmware or as dedicated hardware integrated into the CPU package. As they do not have any physically exposed pins or communication interfaces, attacks on them focus mostly on side channels, fault injection, and firmware exploits. FaultTPM [41] is a voltage-glitching fault injection attack compromising the entire internal state of AMD Zen 2/3 fTPMs. TPM-Fail [4] is a private key recovery attack using leakage of secret-dependent execution times during the generation of signatures based on elliptic curves, affecting certain Intel fTPMs and STM dTPMs certified up to the CC EAL4+ security level.

2.3 Common Criteria and FIPS certification

As TPMs are often used in environments imposing strict security requirements, they may need to undergo a security certification process to comply with existing regulations. To shed more light on the current state of the TPM ecosystem from the perspective of security certifications, we analyzed public certification documents released under Common Criteria [11] and FIPS 140 [12] certification schemes using the *sec-certs* project [42].

To perform the analysis, we first downloaded all public CC and FIPS certification documents. As certification documents are not de-

signed for automated processing, the source PDFs were first converted to text files. Subsequently, we filtered documents that mentioned TPM in its title or multiple times in the document’s body to avoid false positives². Furthermore, we had to manually filter out some documents that were coincidentally using the TPM abbreviation for a different technology or only included TPM as a component.

For each matched TPM certificate, we identified the TPM specification version (1.2 or 2.0) and the revision it complies with. We did this by matching regular expressions of version numbers against the text content of the documents, and in case we did not find enough clear matches in some certificates, we manually annotated them. For the rest of the analysis, we used only TPM 2.0 certificates. We identified 85 such certification documents (58 Common Criteria and 27 NIST FIPS 140) by four vendors. All CC certificates were issued at EAL4+; FIPS certificates mostly targeted security level 2.

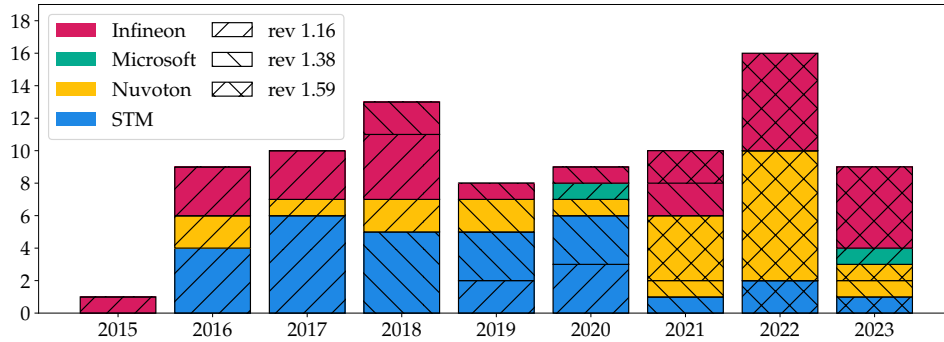


Figure 2.1: Number of TPM 2.0 certificates issued to vendors by year. The specification revision the certified TPM complies with is shown with a bar pattern.

Figure 2.1 shows that TPM 2.0 devices started being certified in 2015, with the first certified TPM by Infineon, who, to this day, published 30 TPM 2.0 certificates. STM had its first TPM 2.0 chip certified a year later, but since then, they have also obtained 30 TPM 2.0 certificates. Nuvoton has 23 certified TPM 2.0 chips. Lastly, Microsoft has two virtual TPMs certified under the FIPS scheme. Neither Intel and

2. Some documents only mention TPM technology as a side note or in references.

AMD fTPMs nor Pluton-based iTPMs are certified under any of the schemes by October 2023.

The first TPMs certified with revision 1.38 started appearing in 2018, two years after the revision had been published. Revision 1.59 started appearing in 2021, again, two years after its publication. We did not encounter any certificate document mentioning versions of the TPM 2.0 specification lower than 1.16.

The documents of selected TPMs were inspected for information on changes in cryptographic libraries with respect to discovered vulnerabilities in earlier versions. However, we did not find this information present in any of the public documents.

2.4 Data collection from real devices

We developed TPM analysis tools for Linux (`tpm2-algtest`) and Windows (`tpm_pcr`) operating systems and collected TPM data from three principal sources: university-managed machines, a software-compatibility testing cluster, and end-user machines of volunteers. While we also collected some results for legacy TPM 1.2 chips, we focused primarily on version 2.0.

The simpler `tpm_pcr` tool collects only PCR register values³, system information, and Endorsement and Storage Root Keys on Windows OS, with its implementation being based on Microsoft’s PCPTool⁴. The data collection typically takes around 10 seconds and can be scheduled to run at regular intervals. While allowing for easy and quick data gathering by volunteers, the collected properties are limited.

The `tpm2-algtest` tool collects a wide range of characteristics of TPMs on Linux OS. The implementation relies on the `tpm2-tools` library⁵ to communicate with the TPM via Linux kernel drivers. The tool is also available in the form of a Fedora-based bootable image with a custom graphical interface for controlled data collection on any computer, independently of the installed OS. The tool’s runtime depends heavily on the number of measured operations and the speed

3. Memory registers with a hash chain of configuration data and binaries executed.

4. <https://github.com/microsoft/TSS.MSR/tree/master/PCPTool.v11>.

5. <https://github.com/tpm2-software/tpm2-tools>.

of the target TPM chip, taking between tens of minutes (fTPMs) up to ten hours (dTPMs) in the default configuration.

The tool collects public keys of freshly generated RSA and ECC key pairs for all relevant algorithms and key lengths, and the corresponding private keys. The key pairs are created using the `TPM2_Create` command, which outputs the public key in plain and the private key encrypted. To extract the private key, we generated all key pairs with a policy allowing for duplication that enables the key to be used with the `TPM2_Duplicate` command. When this command is called on the generated key with *newParentHandle* and *symmetricAlg* parameters set to `TPM2_ALG_NULL`, the encryption is disabled, and the private key is also output in plain. A summary of collected properties is listed in Table 2.1.

Table 2.1: TPM properties collected by `tpm2-algtest`. The number of samples listed is per single collection run and can be increased by configuration option or by repeated runs.

Persistent properties	#	Temporal properties	#
System info	—	PCR ₀ –PCR ₂₃ values	—
TPM capabilities	—	RSA & ECC keys	1000x
Algorithm performance	1000x	RSA & ECC signatures	1000x
EK / SRK	2B+2B	RNG output	512kB

As `tpm2-algtest` has been developed in parallel with the collection of data samples, some collected data may be missing for some TPM firmware. In particular, ECC keys and operation performance are collected for 45 firmware versions since the functionality was included only in 2022, and we could not re-run the data collection for a small number of older, community-provided results.

We focus on the analysis of the following TPM-related properties:

- (i) persistent properties, like algorithm support and operations performance and data timing dependency of different TPM chips (different vendor and/or firmware version),
- (ii) variations between physical chips of the same type (same vendor and firmware version), caused, e.g., by infrequent malfunctioning or manufacturing variability,

2. TRUSTED PLATFORM MODULES

- (iii) observed changes caused by a different firmware version (same vendor but different firmware versions, possibly also different underlying TPM hardware),
- (iv) properties of TPM-unique Endorsement Keys and Storage Root Keys,
- (v) properties of cryptographic material produced by target TPM like generated keys, signatures, and random data.

All the operations executed and measured during data collection are within the temporary, transient key hierarchy, which is erased during any reboot by the TPM itself. We do not perform any firmware updates. We do not collect personally identifiable data, attestation keys, or user-specific content of the non-volatile TPM memory (NVRAM). Only the first and the last two bytes of an EK and SRK are collected.

Dataset of 78 unique TPM firmware samples

The dataset was collected over the period of seven years and contains 78 unique firmware produced by 6 different vendors: Infineon (IFX), Nuvoton (NTC), STMicroelectronics (STM), Advanced Micro Devices (AMD), Microsoft (MSFT), and Intel (INTC). A summary of collected firmware versions is shown in Figure 2.2, with the most recent versions released till the middle of 2023, including Pluton-based iTPMs. Only a few versions certified in 2023 (STM, IFX) are missing, as they are not yet available on end-user platforms. We executed two primary data collection scenarios:

1. **Many computers with different TPM chips** (different vendor or version), collected with `tpm2-algtest` tool on more than two hundred computers mostly by volunteers and on the testing cluster. This case examines properties (i), (iii), (iv), and (v).
2. **Many computers with the same TPM chip** (same vendor and version). This scenario primarily examines property (ii). Properties (i) and (v) are also addressed with orders of magnitude more data, but only for two specific TPM chips (IFX 5.63.13.6400 and NTC 7.2.2.0). Property (iv) was also analyzed as different TPM chips from the same vendor and firmware version should

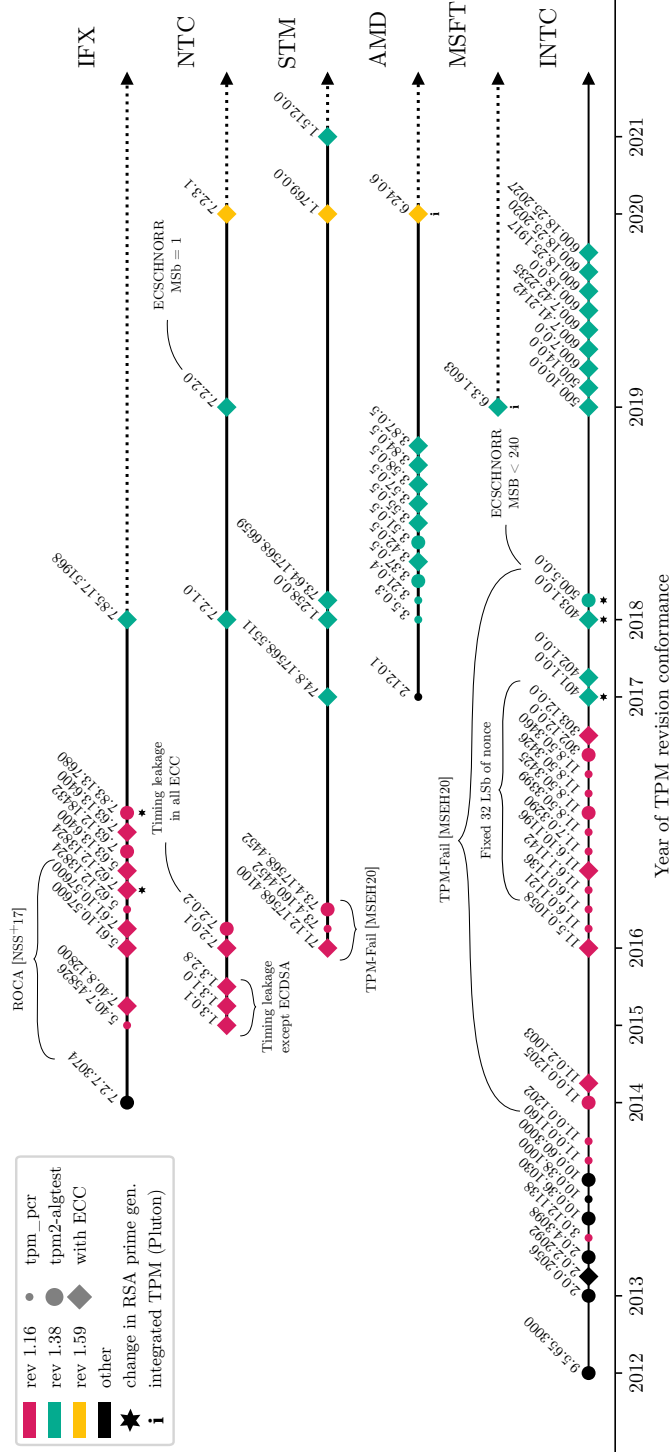


Figure 2.2: The collected dataset annotated with changes observed in cryptographic implementations, specification revisions, and with detected vulnerabilities. The listed year is of the errata the firmware conforms to, not necessarily the year of the firmware release, and is typically 1-2 years behind a release of the actual platform with such a TPM chip. The year value for measurements from tpm_pcr was interpolated from related firmware versions. Vulnerabilities in ECC implementations were verified only on samples with ECC data collected. Vulnerabilities without a citation are newly found ones.

still have unique EKs. Data was collected with `tpm2-algtest` and `tpm_pcr` tools on 248 managed university computers and more than a hundred of computers from volunteers.

The majority of TPM firmware in our dataset claims compliance with revision 1.16 (35x), followed by 31 samples of the firmware of revision 1.38, and lastly, three firmware samples of the 1.59 revision, originally published in 2019. The TCG website provides a specification of six TPM 2.0 revisions: 0.96, 0.99, 1.16, 1.38, 1.59, and 1.83, but we have also encountered some TPMs declaring compliance with revisions that we could not find in public sources: 0.93 (1x), 1.03 (6x), 1.15 (1x), and 1.21 (1x).

Note that as volunteers contributed some of the measurements, we do not always have full control over the measurement platform, which may exhibit higher variability of measurements due to other tasks executed on the platform. We cross-verified results for the same TPM version where multiple measurements were available, and we did not observe any “unexplainable” deviations in the presented results. The sanitized dataset, atop the tooling, is openly released to facilitate reproducibility and further research⁶.

2.5 Analysis of ECC implementations

A detectable bias in ECC private key or signature nonce can decrease its security level and, in some cases, even lead to full private key recovery. Such an attack on TPMs has been previously demonstrated by the authors of TPMFail [4], which has exploited a timing side-channel present in scalar multiplication in several Intel and STM TPMs.

To identify this class of problems, we collected at least 1000 key pairs and signatures together with the time required to generate them for all supported combinations of elliptic curves and signature algorithms on 45 unique TPM firmware versions. As we collected private keys corresponding to the issued signatures, we were able to extract nonces used by the signing algorithm. All this information was subsequently subjected to randomness analysis to identify static biases, and timing analysis to look for non-constant behavior.

6. Supplementary materials, database of results, and open tools are available at https://crocs.fi.muni.cz/papers/tpm_ches2024.

The nonce extraction formulas were derived from the signing algorithm used to create the corresponding signature [26]:

$$\begin{array}{ll} \text{ECDSA: } s^{-1}(e + rx) \mod n & \text{ECSCHNORR: } s - rx \mod n \\ \text{SM2: } s + sx + rx \mod n & \text{ECDAA}^7: s - H(r||e)x \mod n \end{array}$$

where (r, s) are signature components output by the signing algorithm, x is the private key, e is the signed challenge, n is the order of the used elliptic curve, and H is the used hash function.

To ensure that we used the correct extraction approach, we first verified each signature with the corresponding verification algorithm. As it turned out, not all TPM implementations comply with the specifications they claim, which we had to reflect during nonce extraction. The ECSCHNORR computation has changed⁸ in revision 1.33, and the ECDAA computation in revisions 1.35 and 1.36, but some of these changes were already included in TPMs still claiming compliance with an earlier revision 1.16.

In particular, INTC 303.12.0.0 implements the new ECSCHNORR and ECDAA algorithms, IFX 7.63.13.6400 and 5.63.16.6400 implement the new ECDAA, and NTC 7.2.0.1 and 7.2.0.2 implement the new ECSCHNORR algorithm. Additionally, we discovered that NTC 7.2.1.0 and 7.2.2.0 do not seem to produce valid ECDAA signatures according to any published revision⁹.

2.5.1 ECC secret's randomness

Typical randomness testing tools like NIST STS [43], Dieharder [44], and TestU01 [45] are primarily designed for analyzing large volumes of data, but they do not work well with limited data sources like ours. Therefore, to identify static biases present in the generated keys and nonces, we used the `booltest` tool [46], which is more suitable for

7. This formula was used only for ECDAA from TPMs complying with the specification revision 1.36 or higher; otherwise, the ECSCHNORR approach was used as it corresponds to the original computation.

8. This change does not affect the nonce extraction of ECSCHNORR signatures, as the difference is only present in the challenge computation, and that value is directly output by the signing algorithm as r .

9. It might be compliant with ECDAA of revision 1.35, but we did not find it published.

2. TRUSTED PLATFORM MODULES

performing statistical tests on a low amount of data. The tool was run on concatenations of all generated keys and nonces¹⁰ in the default configuration, in which we only adjusted the block size to correspond to the bit-length of analyzed keys and nonces.

No statistically significant deviations were found in private keys, which was expected, as the algorithm for ECC key generation is clearly defined in the TPM specification [26], and any observable deviation would imply diverging from the specified key generation. However, the analysis tool pinpointed biases in nonces generated by a number of Intel firmware and one Nuvoton firmware. Upon further inspection, we discovered the following three new problems:

1. The lowest four bytes of nonces of ECDSA and ECSCHNORR algorithms were set to 0x00000001 (INTC 11.5.0.1058, 11.6.10.1196, and 303.12.0.0).
2. The most significant byte of ECSCHNORR nonce never exceeded 240 (INTC 403.1.0.0).
3. The most significant bit of ECSCHNORR nonce was always set to 1 (NTC 7.2.2.0).

The first problem is the most severe and leads to an easily exploitable key extraction vulnerability. It sets 32 least significant bits to a fixed value, which leaks a lot of information about the private key when used in a signature computation. By following approaches used in earlier works [4, 5], we implemented an attack on this type of information leakage utilizing the LLL algorithm [47] to solve the hidden number problem [48]. In our experiments, we were able to reconstruct the private key using *only* 9 signatures with no need for active observation of the signing process. For comparison, the TPM-Fail attack [4] needs *active* observation of the creation of around 1300 signatures. Coincidentally, this issue is present on TPM firmware versions that also suffered from the TPM-Fail vulnerability [4] and was fixed after version 403.1.0.0. Still, this attack can have severe consequences for systems with outdated firmware or systems that created signatures with vulnerable versions in the past, as mere ex-post availability of

10. We always kept keys and nonces from different curves and algorithms separated.

9 signatures leads to the reconstruction of the used private key and subsequent signature forgery.

The other two issues leak only one or fewer bits of information, which is too little to practically attack schemes on 256-bit curves with published approaches and a realistically small number of obtained signatures. Still, the presence of such biases signifies a lack of implementation testing and should have been discovered at an earlier stage or at least during certification evaluation (Nuvoton’s ANSSI-CC-2020/21 certificate for 7.2.2.0).

2.5.2 ECC operation timing

As the typical ECC timing side-channel attacks [4, 5] utilize the time dependency of scalar multiplication on the significant bit-length of the used scalar, we plotted the computation time of key generation and signing against the most significant byte of a given private key and nonce, respectively.

The plots (Figure 2.3 shows selected samples) revealed an observable time dependency in implementations of three TPM vendors: Intel, STM, and Nuvoton. For the former two, this was a mere confirmation of the previous work [4], but for Nuvoton TPMs, this vulnerability has been reported only partially.

The dependency was observable for all key generation and signing configurations for Intel and STM firmware¹¹, which was expected, as the algorithms are likely to be using the same scalar multiplication implementation. However, it was not the case for Nuvoton firmware 1.3.0.1, 1.3.1.0, and 1.3.2.8, where ECDSA did not exhibit the time dependency, while ECDAA, ECSCHNORR, and key generation algorithms did.

Therefore, we presume that Nuvoton’s ECDSA implementation was using a different, constant-time implementation of scalar multiplication. These are new findings that have not been reported in previous research or security bulletins.

11. We detected the leakage in Intel firmware 11.0.2.1003, 11.5.0.1058, 11.6.10.1196, 303.12.0.0, 401.1.0.0, 402.1.0.0, 403.1.0.0, and STM firmware 71.12.17568.4100.

2. TRUSTED PLATFORM MODULES

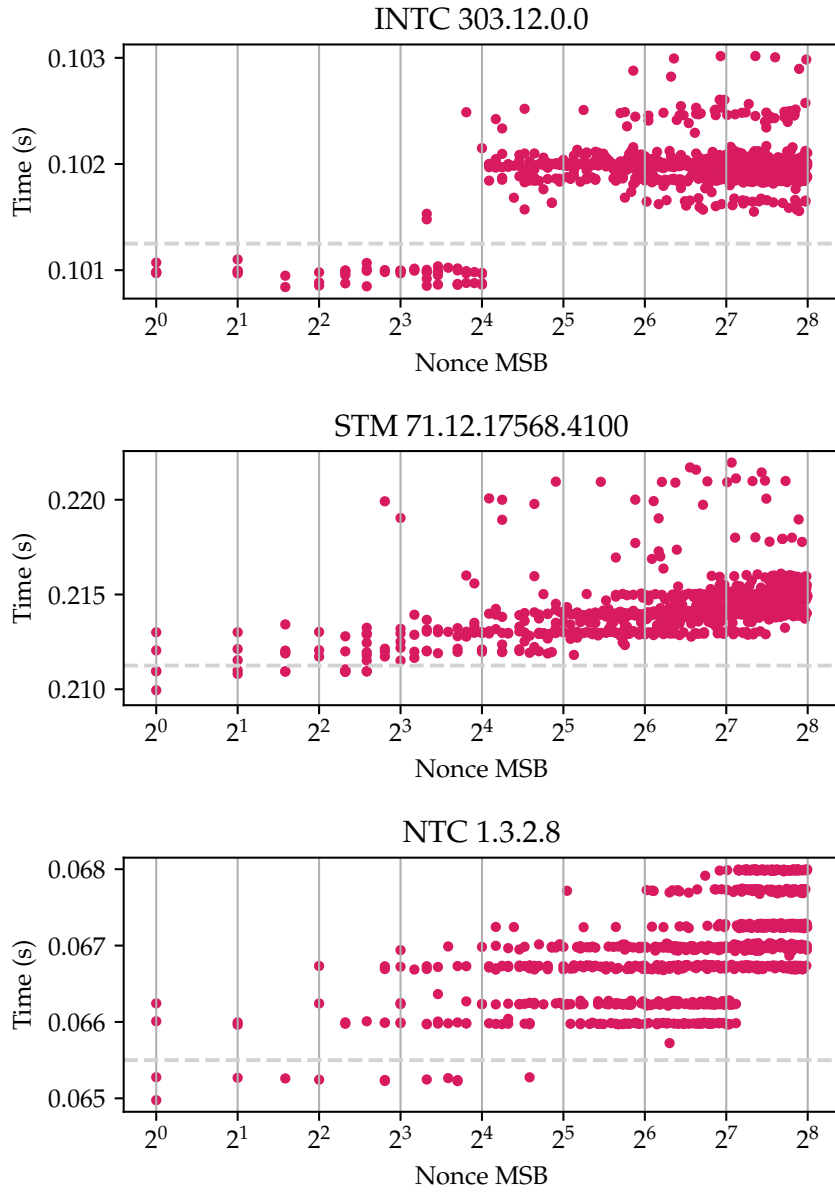


Figure 2.3: The signing time dependency on the most significant byte of nonce using the ECDSA algorithm on the BN_P256 curve on TPM firmware by Intel, STM, and Nuvoton. Signing time reveals nonces' topmost bits (e.g., zero bits in samples below the dashed line). This information can be exploited to recover the private key.

Furthermore, the Nuvoton firmware 7.2.0.1 had been reported to have an observable timing dependency only in ECDSA signing¹², yet we noticed the same dependency also in other algorithms (key generation, ECSCHNORR, and ECDAA). We have observed this only with computations on the NIST_P384 curve, but it is likely also present on 256-bit curves, only below the measurement resolution we obtained for that particular sample.

2.6 Analysis of RSA implementations

We analyze the outputs of RSA implementations to discover observable changes in TPM cryptographic libraries and the origin of Endorsement Keys for specific TPM version. We utilize fingerprinting techniques developed in previous works [33, 49]. Although both works used hundreds of thousands or even millions of keys, 1000 keys are typically sufficient to identify one out of previously detected distributions. If multiple measurements for the same TPM chip version are available, we aggregate the collected keys.

Janovský et al. [49] identified four *features* extracted from private keys and used to distinguish between different implementations:

1. the most significant bytes of both p and q (influenced by different strategies to maintain modulus of expected length),
2. existence of small factors between 5 and 17863 in $p - 1$ and $q - 1$,
3. second least significant bit of both $p - 1$ and $q - 1$ indicating Blum integers (together with the second feature) believed to be slowing older factorization methods),
4. ROCA-specific fingerprint in public key (Infineon's RSA prime generation algorithm vulnerable to practical factorization [3]).

For every batch of keys from a given TPM firmware version, we computed the fingerprinting properties and observed if subsequent TPM versions exhibit the same characteristics or if any difference is detected, signaling a change in the underlying RSA key generation implementation.

12. <https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2020-25082>.

2.6.1 Detected changes in key generation

Figure 2.4 visualizes the detected changes in RSA keys generated by different TPMs grouped by their manufacturers. The figures plot the most significant byte of both RSA moduli factors, which corresponds to the first feature. The ROCA fingerprint is also visible in these plots for Infineon TPMs prior to versions 5/7.61.10.57600. The remaining two features did not contribute to detecting changes in key generation. We did not encounter any TPM 2.0 implementation avoiding smaller factors in $p - 1$ or $q - 1$ or generating Blum integers consistently.

For Intel fTPMs (Figure 2.4a), four different ranges are distinguishable, following the evolution of Intel’s CPU. The initial implementation, used in 5th gen Braswell, 6th gen Skylake and 7th gen Kaby Lake (2015-2017, 2-303.x), was changed twice in the 8th gen Coffee/Whiskey Lake (2017, 401/402.x) and (2018, 403.x), finally ending with observably the same RSA implementations from the 10th gen Comet Lake (2020, 500.x) to the newest 11-13th gens Tiger, Alden and Raptor Lake (2020-2022, 600.x). No version from 9th gen Coffee Lake Refresh was observed, possibly being the same as the 8th gen Coffee Lake (403.x).

For Infineon dTPMs (Figure 2.4c), three different ranges are distinguishable: versions containing ROCA vulnerability and certified till 2016 (5/7.61.10.57600), then versions till 2018 (7.63.x, likely only with ROCA-related RSA key generation fix), and the most recent range staying the same since 2018 (7.83.x). Infineon additionally certified a range of versions for 1.59 TPM revision during the year 2023, which are so far missing from our analysis due to their unavailability.

No observable changes in RSA keys properties were detected during each vendor’s timeline for AMD fTPMs (Ryzen 4000 and 5000 series), AMD iTPM (Ryzen 6000, Pluton-based), Microsoft iTPM (Ryzen 6000, Pluton-based), Nuvoton dTPMs, and STM dTPMs (visualized in Figures 2.4b and 2.4d).

Due to a potentially incomplete list of versions available and analyzed, the range boundaries may be imprecise – the interval may extend to other numerically close but unavailable versions. Similarly, the detected interval may also possibly split into more intervals if the implementation would change and revert in a subset of unavailable firmware versions.

2.6.2 Detection of off-chip EK generation

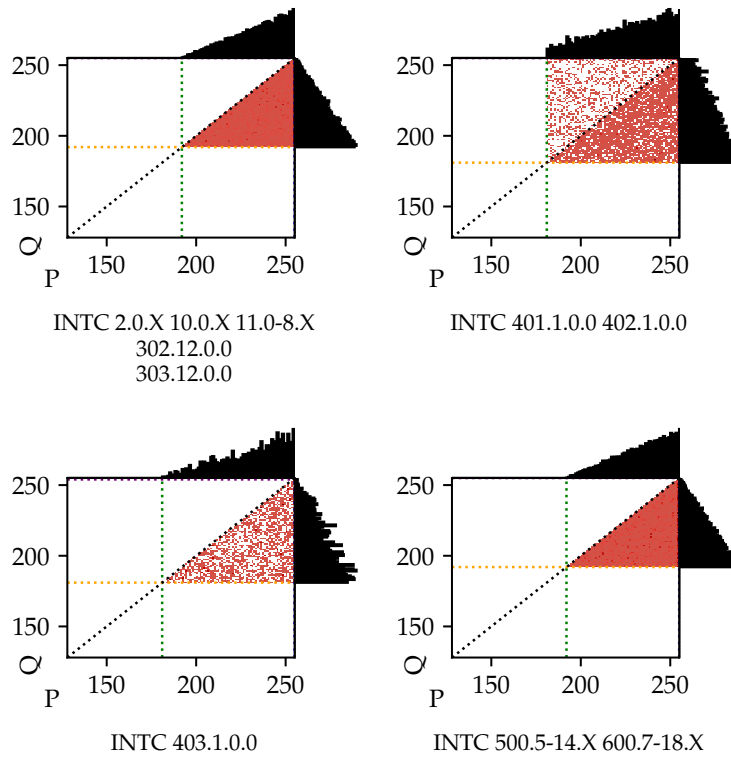
We assess if Endorsement Keys (EKs) were generated directly on a TPM chip or first generated outside the TPM (e.g., in a manufacturer's HSM) and only then injected into the TPM during the manufacturing procedure. Initial versions of TPM specification [21] mandated on-chip EK generation, but later revisions relaxed this requirement and allowed for injection from outside environment to accelerate the manufacturing process. Although off-chip generation is now allowed, it increases the risk of EK compromise.

At present, 2048-bit RSA keys are the most commonly used type of EKs. The RSA key fingerprinting methods [33, 49] provided means for assessing if a given key is (likely) generated by a given cryptographic implementation using an observable difference in the generated key resulting from different implementation choices of the prime generation algorithm. We utilize this approach, but with several limitations when applied to the assessment of EKs.

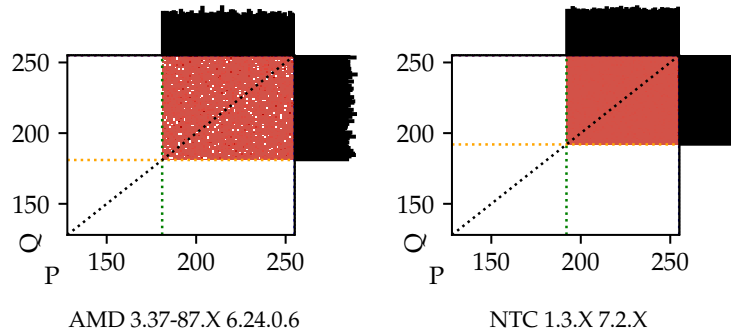
As only the public part of EK is exportable from TPM, we cannot utilize the more precise fingerprinting method [49], which requires a private key. Additionally, due to privacy reasons, we do not collect a whole EK in community-provided contributions, as it would serve as a unique identifier of a given machine. Finally, we typically have only one or very few EKs for every TPM version tested and not the large numbers used in the original works. Therefore, we try to detect EK injection using only the most significant byte (MSB) of the modulus of EK(s) compared to the expected distribution of on-chip generated keys. Additionally, we test for the presence of ROCA [3] fingerprint. We mark an EK as *injected* if its MSB value was never observed in RSA keys generated on-chip. Figure 2.5 visualizes the observed MSB distributions for different TPM firmware and shows the corresponding values of collected EKs, highlighting injected ones.

Another option is to detect improbable values of MSBs by statistically testing the match between the distribution of MSBs extracted from EKs and MSBs observed in on-chip generated keys. However, such an option requires a reasonably high number of Endorsement Keys (at least higher tens) to have high confidence in the result obtained. As we do not possess such a number of EKs (with the exception of IFX 5.63.13.6400 and NTC 7.2.2.0), we do not use this method.

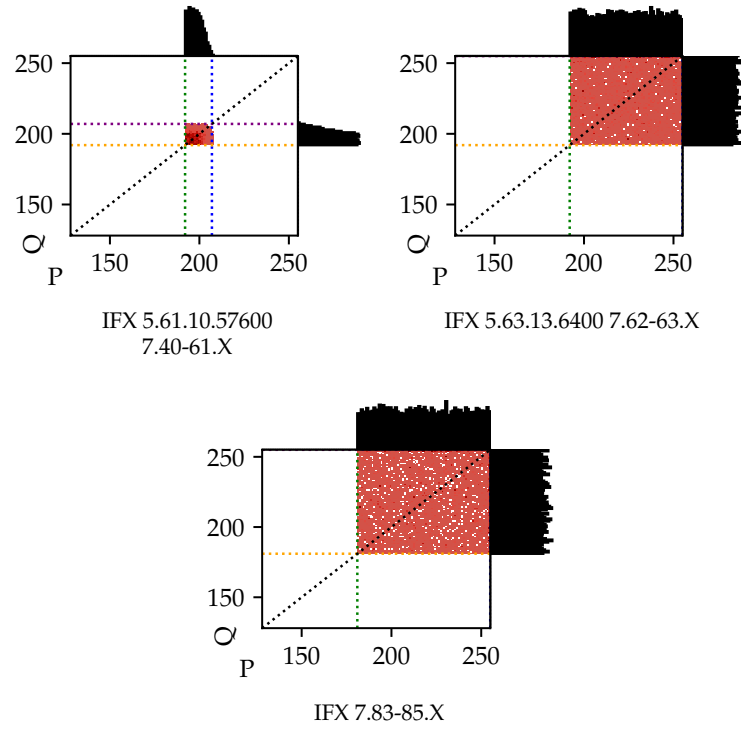
2. TRUSTED PLATFORM MODULES



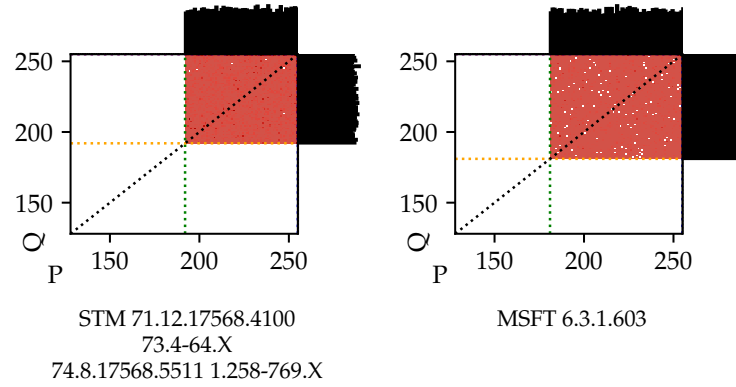
(a) Intel TPMs 500/600.x are the same as the initial versions till 303.12.x.



(b) No changes in AMD and NTC TPMs were observed.



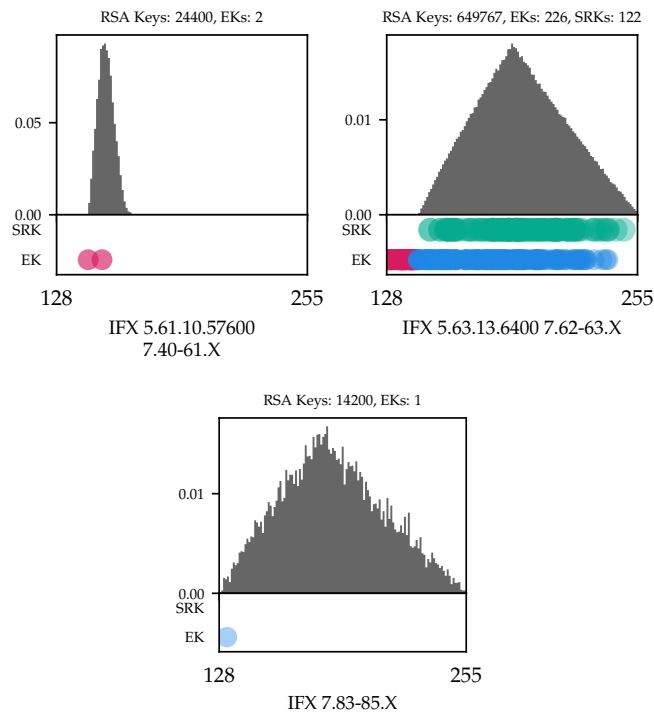
(c) Infineon TPMs till 5/7.61.x contain ROCA vulnerability.



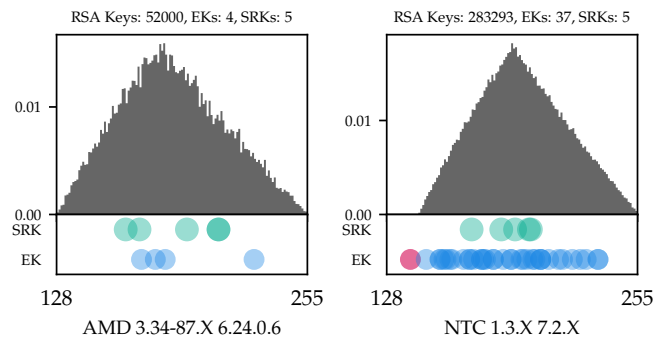
(d) No changes in STM and Microsoft TPMs were observed.

Figure 2.4: Differences in RSA moduli across firmware versions. The figure shows the MSB of RSA factors from given firmware, complemented by histograms of both primes' MSBs.

2. TRUSTED PLATFORM MODULES

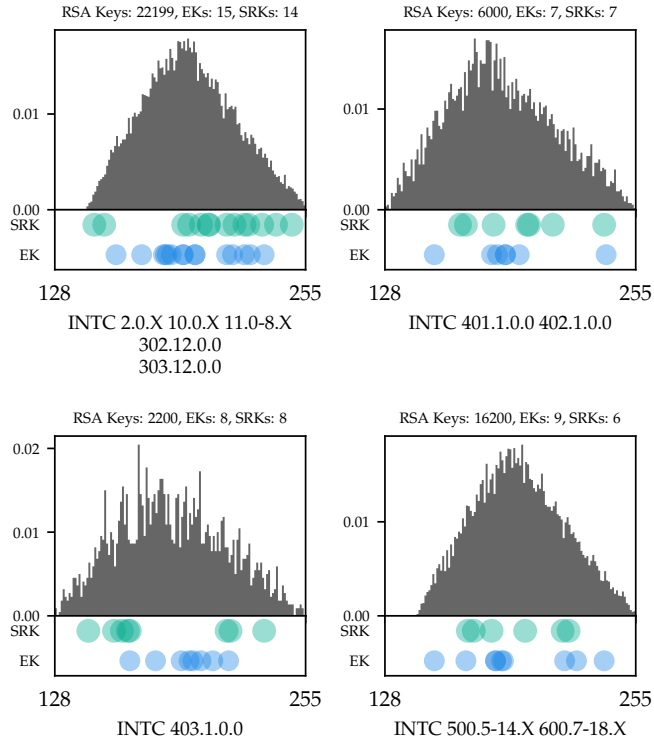


(a) Infineon 5.61.x/7.40-61.x generate keys with ROCA fingerprint, but EKs do not exhibit it.

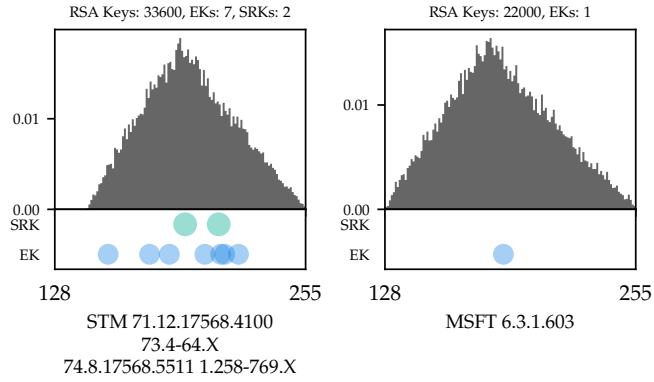


(b) AMD TPMs show no signs of injected EKs. One injected EK was detected in a Nuvoton TPM.

2. TRUSTED PLATFORM MODULES



(c) Intel TPMs show no signs of injected EKs.



(d) STM and Microsoft TPMs show no signs of injected EKs.

Figure 2.5: Distribution of MSB of Endorsement Keys (blue) and Storage Root Keys (green) over a histogram of on-chip generated keys. Externally injected Endorsement Keys are shown in red.

Externally injected Endorsement Keys

For AMD and Intel firmware TPMs, we detected no impossible MSBs indicating injected keys. As these fTPMs are not certified under CC or FIPS 140, no information about the EK generation process is publicly available. However, the relatively high performance of fTPMs removes the need for off-chip generation. Therefore, detecting injected keys in fTPMs would be a potential concern.

On the contrary, we detected externally generated keys for almost all the physical dTPMs. Impossible MSBs or missing ROCA fingerprints for ROCA-vulnerable implementation were found for two out of three ranges of observed RSA implementations for Infineon TPMs. Although the only single EK value available to us for the range IFX 7.83-85.x is technically not impossible (as this implementation sometimes generates all MSB values), it is very improbable with less than 0.1% of on-chip generated keys having such MSB.

Impossible MSB was detected for the Nuvoton 7.2.2.0 version. We detected no impossible MSBs for TPMs manufactured by STM despite the corresponding certificates explicitly describing EK generation with a FIPS 140 compliant HSM. The reason is likely due to the low number of EKs available to us (only 7), possibly also due to the usage of HSM with a similar distribution of MSBs as the on-chip generation.

Similarly to Endorsement Keys, we tested Storage Root Keys' properties for any signs of deviation from the distribution expected for on-chip generated keys. The Storage Root Key (SRK) is a long-term key regenerated only when a user takes ownership of the module (operation typically performed only sporadically) and stored on a chip. SRK must be generated by the TPM chip itself, as the end user can repeatedly take ownership. SRK's properties may still be different (e.g., backdoored implementation would have a different way to generate SRKs, which are regenerated only very infrequently) but are expected to be the same as a chip's common ones. As shown in Figure 2.5, we detected no deviation from the expected SRK distribution.

Overall, we can conclude that EK injection was detected where expected for dTPMs based on their certification claims and *not* detected for fTPMs and iTPMs where it is functionally viable to generate such key directly on-chip. No suspicious deviance was detected for SRK.

2.7 Algorithm support and performance

The main TCG TPM specification [26] defines a number of algorithms, elliptic curves, and commands, not all of which need to be supported on all platforms, and thus their support varies among different TPM implementations. Platform-specific requirements for personal computers are defined by the TCG PC Client Platform TPM Profile (PTP) specification [50]. In this section, we present how frequent is the support of particular algorithms, elliptic curves, and selected commands in TPM firmware we encountered. The dataset contains this data only for measurements performed using the `tpm2-algtest` tool, which used the `tpm2_getcap` command from `tpm2-tools`¹³ with options `algorithms`, `ecc-curves`, `commands` to obtain the support information.

The observations are discussed below, with percentage support for selected algorithms summarized in Table 2.2, complemented by the information on whether the support of a particular algorithm, elliptic curve, or command is mandatory or only optional according to the latest PTP [50].

Algorithms The common cryptographic algorithms, including RSA, SHA1, SHA256, AES, and HMAC, were supported by all TPM firmware we encountered. Similarly, common ECC algorithms like ECDH and ECDSA on the NIST_P256 curve were supported by all but one old Intel fTPM (INTC 9.5.65.3000).

The support of the ECDAA algorithm is usually related to the support of BN_P256 curve and the `Commit` command, which are supposed to be used jointly, and they were absent only on Intel fTPMs that comply with the TPM revision from the year 2013 or earlier. Nuvo-ton TPMs have stopped supporting both the ECDAA algorithm and the BN_P256 curve with the version 7.2.3.1. Surprisingly, the MSFT 6.3.1.603 firmware seems to be supporting the ECDAA algorithm but not the BN_P256 curve. Algorithms from the SM specifications were supported only by Intel firmware from the latest 600.x series. The Schnorr signature algorithm ECSCHNORR used to be mandatory by the PTP, but it is not anymore since the last revision 1.59 [50]. It was not supported by Intel fTPMs complying with the 2013 specification or

13. <https://github.com/tpm2-software/tpm2-tools>.

2. TRUSTED PLATFORM MODULES

earlier and any of the Infineon TPMs. Nuvoton TPMs used to support this algorithm, but removed the support with version 7.2.3.1.

Hash algorithms from the SHA3 family were supported only by STM firmware 1.258, 1.512, and 1.769. Hash algorithms with 384-bit output were also supported by these STMs, but additionally also by Intel 600.x series and 7.2.x.x Nuvoton TPMs. No TPM supported hash algorithms with 512-bit output. All TPMs supported CFB encryption mode. No other mode was supported by Infineon TPMs, and ECB and CBC were also not supported by Nuvoton TPMs.

Table 2.2: Observed percentage support of TPM algorithms by firmware in the dataset. Each firmware sample was counted only once. M (mandatory), O (optional), M- (used to be mandatory but is not anymore), M+ (used to be optional but now is mandatory), M? (used to be mandatory but now is not mentioned), – (not mentioned).

TPM2_ALG_*	PTP	%	TPM2_ALG_*	PTP	%	TPM2_ECC_*	PTP	%
<i>ECC-related</i>			<i>Block ciphers</i>			NIST_P192	–	0
ECC	M	98	TDES	O	0	NIST_P224	–	7
ECDH	M	98	AES	M	100	NIST_P256	M	98
ECDSA	M	98	CAMELLIA	O	0	NIST_P384	M+	41
ECDA	M-	89	SM4	O	11	NIST_P521	–	0
ECSCHNORR	M-	72	<i>Encryption modes</i>			BN_P256	M-	87
ECMQV	O	0	ECB	–	70	BN_P638	–	0
SM2	O	12	CBC	–	70	SM2_P256	–	11
<i>RSA-related</i>			CTR	–	84			
RSA	M	100	OFB	–	84	TPM2_CC_*	PTP	%
RSASSA	M	100	CFB	–	100	ECDH_KeyGen	M	98
RSAES	M	100	<i>Other</i>			ECDH_ZGen	M	98
RSAPSS	M	100	XOR	M	100	EC_Ephemeral	O	82
OAEP	M	100	HMAC	M	100	ZGen_2Phase	O	82
<i>Hash functions</i>			CMAC	–	13	Commit	M-	97
SHA1	M	100	MGF1	M	74	Sign	M	100
SHA256	M	100	KEYEDHASH	M	100			
SHA384	M+	28	SYMCIPHER	M	100			
SHA512	O	0	KDF1_SP800_56A	M?	87			
SHA3_256	O	5	KDF1_SP800_108	M?	100			
SHA3_384	O	5	KDF2	–	0			
SHA3_512	O	0						
SM3_256	O	11						

Elliptic curves The support of at least NIST_P256 curve has been mandatory for all TPMs for the PC platform [50], and this is confirmed by our observations, where the only TPM not supporting this curve is INTC 9.5.65.3000, predating the PTP specification and not supporting ECC at all. The second most supported curve present on all TPMs complying with a specification revision from the year 2014 or later is the pairing-friendly BN_P256; support of this (or an alternative pairing-friendly) curve is required to enable the computation of the ECDAA protocol. The support of curves with lower bit-length than 256 bits is quite rare and has been observed only on older STM TPMs. The curves with a larger bit-length are more common, which is partly caused by NIST_P384 being mandatory in later revisions of the PTP specification [50]. The Chinese SM2_P256 curve has been supported only by 600.x series Intel firmware.

Commands The basic building blocks of advanced applications utilizing TPMs rely on six commands, as summarized by Chen and Urian [27]; the support for these operations among TPM firmware we analyzed is listed in the last section of Table 2.2. The majority of these commands are commonly supported by current TPMs; only EC_Ephemeral and ZGen_2Phase are not supported by Infineon TPMs, and Commit by the latest NTC 7.2.3.1.

Asymmetric decryption schemes [27] seem computable on all recent TPMs as they require only support for ECDH_KeyGen or ECDH_ZGen. Anonymous attestation and multi-party protocols utilize the two-stage construction based on Commit and Sign together with the ECDAA algorithm and, in some cases, also pairing friendly BN_P256 curve. Their joint support is quite common, with the only exception being the latest NTC and MSFT firmware.

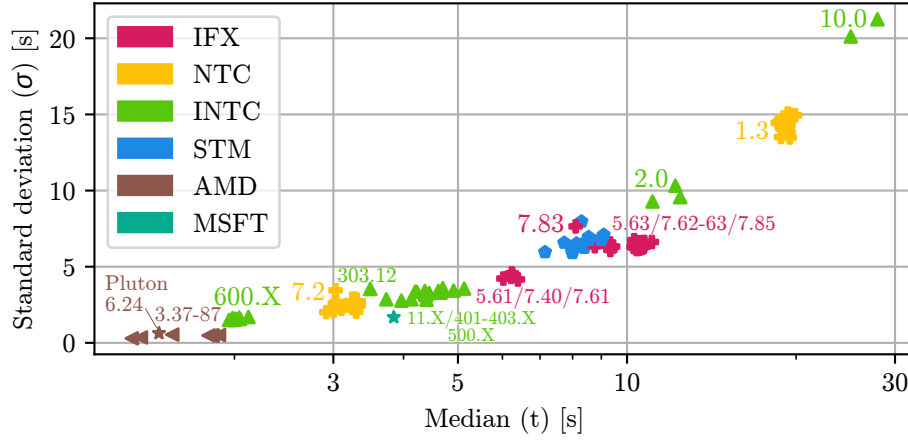
The lowest practical rate of support is for the two-stage key agreement protocols, not only because Infineon TPMs do not support these commands at all but also because they require the support of other algorithms to be fully usable. Apart from the basic ECDH, the SM2 support is present only in 600.x series Intel TPMs and ECMQV support has not been found in any of the firmware.

2.7.1 Variability of performance

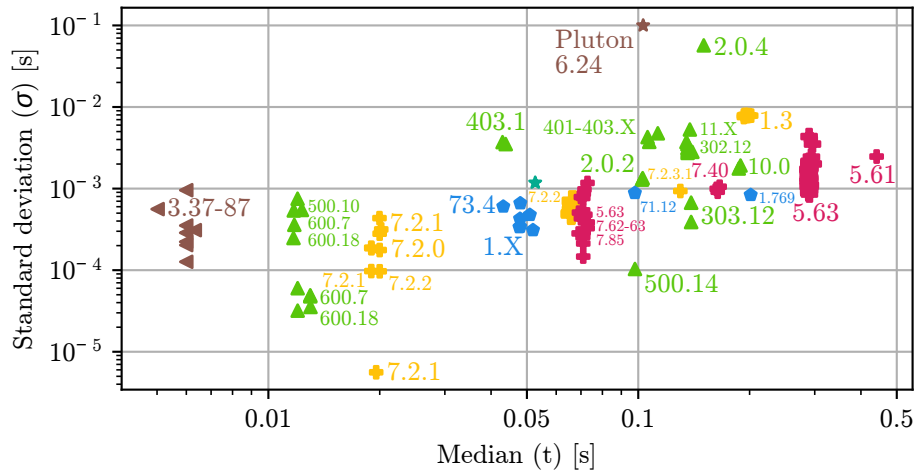
We collected and analyzed performance for all supported algorithms using repeated calls of the corresponding method using `tpm2-algtest` software. Each method invocation was provided with the same fixed input data and was repeatedly measured 1000 times using a high-precision host clock. However, some internal randomization may influence the resulting time, like the generation of RSA/ECC keys, generation of random data from stochastic processes, or execution of protected implementations. If a TPM firmware version is measured multiple times (more machines with the same TPM available), we verify whether the observed properties persist.

Due to the overall large number of firmware versions analyzed, we present only aggregated results for two operations analyzed in sections 2.5 and 2.6: `TPM2_Create` and `TPM2_Sign`, capturing the median and standard deviation of all measurements from physical chips with a given firmware version. The results are shown in Figure 2.6 with notable firmware clusters annotated, providing several insights.

All clusters observed for the generation of RSA 2048-bit key pairs directly map to change in the prime generation algorithm analyzed in Section 2.6.1, providing additional evidence. The only exception is a separation of Intel 500.x (Jasper Lake) and 600.x (Alder and Raptor Lake) versions where faster generation for 600.x can be attributed either to a faster underlying CPU architecture or other unknown changes to RSA key pair generation. The clusters observed for ECDSA-P256 signatures are more complicated to analyze, partly due to significantly shorter operation time, resulting in a higher impact of measurement noise and, thus, more overlapped clusters. Median time is faster with the later firmware revisions on Intel's fTPMs, with one notable exception – version 500.14.x (Comet Lake), which is almost 10x slower than the (numerically) preceding version 500.10.x (Jasper Lake). While the timing difference can be again explained by a change in the underlying CPU architecture, the observed differences also illustrate the difficulty in assessing the impact of a vulnerability based on numerical version ordering if a vendor does not consistently report all the vulnerable versions.



Variability of RSA 2048b key generation



Variability of ECDSA-P256 signature

Figure 2.6: An observed variance of 2048-bit RSA key pair generation and ECDSA-P256 signing operations with median time and standard deviation is shown over all collected firmware versions. Firmware clusters and notable outliers are annotated.

2.8 Conclusion

In this chapter, we studied the TPM ecosystem focusing on its security and capability of computing threshold protocols, allowing to decrease the impact of security vulnerabilities. One of the sources the study utilized was public certification documents. Their analysis aimed to identify discussions of changes in cryptographic implementations due to discovered vulnerabilities, but with negative results. Still, it provided insights into current trends in the TPM certification.

The primary data source for the analysis was a dataset of measurements from real TPM devices. We composed the dataset by collecting measurements from 78 unique TPM firmware using tools we developed. The tools captured information about devices' capabilities, properties of cryptographic algorithm outputs, and their performance.

The collected data revealed relatively frequent externally observable changes in cryptographic implementations. A vulnerability discovery in previous versions preceded some, but not all. Beyond that, the analysis confirmed a vulnerability in a range of elliptic-curve-based algorithm implementations and revealed its presence in algorithms that were previously not reported to be vulnerable. We discovered a new severe vulnerability in certain Intel firmware versions, which requires only *nine* signatures to extract the private key without any side-channel measurement, and it can be exploited even retrospectively. Intel either fixed it unintentionally while moving to the next firmware version with other changes or silently without disclosure.

The dataset consisting of a wide range of TPM firmware versions allowed us to evaluate the level of support and performance of cryptographic algorithms. Commonly used cryptographic operations (ECDH, ECDSA, SHA256, AES) were almost always supported. Adoption of more recent algorithms (SHA3) and larger bit sizes (SHA512, NIST_P512) was quite rare. The majority of firmware versions supported the ECDAA algorithm with a pairing-friendly curve, facilitating the construction of anonymous attestation and simple multi-party protocols [13, 14]. Still, the algorithm interface is too inflexible for the implementation of more advanced protocols beyond these two-round constructions, especially with the revised ECDAA computation.

3 JavaCard-based smartcards

This chapter is based on our two conference papers presented at the SECRIPT conference [A2] and the CARDIS conference [A3].

- [A2] P. Švenda, R. Kvašňovský, I. Nagy, and A. Dufka, “JCAIlgTest: Robust identification metadata for certified smartcards,” in *Proceedings of the 19th International Conference on Security and Cryptography - Volume 1: SECRIPT*, S. D. C. di Vimercati, Ed., INSTICC, SciTePress, 2022, pp. 597–604, ISBN: 978-989-758-590-6. DOI: 10.5220/0000163500003283.
 - [A3] L. Zaoral, A. Dufka, and P. Švenda, “The adoption rate of JavaCard features by certified products and open-source projects,” in *Proceedings of the 22nd Smart Card Research and Advanced Application Conference*, S. Bhasin and T. Roche, Eds., Cham: Springer, 2023, pp. 169–189, ISBN: 978-3-031-54409-5. DOI: 10.1007/978-3-031-54409-5_9.
-

3.1 Introduction

Smartcards are a widely deployed type of secure hardware primarily used in the telecommunication and banking sectors [51]. Their hardware is, to a large extent, defined by the ISO/IEC 7816 standard [52], according to which a smartcard must be tamper-resistant, contain an internal persistent storage, and embed a general-purpose processor with internal clock frequency typically ranging from 20 to 50 MHz. This general processor is usually insufficient to compute cryptographic operations on current security levels with practical performance; therefore, smartcards are often equipped with dedicated *cryptographic co-processors*, accelerating computationally intensive operations (e.g., modular exponentiation and scalar multiplication on an elliptic curve).

Multiple smartcard programming environments have been developed [53, 54, 55], but the JavaCard platform [53] established itself as the major programming interface, allowing the implementation

of smartcard applications in a restricted version of the Java language that is portable across smartcards of different vendors [56]. This platform provides no direct access to the processor and cryptographic co-processors. Instead, it exposes a set of high-level cryptographic algorithms like RSA and ECDSA, whose internal implementation is hardware-accelerated. The support of hardware acceleration, and consequently cryptographic algorithms, significantly varies among different smartcard models. In this chapter, we aim to analyze the algorithm support and performance of current JavaCard-based smartcards and provide data-driven insights into their usage within the JavaCard open-source ecosystem.

To achieve this goal, we first performed an automatic search of documents accompanying security certification performed under the Common Criteria [11] and FIPS 140 [12] schemes for JavaCard-related keywords, and analyzed their changes in time for different API versions and included cryptographic algorithms. Although this analysis yielded valuable insights, there was a need to obtain more complete information about smartcards' actual capabilities and performance.

We developed a tool for collecting smartcard properties and used it to gather data from more than a hundred smartcard samples. This data enabled us to analyze the extent of algorithm support and performance of real smartcards. We identified a large disparity between the features described in the JavaCard API specification and the features actually supported by available smartcards. While JavaCard applets are supposed to be portable between different smartcards, the uneven level of feature support directly influences the portability and adoption of specific algorithms by applets developed for smartcards.

This issue is confirmed by our analysis of the JavaCard open-source ecosystem. We collected around two hundred open-source applet implementations and analyzed their algorithm usage and smartcard compatibility. The analysis revealed that open-source software uses only a minor fraction of constants (algorithms and parameters) from the JavaCard specification. Still, attempts to compile and install the applets on real smartcards were underwhelming. From 187 compilable applets, our most compatible smartcard could run only 124 (66%) applets. In contrast, only 64 (34%) of the applets could run on all three smartcard models we tested, showing a severe limitation of applet portability.

The rest of this chapter is organized as follows. Section 3.2 surveys the related work. Section 3.3 analyzes mentions of JavaCard technology by certification artifacts issued under Common Criteria and FIPS 140 certification schemes. Section 3.4 presents the smartcard capability testing framework and the data it collects. Section 3.5 analyzes the collected data and presents an overview of algorithm support by current JavaCard-based smartcards. Section 3.6 surveys the JavaCard open-source ecosystem and analyzes its activity in time and the portability of applets between different physical smartcards. Finally, conclusions follow in Section 3.7.

3.2 Related work

Many publications are investigating the JavaCard platform, with the majority of them examining its security from either the software perspective [57, 58, 59, 60, 61] or the hardware one [2, 62, 63, 64].

A smaller number of works focus on performance and capabilities. The performance of basic cryptographic primitives on three programmable smartcard platforms (JavaCard, .NET, and MultOS) is examined by Hajny et al. [65], but they focused only on a small subset of operations. Bernabe and Clarke [66] studied RSA performance in JavaCards with a focus on predicting the operation run-time of RSA-4096 functions. The MESURE project [67], founded by the French government and started in 2006, produced a complex benchmarking framework. Exact timing measurements were not published – only an aggregate score for each card.

3.3 Common Criteria and FIPS certification

Smartcards tend to be used in environments imposing strict compliance requirements, and as such, the devices need to undergo security certifications, typically performed under the Common Criteria [11] or FIPS 140 [12] schemes. The certification process results in public human-readable documents that, apart from the scope of the evaluation, the testing process, and evaluation results, contain information about the software of the tested device and supported algorithms.

3. JAVACARD-BASED SMARTCARDS

This information can be used to gain insight into trends in certified smartcards. For the analysis, we downloaded all public certification documents collected by the *sec-certs* project [42]. The dataset already contained JavaCard API version numbers extracted from the documents using regular expressions. However, getting a more insightful look inside the trends in algorithm usage required more precise data processing, which involved semi-manual filtering of raw data.

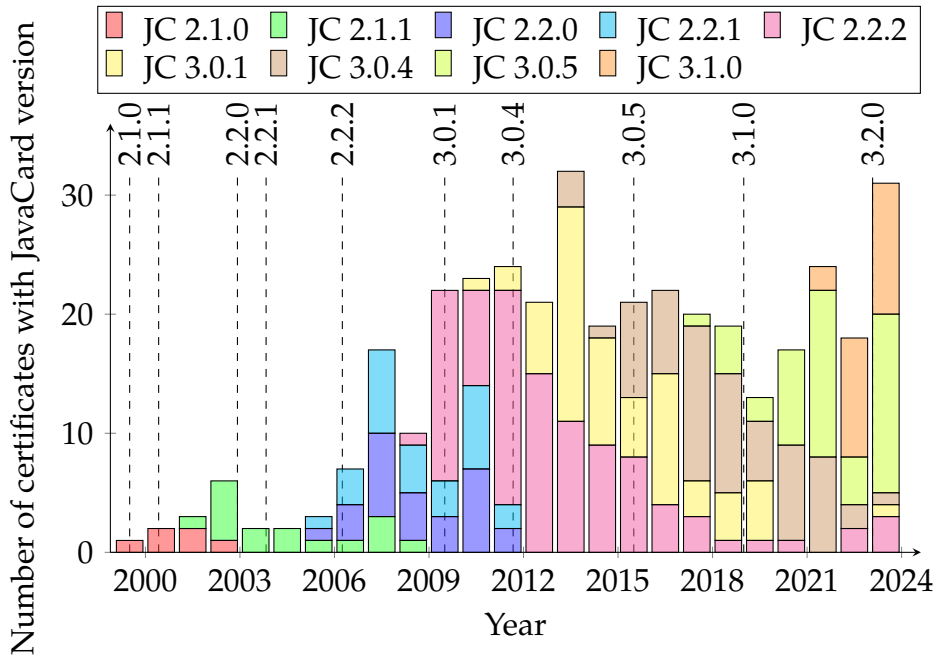


Figure 3.1: The number of certification documents mentioning a specific JavaCard API version per year (the year 2023 only till the end of October). In case multiple versions were detected in a document, only the latest one was included in the chart.

First, we analyzed the dependency of mentions of JavaCard API versions in certification documents on a certificate issuance year. The distribution is shown in Figure 3.1, where we display only the latest version detected per certificate document (406 documents in total). We see that JavaCard API version 2.2.2 was in use for the longest period, starting in 2008 (2 years after its introduction) and being commonly mentioned till 2017. Even in the year 2022, certificates mentioning this

API version were issued. Still, since the year 2010, API with major version number 3 started gradually appearing, and by 2013, they were mentioned in the majority of JavaCard certification documents. For the analysis of new algorithm adoption, we focused on these newer versions, namely 3.0.1, 3.0.4, 3.0.5, and 3.1.0.

Among the 10197 certificate documents in the dataset, we have identified 66 that describe a JavaCard target and include references to strings prefixed with `ALG_`, typical naming of JavaCard algorithm constants. Out of the 66 documents, those issued before the year 2009 were certified only under the FIPS 140 scheme (6 certificates). After that, all 60 subsequent documents were issued under the Common Criteria scheme. In the documents, we identified 139 unique strings that match the pattern; however, only 104 of them correspond to algorithms included in the JavaCard API.

Figure 3.2 displays mentions of JavaCard algorithms introduced in JavaCard API 3.0.1 and later, grouped by the certificate issuance year. In general, algorithms start being mentioned in certification documents only two or more years after the specification version is published. The only exception to this is the certificate of an Oberthur smartcard¹, which mentions algorithm `ALG_SHA_224` even before its specification was released.

JavaCard API version 3.0.5 was released in 2015, and the first algorithm added to the specification appeared in a certification document in 2017. Later, an algorithm (`ALG_EC_SVDP_DH_PLAIN_XY`), allowing for efficient ECDH computation outputting full point, started appearing in 2018. This algorithm was the most frequently mentioned in the documents in the year 2022, together with its variant outputting only x-coordinate (`ALG_EC_SVDP_DH_PLAIN`) that was introduced in API version 3.0.1. Version 3.0.4 introduced two new algorithms, which first appeared in certification documents in 2016, five years after its publication. Apart from the Oberthur certificate, mentions of algorithms introduced in JavaCard API version 3.0.1, released in May 2009, started appearing in 2011, but their mentions became more frequent only after 2015.

We also inspected the remaining strings that matched the JavaCard algorithm pattern but are not part of the JavaCard API. Many

1. <https://sec-certs.org/fips/6d094db49a6e2242/>.

3. JAVACARD-BASED SMARTCARDS

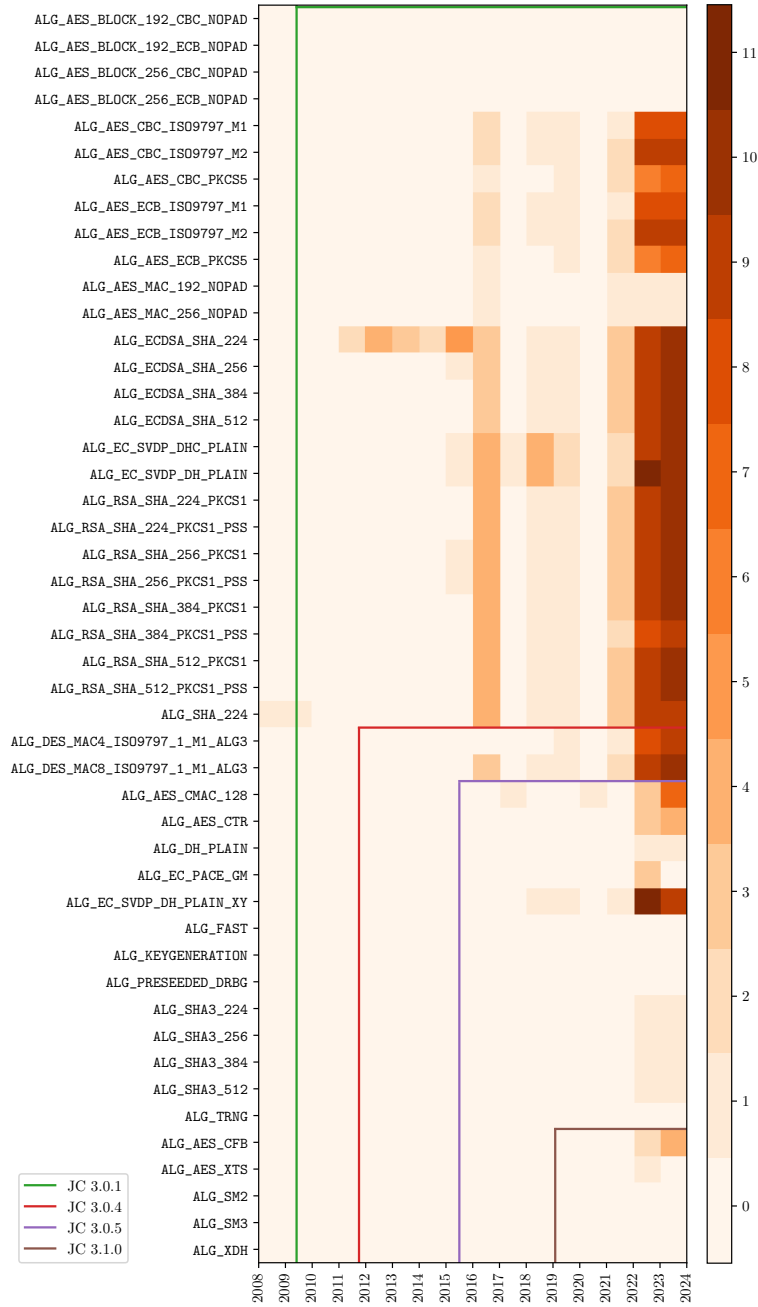


Figure 3.2: Number of the CC and FIPS 140 certificates that mention JavaCard API 3.0.1+ algorithms (by year, 2023 only till October).

Table 3.1: Algorithms detected in CC and FIPS certificates by search of prefix ALG_ that are not included in the official JavaCard API specification.

ALG_AES_BLOCK_128_CBC_NOPAD_STANDARD	ALG_DES_CMAC	ALG_EC_SVDP_DHC_GK
ALG_AES_CBC_ISO9797_M2_STANDARD	ALG_DES_CMAC8	ALG_EC_SVDP_DHC_PACE
ALG_AES_CBC_ISO9797_STANDARD	ALG_DES_ECB_PKCS7	ALG_EC_SVDP_DH_GK
ALG_AES_CBC_PKCS7	ALG_DES_MAC128_ISO9797_1_M2_ALG3	ALG_ED25519PH_SHA_512
ALG_AES_CMAC128	ALG_DES_MAC_8_NOPAD	ALG_MONT_DH_25519
ALG_AES_CMAC16	ALG_ECDSA_RAW	ALG_RSA_SHA256_PKCS1
ALG_AES_CMAC16_STANDARD	ALG_ECDSA_SHA224	ALG_RSA_SHA256_PKCS1_PSS
ALG_AES_CMAC8	ALG_ECDSA_SHA256	ALG_RSA_SHA_1_RFC2409
ALG_AES_ECB_PKCS7	ALG_ECDSA_SHA256_LDS	ALG_RSA_SHA_256_ISO9796
ALG_AES_MAC_128_ISO9797_1_M2_ALG3	ALG_ECDSA_SHA384	ALG_SHA2_CHAIN
ALG_AES_OFB	ALG_ECDSA_SHA384_LDS	ALG_SHA_CHAIN
ALG_DES_CBC_PKCS7	ALG_ECDSA_SHA_LDS	
ALG_DES_CMAC	ALG_EC_SVDP_DHC_GK	

of these algorithms are likely incorrectly input in the source documents; however, some of them clearly denote algorithms unsupported by JavaCard API. The algorithm names are presented in Table 3.1 with several interesting items like ALG_EC_SVDP_DH_GK whose function is not known due to missing public documentation, referring to Oberthur’s proprietary API extensions². Another interesting mentions are ALG_ED25519PH_SHA_512 in seven certificates by NXP³, which most likely refers to Ed25519 signature algorithm with prehashed input and SHA512 function. The same seven certificates also include string ALG_MONT_DH_25519, likely referring to the Diffie-Hellman key agreement on Curve25519 in the Montgomery form.

3.4 Data collection from real devices

We developed a JavaCard testing suite called JCAlgTest, which provides a means to gather, evaluate, and compare the capabilities of JavaCard-based smartcards. The suite consists of a host application and a JavaCard applet uploaded on a card. The on-card testing applet is compatible with smartcards starting from JavaCard version 2.1.1. The following paragraphs describe the collected data and the approach to its collection.

2. <https://sec-certs.org/fips/6d094db49a6e2242/,/a5bef651c8e3fd6c/>.

3. <https://sec-certs.org/cc/03aded94fb04c62e/,/45098872448f5816/,/ae175aa839dbf692/,/b0e6f667d52402df/>.

ATR value The Answer to Reset (ATR) is a byte sequence output by a smartcard when powered or reset, allowing a smartcard reader to recognize and set a proper communication protocol and speed. Beyond the protocol information, the sequence contains a part called *historical bytes*, which may contain arbitrary additional information used to identify the type of the card.

CPLC metadata The CPLC (Card Production Life Cycle) metadata contains detailed information about the chip manufacturer, type, operating system version, release date, serial number, or personalization equipment information. These values are obtained by invoking two APDU commands 0x80CA9F7F and 0x00CA9F7F.

Algorithm support The JavaCard API specification contains a multitude of packages with security-related classes of cryptographic algorithms (e.g., Cipher, Signature, MessageDigest, RandomData), further parameterized using different algorithm types (e.g., ALG_AES or ALG_RSA), key lengths, and modes of operation. In the JavaCard API 3.0.5 [19], the algorithm-parameter combinations add up to more than 200. However, a specific card product does not need to support all combinations. To capture this information, the JCAAlgTest applet attempts to instantiate all defined classes with corresponding parameters and reports the results to the host application.

JavaCard package support A given smartcard may or may not support certain JavaCard packages. To obtain this information, we utilize the fact that an applet requiring an unsupported package will fail to load on a given smartcard.

An empty applet is first compiled and prepared to be uploaded to a smartcard. Before the upload is performed, the installation file is unpacked and internal file `Import.cap` is modified to reference the tested package⁴. Upload of the modified file is attempted, and if it succeeds, the tested package is supported; otherwise, it is not. We test for all packages specified in the JavaCard specification up to version 3.0.5, covering 120 different packages and versions in total.

4. For example, package AID and its version, e.g., 0x020107A00000000620201 for `javacardx.crypto v1.2` as defined in JavaCard API 2.2.1.

RSA & ECC key pairs Cryptographic keys generated by a smartcard may contain a bias stemming from the implementation of the key generation algorithm [33]. We collect RSA and ECC keypairs generated by repeated calls to the `KeyPair.genKey()` method and export both public and private components to the host. The collection can take minutes to several hours, depending on the number of key pairs and their bit length.

Algorithm performance Smartcard performance varies greatly depending on the internal chip frequency, hardware-accelerated subset of algorithms, the clock of the used memory chips, and implementation. Despite that, smartcard manufacturers rarely provide detailed reports on the performance of their products.

In order to evaluate and compare cards for a specific use case, all individual algorithms are benchmarked for all possible parameter combinations (e.g., various key lengths or padding options). Additionally, all methods of a particular class (e.g., `setKey()`, `getKey()` and `eraseKey()` for class `AESKey`) are measured, resulting in more than 2300 possible combinations to be tested.

The smartcard platform lacks an on-card timer, so time measurements must be conducted by the host application. Consequently, we measure not only the target operation but also the time required for data transfer and other surrounding code. To extract the target operation time, a *calibration phase* is first conducted, measuring the overhead of non-target operations by executing all the surrounding code without the target operation. Then, an *experiment phase* is run with the target operation. Each of these phases is repeated multiple times. The runtime of the target operation is determined by subtracting the average overhead time (from the calibration phase) from the average runtime (from the experiment phase).

This approach provides accurate measurements for most algorithms and methods. However, two cases require special treatment: methods preceded by inconsistent execution time (e.g., prime generation) and operations with execution time below the measurement error. In those cases, we preload all used objects to minimize the variability and run multiple iterations of the same operation. The whole set of performance experiments typically takes several hours to finish.

3.5 Analysis of collected data

We built a publicly available⁵ database of results collected from more than a hundred JavaCard-based smartcards. Approximately half of the smartcards were tested directly by us, while the rest were provided by volunteers worldwide. The longer and more complex tests were performed only on cards available in our laboratory. Due to the nature of the study, we were able to include only smartcards that are commercially available in smaller quantities via third-party resellers. The rest of this section provides an insight into the JavaCard ecosystem via the lens of collected data.

3.5.1 JavaCard API and package support

Starting from JavaCard API version 2.1 in 1999, the specification included the DES cipher and the TRNG random data source. As can be seen in Table 3.2, all tested smartcards support their operations. On the contrary, while the AES algorithm [68] was already included in API version 2.2 in 2002, it was not until 2006 that the first smartcards started to support it. The algorithm became commonly supported only after the year 2010⁶. Almost all cards provide both ECB and CBC modes of encryption for block ciphers as well as for the padding schemes PKCS#1 and ISO9797. More recent padding methods like OAEP and PSS are supported by less than 50% of the analyzed smartcards but by almost all recent ones with API version 3.0.5.

JavaCard API version 2.1 also first includes algorithms for RSA cryptosystem up to 2048-bit keys, which was extended to 4096-bit keys in version 3.0.1. Although the key-size limitation was raised to 4096 bits, very few cards support on-card generation of such keys, while a large majority supports only up to 2048-bit keys. This is likely due to the time requirements of longer key generation and the universal shift towards elliptic curve cryptosystems, providing the same level of security with smaller key sizes.

We observed only a small fraction of cards with DSA algorithm support. Newer cards seem to abandon DSA for its modern elliptic curve counterpart, ECDSA. Despite this, the overall picture shows

5. <https://fi.muni.cz/~xsvenda/jcalgtest/>.

6. Based on `ICFabricationDate` from the CPLC information.

that EC algorithms are not yet fully supported, as, for instance, only around 70% of 3.0.1+ smartcards support EC key lengths of at least 256 bits. Moreover, only a hashed version of the Diffie-Hellman Key Agreement protocol (IEEE P1363) is frequently available, while the algorithm outputting a plain coordinate is only sparsely supported (except for the newest JC API 3.0.5 cards). No card with a mode for authenticated encryption (CCS, GCM) was encountered.

Finally, manufacturers implement all classic hashing functions such as SHA-1, SHA-2 (256-bits), and RIPEMD-160, while support for the insecure MD5 seems to be phased out in newer cards. The support for SHA-2 (512-bits) is common in newer cards, but no tested smartcard yet supports SHA-3.

Packages support JavaCard package support was collected in 40 data samples. While standard packages like `javacard.framework` or `javacardx.crypto` are always supported, specialized packages like `javacardx.biometry` (11x), `javacardx.framework.util.intx` (9x), `javacardx.framework.math` (4x) are supported only in a small fraction of the tested cards.

3. JAVACARD-BASED SMARTCARDS

Table 3.2: The level of support for algorithms specified in JavaCard API. For a given feature, the version column specifies the JavaCard specification that defined it first, while the subsequent columns show its availability in cards reporting particular supported version via the `JCSystem.getVersion()` method and maximally supported version of the `javacard.framework` package. Results for smartcards with an unknown version were not included.

Feature	First in version	JC ≤ 2.2.1 (21 cards)	JC 2.2.2 (26 cards)	JC 3.0.1/2 (12 cards)	JC 3.0.4 (29 cards)	JC 3.0.5 (11 cards)
<i>Truly random number generator</i>						
TRNG (ALG_SECURE_RANDOM)	≤ 2.1	100%	100%	100%	100%	100%
<i>Block ciphers used for encryption or MAC</i>						
DES (ALG_DES_CBC_NOPAD)	≤ 2.1	100%	100%	100%	100%	100%
AES (ALG_AES_BLOCK_128_CBC_NOPAD)	2.2.0	52%	96%	100%	100%	100%
KOREAN SEED (ALG_KOREAN_SEED_CBC_NOPAD)	2.2.2	5%	62%	75%	34%	0%
<i>Public-key algorithms based on modular arithmetic (key generation)</i>						
1024-bit RSA (ALG_RSA(_CRT) LENGTH_RSA_1024)	≤ 2.1	76%	96%	100%	93%	82%
2048-bit RSA (ALG_RSA(_CRT) LENGTH_RSA_2048)	≤ 2.1	67%	96%	100%	93%	82%
4096-bit RSA (ALG_RSA(_CRT) LENGTH_RSA_4096)	3.0.1	0%	0%	0%	3%	0%
1024-bit DSA (ALG_DSA LENGTH_DSA_1024)	≤ 2.1	5%	8%	8%	10%	0%
<i>Public-key algorithms based on elliptic curves</i>						
192-bit ECC (ALG_EC_FP LENGTH_EC_FP_192)	2.2.1	5%	62%	83%	66%	82%
256-bit ECC (ALG_EC_FP LENGTH_EC_FP_256)	3.0.1	0%	50%	75%	66%	82%
384-bit ECC (ALG_EC_FP LENGTH_EC_FP_384)	3.0.1	0%	12%	17%	62%	82%
521-bit ECC (ALG_EC_FP LENGTH_EC_FP_521)	3.0.4	0%	4%	8%	45%	82%
ECDSA SHA-1 (ALG_ECDSA_SHA)	2.2.0	24%	84%	100%	69%	82%
ECDSA SHA-2 (ALG_ECDSA_SHA_256)	3.0.1	5%	12%	100%	69%	82%
IEEE P1363 ECDH (ALG_EC_SVDP_DH)	2.2.1	29%	81%	100%	69%	82%
IEEE P1363 ECDH X (ALG_EC_SVDP_DH_PLAIN)	3.0.1	5%	4%	67%	48%	82%
IEEE P1363 EDCH X,Y (ALG_EC_SVDP_DH_PLAIN_XY)	3.0.5	0%	0%	0%	17%	82%
TR03110v2 generic mapping (ALG_EC_PACE_GM)	3.0.5	0%	0%	0%	17%	82%
<i>Modes of operation and padding modes</i>						
ECB, CBC modes	≤ 2.1	100%	100%	100%	100%	100%
CCM, GCM modes (CIPHER_AES_CCM/GCM)	3.0.5	0%	0%	0%	0%	0%
PKCS1, NOPAD padding	≤ 2.1	95%	100%	100%	100%	100%
PKCS1 OAEP scheme (ALG_RSA_PKCS1_OAEP)	≤ 2.1	14%	31%	8%	41%	82%
PKCS1 PSS sheme (ALG_RSA_SHA_PKCS1_PSS)	3.0.1	14%	19%	83%	41%	100%
ISO14888 padding (ALG_RSA_ISO14888)	≤ 2.1	14%	12%	8%	0%	0%
ISO9796 padding (ALG_RSA_SHA_ISO9796)	≤ 2.1	81%	100%	100%	86%	100%
ISO9797 padding (ALG_DES_MAC8_ISO9797_M1/M2)	≤ 2.1	90%	100%	100%	100%	100%
<i>Hash functions</i>						
MD5 (ALG_MD5)	≤ 2.1	90%	77%	92%	62%	0%
RIPEMD-160 (ALG_RIPEMD160)	≤ 2.1	10%	15%	9%	4%	0%
SHA-1 (ALG_SHA)	≤ 2.1	95%	100%	100%	100%	100%
SHA-256 (ALG_SHA_256)	2.2.2	14%	88%	100%	97%	100%
SHA-512 (ALG_SHA_512)	2.2.2	5%	23%	25%	90%	100%
SHA-3 (ALG_SHA3_256)	3.0.5	0%	0%	0%	0%	0%

3.5.2 Information from CPLC data

A total of 71 cards were recorded together with its Card Production Life Cycle (CPLC) data. We analyzed the available security certificates and other public sources to obtain the mapping between the used numeric constants and their human-readable titles of manufacturers (`ICFabricator`), chip type (`ICType`), and operating system (`OperatingSystemID`). In total, we encountered:

- **ICFabricator** – nine different chip fabricators:
 - Renesas (0003, 3060),
 - Infineon (0005, 4090, 4830),
 - Philips (2050),
 - NXP (4070, 4790),
 - Atmel (4180),
 - Samsung (4250),
 - STMicro (5354),
 - Tongxin (008c), and
 - unidentified (4220).
- **ICType** – 45 different chip types with NXP and Infineon responsible for the large majority of the chips encountered. While NXP tends to manufacture hardware chips together with the whole software stack and market resulting cards under its own brand, Infineon, as a chip manufacturer, provides not only complete cards sold under its brand but more frequently provides chips to other vendors.
- **OperatingSystemID** – 23 different codes for operating systems with 52 subvariants.

We also analyzed the current market situation with respect to existing vendors after various company mergers and visualized the resulting aggregate accordingly. For example, the *Gemalto* cluster is aggregating not only cards produced under the *Gemalto* brand but also for *Gemplus*, *Axalto*, and *Schlumberger* (now inactive) brands. The visualization and additional artifacts are available online⁷.

7. https://crocs.fi.muni.cz/papers/jcalgtest_secrypt22.

3.5.3 Public-key algorithm performance

The time requirements of symmetric operations are typically almost negligible compared to asymmetric operations like signing and key agreement protocols. Therefore, in performance analysis designed to map important cryptographic building blocks, we focused on commonly used public-key operations. Figure 3.3 shows the performance of such operations on smartcards that supported them (10 samples).

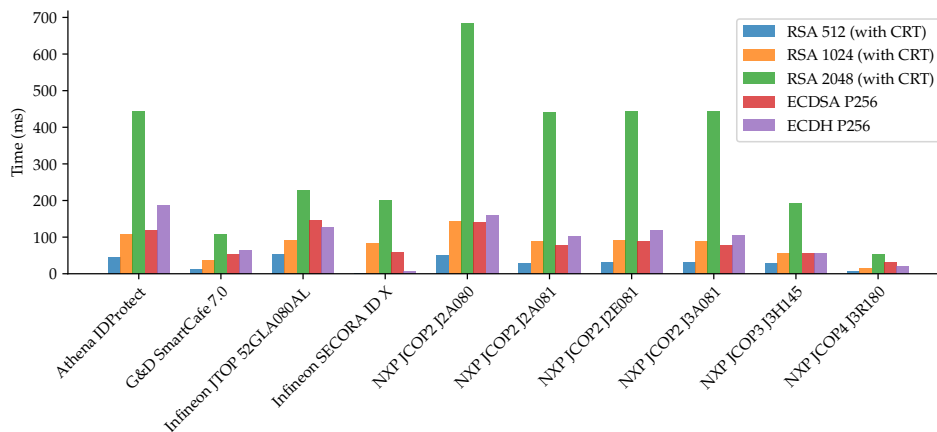


Figure 3.3: Performance of common public-key operations on compatible smartcards.

RSA signing (accelerated using CRT [69]) is the slowest on the NXP JCOP2 J2A081 smartcard, where 2048-bit RSA required on average 683 ms, whereas on other smartcards less than 444 ms. Signing with smaller moduli was substantially faster; 1024-bit RSA signing was computed in less than 150 ms, and 512-bit in less than 52 ms.

Elliptic-curve operations are significantly faster than RSA signing on smartcards at comparable security levels. ECDSA signing on the P256 curve takes less than 146 ms and ECDH key agreement less than 187 ms. The smallest relative performance gap between the RSA 2048 signing and ECDSA P256 signing was observed on the Infineon JTOP 52GLA080AL smartcard, where ECDSA was only 36% faster. On most other smartcards, ECDSA outperformed RSA by more than 70%.

The best-performing smartcard overall was the NXP JCOP4 J3R180, the newest analyzed smartcard. It completed all operations discussed

in this section under 54 ms. Additionally, it can efficiently compute new operations (ALG_EC_SVDP_DH_PLAIN_XY and ALG_EC_PACE_GM) included in JavaCard API 3.0.5, making it a promising target for the computation of more complex protocols like threshold schemes.

3.6 Analysis of JavaCard open-source projects

The capabilities and performance of smartcard models may not directly reflect their feature usage. To assess the actual adoption of JavaCard features, we analyzed the ecosystem of open-source JavaCard applets. We built a database of all available public repositories relevant to the JavaCard platform, performed a static and dynamic analysis of contained JavaCard applets, and tested their deployability on three physical smartcards.

The database is being built since 2017. We regularly searched for JavaCard open-source repositories on GitHub, SourceForge, and GitLab for the occurrence of `javacard.framework` keyword and collected hits into a public list hosted on GitHub⁸. We included almost all discovered repositories without assessing their maturity – only trivial, testing, or unfinished applets were excluded. As some projects are occasionally moved or removed, we created a backup of each repository when included. While we inserted most records into the database, some contributions were also made by the online community.

We have manually examined all collected repositories to create a refined list with an explicit enumeration of JavaCard applets. The refined list comprises 139 repositories containing at least one JavaCard applet or library. The repositories are mainly found on GitHub, with a very small number on SourceForge; no repository on GitLab has been detected. Sources that could not be parsed even with import resolution disabled were omitted from further analysis. The list contains 206 JavaCard projects with 223 possible applet entry points, of which 36 cannot be parsed precisely due to missing dependencies or programming mistakes preventing code compilation with a standard `javac` compiler. For analysis of the usage of specification features, we excluded the applet corresponding to the JCAlgTest project⁹ that

8. <https://github.com/crocs-muni/javacard-curated-list/>.

9. <https://github.com/crocs-muni/jcalgtest/>.

intentionally references (almost) all cryptographic algorithms and parameterization constants found in the JavaCard specification [70].

The following types of JavaCard open-source projects were included in the analysis:

- electronic passports and citizen ID (8x),
- authentication and access control (29x),
- payments and loyalty (20x),
- key and password managers (15x),
- digital signing, OpenPGP, and e-mail security (8x),
- e-Health (1x),
- NDEF tags (6x),
- cryptocurrency wallets (7x),
- emulation of proprietary cards (8x),
- mobile telephony (SIM) (5x),
- library JavaCard code (47x),
- learning (school projects, etc.) (5x), and
- other (24x).

3.6.1 Open-source activity in time

To assess the activity of the JavaCard open-source ecosystem in time, we analyzed each project's state by its git commit closest to (but prior) the next date interval. For a yearly activity, we analyzed the commit closest to the 1st of January of the following year, whereas, for a monthly activity, we used the commit closest to the first day of the following month.

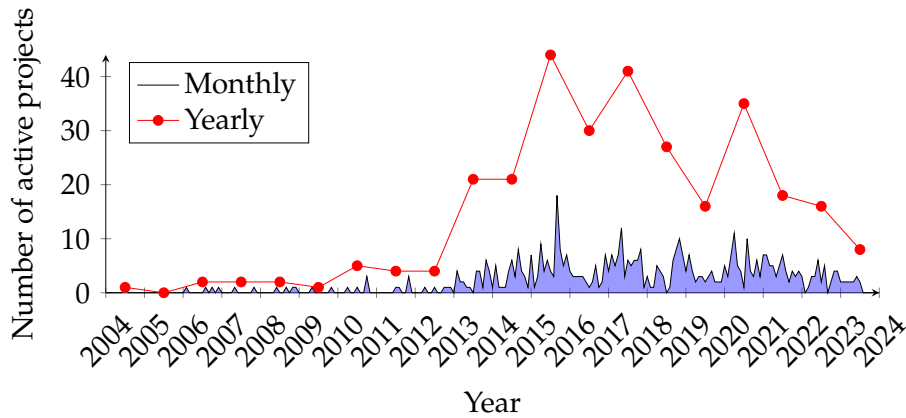


Figure 3.4: Number of open-source projects with at least one commit per month (black line) or per year (red line), respectively. The year 2023 is only till the end of October.

The number of projects with at least one commit in a given period (year or month) is shown in Figure 3.4. While the oldest commits are from 2004, the number of active projects was very low (around three projects) until 2013, when the number of active projects increased significantly to more than 20 projects, reaching more than 40 active projects in 2015 and 2017. Figure 3.5 visualizes included repositories based on the number of forks and stars, with the most popular repositories annotated.

While the development activity fluctuates significantly between years (the year 2019 had only around 50% of active projects compared to 2017 and 2020), there seems to be a significant activity decrease following the year 2020, with only eight active projects till the end of October 2023. If the trend continues, we may be witnessing a decline in open-source activity in the JavaCard ecosystem despite a still high number of newly certified JavaCard products (Section 3.3).

An intriguing possibility might be a correlation with the trends observed in the JavaCard-related certification artifacts but with a lag of about four to six years. The lag can be explained by the delayed availability of certified smartcards to open-source developers for purchase in smaller quantities. If this correlation is genuine, we may observe increased activity in the open-source projects in the following years, as activity in JavaCard certification dipped in 2019 and increased again.

3. JAVACARD-BASED SMARTCARDS

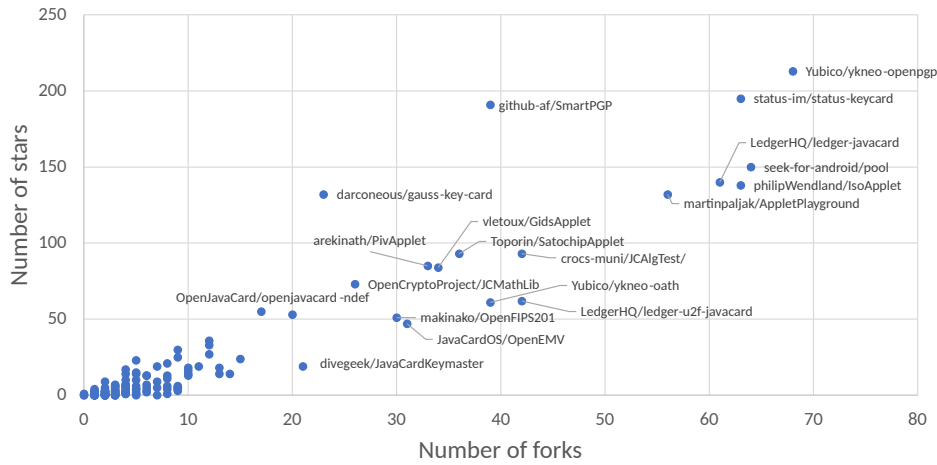


Figure 3.5: The scatter plot of JavaCard repositories popularity using number of forks and stars on the GitHub platform. Projects with at least 20 forks are annotated by name.

3.6.2 Usage of JavaCard API

The extensive list of open-source JavaCard projects available via git repositories allows us to analyze the packages, classes, methods, and constants using static analysis of their source code. The analysis relies heavily on the functionality provided by the Spoon library [71], which is designed for the analysis and transformation of Java source code. Spoon parses the input source code into a complete abstract syntax tree (AST). Therefore, the source code can be analyzed and instrumented more reliably than in the case of the regex-based approaches searching for target keywords. The combination of git history and algorithms extracted from the source code at the given date (e.g., the latest commit in a given year) allows us to analyze the adoption rate for *every* constant defined in the JavaCard specification.

Figure 3.6 shows the popularity of both symmetric (MACs) and asymmetric signature schemes. The most used signature algorithms are ECDSA and RSA with PKCS#1 padding, followed by the 3DES-based and HMAC-based MAC schemes. Relatively few applets utilize other options. Related Figure 3.7 shows the adoption of different variants of hash functions in combination with ECDSA over time, and

Figure 3.8 compares the time of introduction and use of two frequently used signature algorithms. While ALG_ECDSA_SHA_256 is still added to the new code, ALG_RSA_SHA_PKCS1 no longer is.

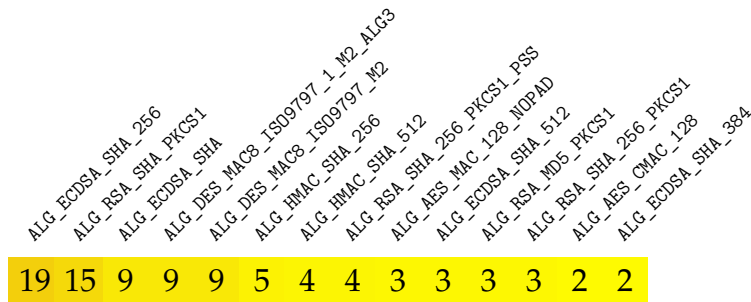


Figure 3.6: javacard.security.Signature ALG constants with at least two usages.

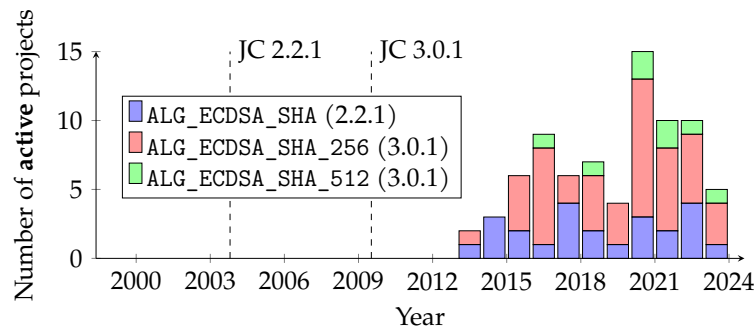


Figure 3.7: Histogram of the adoption of ALG_ECDSA_SHA constants.

Figure 3.9 shows popular key lengths. As expected, AES-128/256 and 3DES-112/168 are used frequently for symmetric cryptography, EC-FP-256 and RSA-2048 for asymmetric one. The only surprise is still a high usage of RSA-1024. Related Figure 3.10 demonstrates the trend for the DES algorithm with one, two, and three keys in time, showing a later adoption of longer keys.

The usage of third-party, sometimes proprietary, APIs is shown in Figure 3.11. GlobalPlatform/OpenPlatform API is mainly used to handle secure channel messages or access the global smartcard state. If proprietary API is used (NXP, Giesecke+Devrient), the applet can

3. JAVACARD-BASED SMARTCARDS

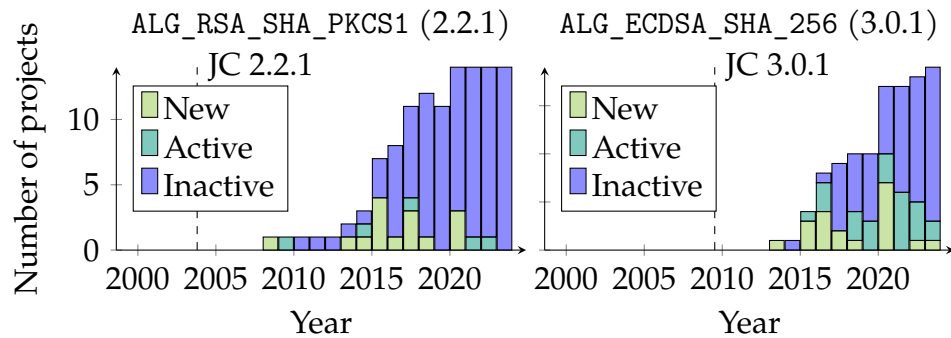


Figure 3.8: Comparison of the adoption of ALG_RSA_SHA_PKCS1 and ALG_ECDSA_SHA_256 constants.

be statically analyzed for the usage of JavaCard features but cannot be dynamically analyzed due to the missing proprietary SDK. The results for other relevant classes are available in supplementary materials¹⁰.

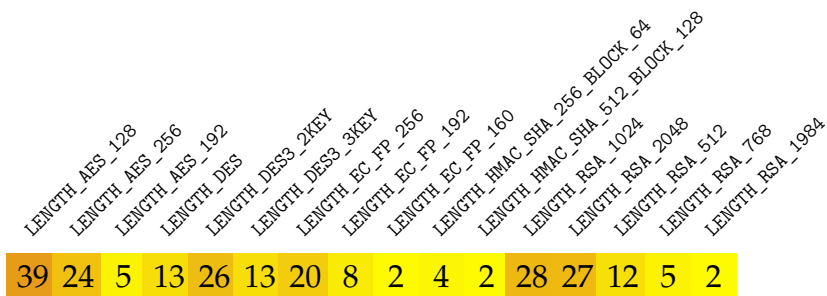


Figure 3.9: javacard.security.KeyBuilder LENGTH constants with at least two usages sorted by the corresponding algorithms.

10. <https://crocs.fi.muni.cz/papers/cardis2023>.

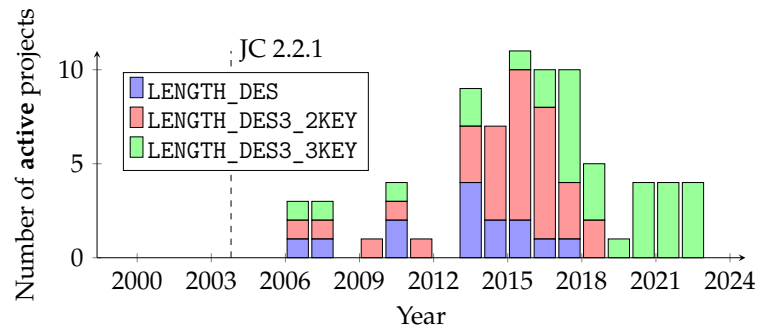


Figure 3.10: Histogram of the adoption of LENGTH_DES constants from JavaCard 2.2.1.

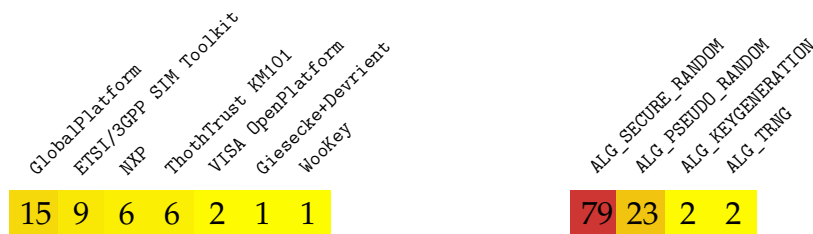


Figure 3.11: Usage of third-party APIs and `javacard.security.RandomData` constants.

We also analyzed the adoption of different constants from JavaCard API up to 3.2, with 583 values in total. A surprisingly high number of constants is entirely unused (404, 69.3%), and 488 constants are used in less than six projects (83.7%). Only 42 constants are used in more than 25 (7.2%) projects. Table 3.3 provides complete statistics.

Table 3.3: The usage of constants from JavaCard API by open-source applets.

Applets	no use	1	2-5	6-10	11-25	26-50	51-75	76-100	100+
Constants	404	36	48	20	33	21	7	4	10

3.6.3 Applet compatibility and portability

We performed the following steps for all collected open-source applets to test the possibility of their deployment on real smartcards:

1. **Compile** the applet using standard javac compiler, resulting in *.class file(s). The step fails if the source code is not compatible with the Java language or references proprietary packages.
2. **Convert** compiled *.class files using Oracle JavaCard converter for specific version of JavaCard SDK, resulting in a *.cap file. The step fails if the bytecode violates JavaCard language specification or references proprietary packages.
3. **Upload** contents of the cap file to smartcard using the GlobalPlatform interface via gppro tool [72]. The step fails if cap file references unsupported packages or is rejected by the on-card verifier for other reasons.
4. **Install** the applet instance from the previously uploaded package (content of the cap file). The step fails if an exception is emitted in the applet's constructor, typically caused by an instantiation of an unsupported algorithm (e.g., ALG_SHA3_512), unsupported parameters (e.g., of LENGTH_RSA_4096 or invalid curve domain parameters), or excessive memory allocation (typically RAM). Occasionally, the exception may be caused by other programming mistakes or platform limitations (e.g., the limited size of the card's transaction buffer).
5. **Select** the installed applet instance, making it active for subsequent commands from a host controller. The step typically fails when additional custom code like delayed allocation or reset of the temporary state is executed.

Subsequent steps are performed only if the preceding succeeded. Steps 1 and 2 do not require a smartcard. Steps 3, 4, and 5 were performed on three smartcards and on a JavaCard simulator [73].

We first analyzed the possibility of performing applet conversion resulting in a cap file, but with load, install, and select steps performed only for the simulator. Such a test detects applets compatible with

some version of the JavaCard specification and can be converted into a valid cap file. Yet, no limitations of actual support for the required algorithms by the target physical card are imposed. Additionally, applets with programming errors that could be fixed easily (e.g., missing typecast from int to short) are listed in a separate column. The results are shown in Table 3.4.

Table 3.4: The number of applets with successful finalization of a particular development step on a JavaCard simulator [73].

Step	Count	(with fixes)
1) Compilation	174	187
2) Conversion (any JC SDK version)	145	174
3) Upload (does not apply)	–	–
4) Installation	121	150
5) Applet selection	111	144

Table 3.5: The portability of applets to different smartcards. The jCard-Sim simulator is listed as a theoretical upper bound of convertible and executable applets without resource limitations imposed by a particular physical platform. The numbers presented in this table also correspond to the applets with success/failure of measurement of entry point class constructor memory usage on physical smartcards. All *skips* correspond to applets that require a newer JavaCard API than supported by a given card.

Card	API	Succ.	Upload	Failure Install	Select	Skip
jCardSim	3.0.5	144	0	0	0	0
NXP JCOP4 J3R180	3.0.5	124	7	13	0	0
Feitian JavaCOS A22	3.0.4	98	3	41	0	2
G&D SmartCafe 7.0	3.0.4	64	3	73	1	2

Secondly, we tested the deployability and portability of applets among three different smartcards, all commonly available to open-source developers. The results are shown in Table 3.5, with the number

of applets deployable to the simulator serving as a baseline. The largest number of applets (124) are deployable to NXP JCOP4 J3R180, the best-performing smartcard from the database described in Section 3.4. The decreasing number of deployable applets for other smartcards illustrates the limited portability of applets among different vendors' smartcards.

3.7 Conclusion

In this chapter, we studied the JavaCard ecosystem with a focus on cryptographic algorithm support, performance, and trends in algorithm usage. The study was approached from three directions: 1) examination of public artifacts of CC and FIPS 140 security certifications, 2) empirical evaluation of capabilities and performance of real smartcards, and 3) analysis of open-source JavaCard applets.

The examination of certification artifacts has shown increased certification activity since 2006, with around twenty certified products every year for the last decade and taking 1 to 2 years before the first certified product's report mentions at least part of the newly released specification. However, the speed of adoption of its features is relatively slow, typically six or more years before a feature is commonly mentioned in certification documents.

The assessment of real smartcard capabilities on more than a hundred smartcard samples confirmed that the availability of those supporting newly issued algorithms is lagging, as only the latest smartcards available for purchase in small quantities started supporting JavaCard API 3.0.5, released already in 2015. Still, these few samples provide an optimistic outlook, as they are notably faster than their predecessors and support crucial features like plain EC operations (ALG_EC_SVDP_DH_PLAIN_XY, ALG_EC_PACE_GM) and RSA decryption with modulus sizes up to 4096 bits, enabling implementation of advanced protocols, yet with limited portability.

The issues with applet portability were highlighted by the analysis of JavaCard open-source projects. Of almost two hundred analyzed applets, only slightly more than half were installable on a real smartcard, and only around half of those could run on all three smartcard models we used for the testing.

PART II

**THRESHOLD CRYPTOGRAPHY ON
SECURE HARDWARE**

Overview

This part of the thesis focuses on the second research goal. It designs approaches for practical computation of threshold signature protocols of major elliptic-curve-based signature types (Schnorr [74], ECDSA [75], BLS [76]) on current JavaCard-based smartcards.

Prior work [15, 77, 78, 79, 80] has proposed several techniques for accelerating advanced protocols on smartcards by utilizing available co-processors. We extended known techniques and proposed new ones suitable for the asymmetric setting of a resource-constrained device connected to a more computationally powerful host. The acceleration techniques can be divided into four classes:

1. Accelerated computation using co-processors (Chapter 4).
2. Computation and storage offloading (Chapters 5, 7).
3. Secure preprocessing (Chapters 5, 6).
4. Optimized interoperable threshold schemes (Chapters 5, 7).

The first technique has been to a large extent investigated by prior work [15, 77, 78, 79, 80]. It is based on utilizing high-level operations provided by the JavaCard API to gain access to hardware-accelerated computation of low-level operations. The process of transforming algorithm inputs to the underlying computation brings inherent overhead, but in many instances, the result can be computed much more efficiently than if computed purely on the main processor within the context of the JavaCard virtual machine. In Chapter 4, we extended the previously known techniques both on the algorithmic level and by utilizing the new API introduced in more recent JavaCard versions.

The second technique utilizes the asymmetric relationship between smartcards and their host device, which is typically much more resourceful. Certain computations with public inputs whose results can be efficiently verified can be offloaded to the host device. The host can provide the computation result together with a proof of its validity to the smartcard which then efficiently verifies the result. We used this approach in Chapter 5 for point decompression and nonce caching, and in Chapter 7 for hash-to-curve computation.

The third technique utilizes an established paradigm from the field of secure multi-party computation in which a function evaluation can

be split into two phases: *offline* and *online*. The offline phase is computationally demanding but can be computed in advance and cached. The online phase depends on the message and cannot be precomputed, but its computation is relatively inexpensive. In Chapter 5, we utilize this technique in the construction of a smartcard-optimized threshold Schnorr signature scheme and also rely on it in Chapter 6 to enable practical computation of two-party ECDSA signatures.

Lastly, the fourth technique proposes a novel approach for combining different threshold protocols while maintaining their interoperability, possibly facilitated by an untrusted mediator. This allows for increased efficiency by utilizing simpler protocols (with potentially stronger security assumptions) on resource-constrained devices together with state-of-the-art constructions on other devices. Beyond the increased efficiency, this also allows increasing resiliency by adding another dimension to a system’s component variability, extending the concept by Mavroudis et al. [81]. We dedicate a major part of Chapter 5 to the interoperability of threshold Schnorr signature schemes and also utilize it to combine BLS with BDHKE in Chapter 7.

With these techniques, we demonstrated practical computation of all major elliptic-curve-based threshold signature classes. Table II summarizes the open-source JavaCard threshold signature implementations and maps the techniques used in their implementation.

Table II: Current open-source JavaCard implementations of threshold elliptic-curve-based protocols. The original scheme’s publication is referenced. If the implementation was part of this work, the corresponding section is also referenced. (*) Scheme without interoperability.

Scheme (this work’s section)	Type	Threshold	Techniques
Myst [81]	Schnorr	n	1
SHINE [A7] (5.5)	Schnorr	n	1, 4
FROST [16] (5.3)	Schnorr	t	1, 2
MultiSchnorr [14]	Schnorr	2	1, 4*
Lindell [20] (6.3.1)	ECDSA	2	1
Mult. triples [82] (6.3.2)	ECDSA	2	1, 3
Presignatures [83] (6.3.3)	ECDSA	t	1, 3
dBDHKE [A6] (7.4)	BLS	n	1, 2, 4

4 Enabling threshold signature computation

This chapter is based in part on our conference paper presented at ARES [A4] and nominated for the best paper award, and in part on our paper accepted to the ACNS conference [A5].

- [A4] A. Dufka and P. Švenda, “Enabling efficient threshold signature computation via Java Card API,” in *Proceedings of the 18th International Conference on Availability, Reliability and Security*, ser. ARES '23, Benevento, Italy: Association for Computing Machinery, 2023, pp. 1–10, ISBN: 9798400707728. doi: 10.1145/3600160.3600180.
 - [A5] A. Dufka, P. Laud, and P. Švenda, “Two-party ECDSA with JavaCard-based smartcards,” in *Applied Cryptography and Network Security: 23rd International Conference on Applied Cryptography and Network Security*, to appear, 2025, pp. 1–18.
-

4.1 Introduction

Threshold signature protocols are being increasingly deployed in practical applications to enhance the security of private keys by decentralizing their storage and eliminating single points of failure. In these protocols, a private key is split into multiple parts using secure secret sharing, with each part stored on a different device. As long as fewer than a specified threshold of devices are compromised, the private key remains protected. To generate a signature, the devices holding the key shares participate in an interactive protocol, during which they do not need to reveal their shares.

Since these protocols can produce signatures indistinguishable from regular, single-party ones, they are easily integrated with existing systems without introducing incompatible changes. Thanks to this property, threshold signatures are already deployed in applications including national identity scenarios [84] or transaction authorizations both in traditional finance and cryptocurrencies [85, 86]. Furthermore,

the recently published call for threshold scheme standardization by NIST [10] testifies to their increasing importance.

A commonly faced problem in the practical deployment of threshold schemes is that their implementations are not available on security-certified hardware devices that are otherwise used to handle secrets securely. Because of this, computation with the secret shares needs to be performed by a general-purpose processing unit, increasing the risk of their compromise. This situation raises the question of whether it is possible to compute threshold protocols with practical performance on the current security-certified hardware using interfaces they provide. In this part of the thesis, we focus on answering this question in the area of cryptographic smartcards, particularly those based on the JavaCard platform [53], being the most widely deployed ones.

JavaCard smartcards provide a generic programming interface capable of arbitrary computations yet too slow to achieve practical cryptographic computation at current security levels, especially for asymmetric cryptography. For that reason, the devices are typically equipped with cryptographic co-processors, accelerating particular operations needed by algorithms like RSA decryption or ECDH key agreement. While the high-level algorithms can be accessed via public JavaCard API, the underlying operations are not directly accessible, severely limiting smartcard's programmability. To address this issue, we analyzed the public high-level algorithms available on the latest smartcards and repurposed them to compute lower-level operations.

To identify which operations are commonly needed in threshold protocols, we analyzed state-of-the-art protocols for computing threshold Schnorr [16] and ECDSA signatures [20, 82, 87]. The protocols rely on a range of standard operations, including modular arithmetic, elliptic curve operations, and the use of hash functions, with differing requirements on the number of performed operations and the used key sizes. For each operation, we discuss how it can be (partially) accelerated with existing interfaces and implement the relevant extensions to the JCMATHLib library [15], a generic library constructing these operations. We provide measurements of the achieved performance of individual operations with different input sizes on six smartcards.

The rest of this chapter is organized as follows. After the introductory section, sections 4.2 and 4.3 present the related work and background, covering the JavaCard platform, secure secret sharing,

and threshold signatures. Section 4.4 describes the required operations used in threshold protocol implementations and how they can be computed via public JavaCard API. Section 4.5 presents the extensions to the JCMathLib library and their performance measurement. Section 4.6 discusses possible applications utilizing the unique combination of threshold schemes executed on smartcards, and conclusions are presented in Section 4.7.

4.2 Related work

Since the JavaCard platform's inception, its general programmability has attracted interest in implementing and running advanced cryptographic protocols on compatible smartcards. However, without utilizing hardware-based acceleration, not directly exposed by the public API, the implementations lacked performance for most practical public-key applications. The first successful works in this area achieved sufficient performance by repurposing the RSA co-processor to accelerate operations needed in computing anonymous credentials [78] and direct anonymous attestation [79].

As platform capabilities advanced with increases in memory, CPU frequency, and support for elliptic-curve-based operations and larger key sizes, implementing more complex cryptographic protocols became feasible. One of the earliest applications of elliptic curve support was Vullers' work on self-blindable credentials [80]. Building on these capabilities, Camenish et al. [88] developed privacy-enhancing attribute-based credentials. Balli et al. [89] leveraged elliptic curve features to create a suite of protocols enabling smartcards to function as biometric identification documents. Casanova et al. [90] utilized these advancements to implement revocable keyed-verification anonymous credentials.

In parallel to these applications, there have been attempts to create reusable libraries for the JavaCard platform, providing low-level operations needed for cryptographic implementations relying only on public JavaCard API. One of the first such projects is the implementation of modular arithmetic by Tews and Jacobs [77], who first published the method for accelerating modular multiplication by reusing RSA algorithm outputs. Their library was used as a foundation for

JCMathLib [15], which extended it by including support for elliptic curve operations, tools for improving memory management, and performance profiling. We further extended this library in Section 4.5 by new operations needed to implement threshold signature protocols.

In addition to privacy and identity management, JavaCard-based smartcards have emerged as a promising foundation for security-critical systems that distribute the storage of private keys. The first of such works was published by Mavroudis et al. [81], who proposed using a grid of smartcards for achieving backdoor tolerance, i.e., secure operation even in the presence of malicious parties. Their core idea is to use distributed key generation and threshold cryptography to run collaborative computation on smartcards produced by different manufacturers. Such a system remains secure as long as not too many of the devices are corrupted. After this publication, several works implementing threshold protocols on smartcards emerged [A4, A5, A6, A7].

4.3 Secret sharing and threshold protocols

In this section, we introduce the necessary background and notation for secure secret sharing and describe how it is involved in a threshold signature computation.

Secure secret sharing allows for splitting a secret into n shares in a way that at least a certain number t (called threshold) of the shares is needed to reconstruct it, while any set of less than t shares provides no information about the secret. Secure secret sharing of a secret s from the finite field of q elements $\mathbb{Z}_q$ is typically realized using Shamir's secret sharing [9]. In Shamir's secret sharing, a random polynomial $f(x)$ of degree $t - 1$ over $\mathbb{Z}_q$ is sampled so that $f(0) = s$, and the shares are computed as evaluations of the polynomial in points $1, \dots, n$.

Given a set $S = \{(x_1, f(x_1)), \dots, (x_t, f(x_t))\}$ of t evaluations of the polynomial in different points, it can be uniquely reconstructed using Lagrange interpolation as:

$$f(x) = \sum_{(x_i, f(x_i)) \in S} f(x_i) \lambda_i(x)$$

where

$$\lambda_i(x) = \frac{(x - x_1)}{(x_i - x_1)} \cdots \frac{(x - x_{i-1})}{(x_i - x_{i-1})} \cdot \frac{(x - x_{i+1})}{(x_i - x_{i+1})} \cdots \frac{(x - x_t)}{(x_i - x_t)}.$$

Evaluation of this reconstructed polynomial at 0 yields the secret. We use λ_i to denote $\lambda_i(0)$ in the rest of this chapter.

Threshold signature schemes allow a group of at least t signers holding secret shares of a private key to compute a signature verifiable with the public key corresponding to the securely shared private key without having to reconstruct it, while any set of less than t parties is not able to do so. Signatures produced this way may be indistinguishable from standard single-party signatures and thus compatible with applications that rely on signatures of a particular form.

There have been proposed threshold protocols for all practically used signature schemes; in this thesis, we focus on those based on elliptic curve cryptography, primarily ECDSA-based [20, 82, 91] and Schnorr-based [16] threshold signatures.

4.4 Required operations

In this section, we map the base operations required for the computation of state-of-the-art threshold Schnorr [16] and ECDSA [20, 82, 87] signature protocols and discuss how they can be efficiently computed via the public interface provided by the JavaCard platform.

4.4.1 Modular arithmetic

All the protocols need to perform modular arithmetic with a modulus of at least 256 bits for current security levels, with some protocols requiring 4096-bit or even larger moduli. The JavaCard platform does not provide any direct support for modular arithmetic with such sizes. It natively supports only word-sized arithmetic (typically 2- or 4-byte wide). Some implementations may optionally provide class `BigInteger` that allows performing basic operations with larger integers, but it is typically constrained to eight bytes and cannot perform modular arithmetic required by cryptographic implementations.

Therefore, a different approach has to be used to support modular arithmetic without relying on any proprietary interface. The basic

word-sized arithmetic may suffice for the computation of addition and subtraction, but multiplication and exponentiation require the aid of cryptographic co-processors to achieve practical performance. The co-processors are accessible only via high-level algorithms provided by the JavaCard API. Fortunately, outputs of these algorithms can often be repurposed for constructing more generic operations, though with an inherent overhead.

Addition and subtraction No algorithms in the JavaCard API enable modular addition and subtraction acceleration. However, as long as the inputs to the computation are smaller than the modulus, these operations can be computed efficiently with just the basic word-sized arithmetic.

Multiplication Modular multiplication can also be implemented using word-sized arithmetic; however, its quadratic complexity substantially affects computation time. JavaCard API provides two algorithms that allow accelerating modular multiplication: RSA decryption (ALG_RSA_NOPAD) and plain Diffie-Hellman key agreement (ALG_DH_PLAIN). Both these algorithms compute modular exponentiation, which is by an order of magnitude more complex than multiplication. However, since a dedicated co-processor performs this computation, it is able to outperform multiplication implemented with the basic arithmetic within a JavaCard virtual machine.

The ability to perform modular exponentiation does not immediately yield multiplication, but it can be transformed to it using the well-known binomial formula:

$$2ab \equiv (a + b)^2 - a^2 - b^2 \pmod{q}$$

using which the modular multiplication can be constructed at the cost of three squarings, one addition, two subtractions, and a division by two that can be efficiently realized by a bit shift. Counterintuitively, this computation is still faster than software implementation of modular multiplication and has been previously utilized in a number of works [15, 77, 78, 79]. Furthermore, Sterckx et al. [79] have proposed an alternative formula for the multiplication:

$$4ab \equiv (a + b)^2 - (a - b)^2 \pmod{q}$$

but by their experiments, the division by four has turned out to be slower than the additional squaring on some cards.

Inversion Computation of modular inversion is equivalent to the ability to perform modular exponentiation as by Fermat's little theorem $a \cdot a^{q-2} \equiv 1 \pmod{q}$ for prime q . The value $a^{q-2} \pmod{q}$ can be computed directly with the aforementioned algorithms for RSA decryption or plain Diffie-Hellman key agreement.

Modular reduction The JavaCard API does not provide any algorithm that would allow for direct acceleration of modular reduction, as all the algorithms require their inputs to be smaller than the modulus to run the algorithm. It is possible to perform this computation using accelerated modular multiplications with precomputed constants and modular additions but with significant overhead. Alternatively, this operation can be computed within the JavaCard virtual machine by performing remainder division with word-sized arithmetic. The complexity of this computation is quadratic in the difference between the bit lengths of the reduced number and the modulus, rendering it costly for large inputs.

4.4.2 Elliptic curves

The studied protocols need to perform operations with points on an elliptic curve. Scalar multiplication is always required due to its use in nonce point computation, but some protocols also require point additions [16]. Additionally, protocol specifications [17] usually require that points are encoded using SEC1 compressed encoding [92] or a similar compressed encoding in the case of curves like Ed25519 and Ed448, in which one of the point coordinates has to be reconstructed.

JavaCard platform supports specifying parameters of curves in the short Weierstrass form, allowing for the direct support of curves like P-256 and secp256k1. These curves can then be used with algorithms for plain elliptic curve Diffie-Hellman key agreement and the generic mapping computation according to specification TR-03110 [93]¹ used

1. Introduced in JavaCard API version 3.0.5.

in electronic passports, both of which output points without additional processing and thus can be used in further computations.

As any elliptic curve whose field characteristic is different from 2 and 3 can be mapped to a curve in the short Weierstrass form [94], it is possible to convert twisted Edwards curves like Ed25519 to their Weierstrass equivalents (like Wei25519 [95]) and use them with JavaCard algorithms. This will incur additional overhead with point transformations, but in some applications, the transformations can be trustlessly offloaded to a different device. Such an approach has already been demonstrated with JavaCard smartcards [96].

Starting with JavaCard API version 3.1 [97], a new API introducing a set of named elliptic curves became available. This API allows using curves like Ed25519 and Ed448 directly, but their usage is limited only to specific operations, e.g., EdDSA signing [98], which do not allow reusing them to derive more generic operations.

Point addition Algorithm ALG_EC_PACE_GM outputs the point $sP + Q$, where s is a scalar from $\mathbb{Z}_q$ and P, Q are points of E . Since the scalar can be set to 1, this algorithm can be used to perform point addition on smartcards that do support it. Furthermore, in case scalar multiplication and point addition operations need to be performed in a sequence in some application, using this algorithm, it can be achieved in a single API call without increasing the cost.

In case this algorithm is not supported, it is still possible to implement point addition using modular arithmetic over the elliptic curve's field, requiring several modular subtractions and multiplications, and one modular inversion. However, this approach is significantly more costly as the modular arithmetic is performed as described earlier.

Scalar multiplication The generic mapping from the previous paragraph can be repurposed even for plain scalar multiplication, though another slightly more efficient algorithm can be used just for this purpose: ALG_EC_SVDP_PLAIN_XY. The algorithm is intended to compute the elliptic curve Diffie-Hellman key agreement, but as it outputs the resulting point without any further processing, it can be directly used for scalar multiplication.

If the previous algorithms are not supported, it may still be possible to compute the result using `ALG_EC_SVDP_DH_PLAIN` algorithm, which, unlike the previous algorithms, outputs just the X coordinate of a point. In this case, it is necessary to compute the corresponding Y coordinate, but since there are two such values, additional checks are needed to determine the correct sign. This computation is quite costly, as it involves the computation of modular square root requiring modular multiplications and exponentiations. Still, this approach has already been successfully used [15, 80].

Point encoding and decoding Although JavaCard API supports SEC1 encoding [92] used with curves in the short Weierstrass form (e.g., P-256 and `secp256k1`), the support of compressed points is only optional and rarely available. Encoding of points used with twisted Edwards curves (e.g., Ed25519 and Ed448) is supported only for the named curves API, which does not seem usable for generic computations. In any case, both encoding and decoding can be implemented using modular arithmetic; however, decoding can be quite costly.

4.4.3 Hash functions

All the protocols also use a cryptographic hash function to process the message, bind the nonce, or compute various commitments within the protocol. The JavaCard API contains a wide range of standard cryptographic hash functions, including SHA-256 and SHA-512, and SHA-3. If a hash function is not supported by the JavaCard API on a particular card, it would need to be implemented on the level of the JavaCard virtual machine, making it notably slower.

4.5 Extensions of JCMathLib

We used the findings of the previous section to extend the JCMathLib library [15], an open-source library for the JavaCard platform providing modular arithmetic with large numbers and operations with points on elliptic curves. The library depends only on public JavaCard API, and it has already used some of the described approaches.

However, as the library was originally developed and tested on smartcards supporting only JavaCard API 3.0.1 and has not been expanded since, it lacked the support of new algorithms introduced in later versions, especially in version 3.0.5. These algorithms (e.g., `ALG_EC_SVDP_DH_PLAIN_XY` and `ALG_EC_PACE_GM`) facilitate much more efficient computation of point addition and scalar multiplication. Additionally, constraints on inputs passed to the `ALG_RSA_NOPAD` algorithm used for accelerated modular exponentiation have evolved, and its input preprocessing had to be reinvented for the latest smartcards. In the rest of this section, we give an overview of changes made to the library to enable the implementation of threshold signature protocols.

Backward-compatible extensions The main building block of most of the functionality of JCMathLib is the modular exponentiation utilizing the `ALG_RSA_NOPAD` algorithm. However, since the time of the original implementation, the constraints on inputs passed to the API have changed, and new cards could not run the library. There are minor differences in what various JavaCard implementations are able to accept, and it needs to be addressed on a per-card basis. We identified which input transformations are needed for the API call to succeed for each of our tested cards and changed the implementation to be able to accommodate all permissible execution paths, depending on which card it is being executed on. This way, we maintain backward compatibility with older smartcards and are able to support various constraints of the new ones.

To compile the code even with older versions of JavaCard SDK that do not support constants defined in later versions, we use the corresponding numbers directly. If an algorithm is not supported and thus cannot be instantiated, the code either selects an alternative implementation of the operation or fails. Furthermore, smartcards sometimes support algorithms introduced in later versions of JavaCard API even though they do not fully support this API yet, and they can be used this way. For example, this approach allows us to use efficient elliptic curves operations on an NXP J3H145 card with 3.0.4 API.

Compressed point encoding and decoding As none of our tested cards supported compressed points, we implemented this function-

ality. Encoding is straightforward, as it only needs to output the X coordinate prefixed with a value depending on the parity of Y . In contrast, decoding requires recomputing the Y coordinate, which involves costly modular square root computation and several modular multiplications and additions.

Hardware-accelerated point addition When JCMathLib was originally implemented, JavaCard API did not include the algorithm for computing the elliptic curve generic mapping (`ALG_EC_PAGE_GM`), and the point addition had to be computed using modular arithmetic. We integrated the new algorithm into the library to enable more efficient point addition on smartcards that support it, resulting in more than 7x speedup² on such cards. We also extended JCMathLib API to expose multiplication and addition in a single call so that its users can take advantage of a single JavaCard API invocation in their applications.

Faster scalar multiplication Similarly to the previous paragraph, the algorithm for elliptic curve Diffie-Hellman computation that outputs full point (`ALG_EC_SVDP_DH_PLAIN_XY`) was not available when the library was implemented; instead, the authors had to resort to recomputing Y coordinate with a significant impact on the performance. We integrated this new API call into the library and set it as the default option for scalar multiplication for smartcards that support it, resulting in more than 4x speedup² on tested cards.

Faster modular multiplication The original modular multiplication implementation in JCMathLib was based on the $2ab$ formula, and the computations were performed with a modulus of bit length greater than $(a + b)^2$, requiring an extra remainder division of the result. This was done for two reasons. The first reason is that constraints on inputs to the RSA algorithm on older smartcards caused modular squaring with a modulus of a lower bit length than the minimal allowed by the RSA algorithm³ to be inefficient, as each output would need to get reduced using the remainder division. The second reason is that

2. Measured on operations with points on an 256-bit elliptic curve.

3. The `ALG_RSA_NOPAD` algorithm usually requires at least 512- or 1024-bit modulus.

this way, the intermediary values would not get reduced in any of the squarings, obtaining even $2ab$, which could be bit-shifted to get ab .

For older smartcards, we changed this functionality to use the $4ab$ formula, as the unreduced variant does not suffer from the issue with slow division by four, reported by Sterckx et al. [79], as the bit shift can be performed directly by 2 bits. This change resulted in only 8% speed improvement² on tested cards, as the cost is still dominated by the remainder division.

For newer smartcards, we reimplemented the functionality so that the computations are performed directly mod q , lowering the cost of additions and subtractions and avoiding remainder division entirely. Furthermore, we based the computation on the $4ab$ formula and managed to avoid dividing by four using the following trick:

$$\begin{aligned} c &\equiv (a + b)/2 \pmod{q} \\ ab &\equiv c^2 - (c - b)^2 \pmod{q} \end{aligned}$$

With this approach, we were able to achieve more than 4.5x speedup² over the previous implementation, of which around 0.7x can be attributed to using the $4ab$ formula.

Extended modulus size support The JCMathLib library was designed to support modular arithmetic with integers up to the size of the elliptic curve group in use, typically 256 bits. However, the acceleration techniques implemented in the library can be applied to larger sizes, provided their support by the modular exponentiation co-processor. To leverage this capability, we added an option to initialize the library with integers up to 4096 bits, the maximal currently supported value. This feature significantly increases the library's memory requirements – up to 16 times more than with 256-bit integers. Consequently, when using this option, some internal variables do not fit within the available RAM and need to be allocated to persistent memory, causing performance degradation.

4.5.1 Achieved performance

We measured the performance achieved using the extended library on six smartcard models:

- | | |
|---------------------------|-----------------------|
| (1) G&D SmartCafe 6.0, | (4) NXP JCOP2 J2E145, |
| (2) G&D SmartCafe 7.0, | (5) NXP JCOP3 J3H145, |
| (3) Infineon Secora ID S, | (6) NXP JCOP4 J3R180. |

The measurements were performed from the host side by measuring the time it takes the card to reply to an APDU command that invokes the measured operation. Since this approach includes the overhead associated with the data transmission, we also measured it separately and subtracted it from the operation measurement. Each measurement was repeated 100 times with freshly sampled inputs, and the results were averaged.

Note that the performance of individual operations can significantly vary among smartcards. The differences in the underlying hardware naturally cause a part of the difference. However, the approach using which the operation is computed within the JCMathLib library can also vary, as it needs to accommodate different sets of supported algorithms and input preprocessing requirements. Furthermore, with configurations supporting 2048-bit or larger values, only some internal variables could fit into the available RAM, while others had to be kept in persistent memory and cause the associated performance penalty.

Table 4.1 presents the measured performance of operations with points on a 256-bit elliptic curve. The measurements show that on all tested smartcards, scalar multiplication and point addition can be computed within 6 seconds. However, smartcards that support efficient elliptic-curve operations (5-6) can compute the operations faster by an order of magnitude. This contrast is especially apparent in the combined operation of scalar multiplication followed by point addition, which can be computed on these cards in less than 250 ms.

Table 4.2 presents the performance of modular operations for different input sizes. The implementation on some smartcards supports only a subset of tested sizes, due to restrictions on maximal key sizes by co-processors and other API limitations, in which case the corresponding measurements are omitted. In those cases, it would still be possible to compute some of the operations, but the design choice of the library does not support its initialization unless the complete operation set is available. The modulus was set as the largest prime

4. ENABLING THRESHOLD SIGNATURE COMPUTATION

Table 4.1: Achieved performance of elliptic-curve operations with points on the P-256 curve in milliseconds (average of 100 repetitions).

Smartcard	(1)	(2)	(3)	(4)	(5)	(6)
API	3.0.1	3.0.4	3.0.5	3.0.1	3.0.4	3.0.5
Add	3194	2709	5280	3819	225	48
Mul	4224	3119	181	5641	185	39
Mul + Add	7382	5797	5396	9360	213	48

of the corresponding bit size. Modular addition and subtraction have very similar performance; therefore, we present only the former.

The results show significant performance differences between the older smartcards and the more recent ones (5-6). This is not only because of hardware differences but also due to varying input requirements for the modular exponentiation algorithm. Older smartcards seem to require more costly transformations of their inputs than newer models. One exception to this is smartcard (3), which supports API 3.0.5 but performs notably slower than other models. This is due to its modular exponentiation algorithm, which does not allow setting the exponent to two, thus preventing modular squaring – an essential operation for the used accelerations. This issue can likely be circumvented by additional transformations, which we leave as future work.

Table 4.2: Achieved performance of modular operations with varying input sizes on tested cards. Modular reductions were performed by a number with half the specified bit size without any acceleration.

Smartcard		(1)	(2)	(3)	(4)	(5)	(6)
API		3.0.1	3.0.4	3.0.5	3.0.1	3.0.4	3.0.5
256-bit	Add	43	33	46	60	39	25
	Mul	761	650	1046	877	213	165
	Sq	599	534	1036	678	30	35
	Exp	610	538	1589	681	42	36
	Inv	625	548	1620	707	53	43
	Red	234	213	223	248	282	146
512-bit	Add	66	50	65	78	64	37
	Mul	2262	1984	3694	2372	292	196
	Sq	2021	1827	3690	2146	26	28
	Exp	2111	1860	1926	2226	44	31
	Inv	2152	1877	1956	2279	57	38
	Red	782	692	761	786	906	471
1024-bit	Add	—	—	101	—	117	61
	Mul	—	—	3682	—	470	308
	Sq	—	—	3695	—	28	38
	Exp	—	—	89	—	112	54
	Inv	—	—	113	—	146	61
	Red	—	—	2699	—	3298	1757
2048-bit	Add	—	—	184	—	210	118
	Mul	—	—	14109	—	912	558
	Sq	—	—	14170	—	80	78
	Exp	—	—	354	—	622	191
	Inv	—	—	453	—	812	211
	Red	—	—	10434	—	12595	6454
4096-bit	Add	—	—	—	—	—	220
	Mul	—	—	—	—	—	1080
	Sq	—	—	—	—	—	173
	Exp	—	—	—	—	—	986
	Inv	—	—	—	—	—	1006
	Red	—	—	—	—	—	25758

4.6 Application scenarios

Disclaimer: Before discussing application scenarios for secure hardware components computing threshold protocols, we stress that the techniques presented in this work are intended to demonstrate that it is possible to efficiently compute operations underlying threshold signature protocols on current JavaCard-based smartcards while relying only on public JavaCard API. The library is not side-channel resistant and is susceptible to attacks utilizing such information, including basic timing side-channel attacks. Furthermore, no protection against fault induction was integrated. Consequently, this library shall not be used in production deployments.

Cryptocurrency hardware wallets Cryptocurrency hardware wallet designs have been utilizing multiple secure elements to prevent a signing key breach in case one of the elements turns out vulnerable [99]. However, the secure elements usually only store secrets in currently deployed devices and are not used for actual computations with the signing key, which is thus at increased risk of compromise once loaded into the device's memory.

Secure elements supporting threshold protocol computation could perform operations with key shares directly without loading them to an unprotected memory. As Bitcoin adopted a variant of Schnorr signatures [100], threshold Schnorr signature protocols became viable for this use case. We present such protocols and their implementation on JavaCard-based smartcards in Chapter 5.

Security of keys on smartphones Secure storage of keys on modern smartphones relies on specialized interfaces (e.g., Android Keystore system and iOS Secure Enclave), backed by secure hardware elements. While these interfaces support certain most commonly used cryptographic algorithms like ECDSA or RSA signatures, they do not support more recent or application-specific schemes like EdDSA [98] or BIP-Schnorr [100] signatures. If application developers need to use one of the unsupported schemes, they cannot benefit from these specialized interfaces and have to resort to less secured key storage methods.

The security of those approaches can be enhanced using smartcards and threshold signatures by splitting the key into two shares, one held by a smartphone and the other by a smartcard. Whenever the key needs to be used, a user taps the smartphone with the smartcard, which executes the signing protocol and outputs a signature. In order to obtain the signing key, an attacker would need to compromise both devices; thereby, the overall security would be increased. Even if the smartcard's implementation were compromised, it would not weaken the security beyond the original setup, and vice versa.

Threshold variants of less common elliptic-curve-based schemes (e.g., Schnorr [74] and BLS [76]) are typically more efficient and suitable for this use case, compared to the widely used ECDSA, and we study them in a broader context in later chapters. Still, two-party computation of ECDSA with smartcards is feasible, and we investigate it in Chapter 6.

Minimizing trust in service operators Smartcards are primarily used in the financial and telecommunication sectors to secure user keys. Users possessing smartcards are able to use them in operations without the keys being (easily) extractable – either by an unauthorized party or the user. This property can be used in a setting where, instead of an organization, a user has a secret that she wants some service operator to use on her behalf without sharing the secret directly. This need arises in applications with low-latency requirements, where the user is expected to react immediately, yet she may not always be online, for example, in cryptocurrency payment channel routing [101] and coin mixing [102].

However, giving sole control over a secret key to a service operator puts much trust in a single entity. The required level of trust can be decreased by decentralization, achieved using threshold protocols computed on smartcards provided to independent service operators. Such a setup reduces the risk of key misuse and increases resiliency in case some operators become unavailable. We explore a similar use case in Chapter 7.

4.7 Conclusion

In this chapter, we focused on the problem of acceleration of unsupported cryptographic computation on JavaCard-based smartcards. We mapped operations commonly required in threshold signature computation and proposed approaches to their partial acceleration. The core idea behind the partial acceleration is to repurpose outputs of high-level algorithms exposed by the JavaCard API that are accelerated on the hardware level. Although this approach may not deliver performance comparable with a native implementation using full access to the underlying hardware, it can be open-source, portable, and adequate for proof-of-concept projects.

Some of the discussed acceleration approaches were already used by the original JCMATHLib [15], a generic library providing modular arithmetic and elliptic curve point operations on smartcards. We extended the library with newly proposed approaches that utilize more recent JavaCard API algorithms and new algorithmic improvements. We also extended the library's compatibility with a broader range of smartcards with different input preprocessing requirements. All of these changes were merged with the project's upstream repository⁴.

We demonstrated the compatibility of the extended library with six different smartcards and measured its performance on them. The achieved performance on more recent smartcard models is practical for the computation of threshold protocols in many use cases. However, implementations on older smartcards will likely need additional protocol-specific optimizations to achieve practicality.

We hope these efforts enhance research and development of proof-of-concept applications utilizing smartcards and help the ecosystem to be more open. In this direction, we proposed three application scenarios utilizing a smartcards computing threshold protocols, which we further develop in subsequent chapters.

4. <https://github.com/OpenCryptoProject/JCMATHLib>.

5 Threshold Schnorr signature protocols

This chapter is based on our conference paper presented at ARES [A4] and nominated for the best paper award, and our prior publication presented at the SECURITY conference [A7].

- [A4] A. Dufka and P. Švenda, “Enabling efficient threshold signature computation via Java Card API,” in *Proceedings of the 18th International Conference on Availability, Reliability and Security*, ser. ARES '23, Benevento, Italy: Association for Computing Machinery, 2023, pp. 1–10, ISBN: 9798400707728. DOI: 10.1145/3600160.3600180.
 - [A7] A. Dufka, V. Sedláček, and P. Švenda, “SHINE: Resilience via practical interoperability of multi-party Schnorr signature schemes,” in *Proceedings of the 19th International Conference on Security and Cryptography - Volume 1: SECURITY*, S. D. C. di Vimercati, Ed., INSTICC, SciTePress, 2022, pp. 305–316, ISBN: 978-989-758-590-6. DOI: 10.5220/0011145600003283.
-

5.1 Introduction

Since the expiry of Schnorr’s patent [103], Schnorr signatures have made a considerable comeback, fueling digital signature specifications like EdDSA [98] and BIP-Schnorr [100]. EdDSA is being gradually incorporated into several major protocol implementations like TLS, SSH, Tor, and WireGuard [104]. BIP-Schnorr is now being used in Bitcoin [105] as a part of the Taproot consensus upgrade [106].

Practical problems these applications face have reignited research interest in threshold Schnorr signature protocols. New designs [16, 107, 108, 109, 110, 111] improved the practicality of the protocols, e.g., decreased the number of communication rounds, achieved concurrent security, and enabled security in the plain public-key model. Two of these new schemes are already specified within informational standards: the n -of- n protocol MuSig2 in BIP 327 [112] and t -of- n protocol FROST in RFC 9519 [17].

In this chapter, we study the new threshold Schnorr signature protocol designs with the aim of their computation on JavaCard-based smartcards. First, we analyze the FROST [16] protocol, implement it using the JCMathLib library in full compliance with its standard [17], and propose and include smartcard-specific optimizations, significantly improving its performance without introducing any incompatibility. Performance evaluation of the implementation on three different smartcards revealed that FROST can be computed efficiently on the latest models that provide elliptic curve algorithms that output plain points. However, its performance on other models is notably lower.

To investigate whether this performance difference could be decreased, we studied other threshold Schnorr signature designs and discovered they facilitate much faster computation. However, their compromises bring little benefit to less-constrained devices, making them undesirable in more diverse setups. This conflict raised the question of whether it is possible to design a smartcard-optimized protocol interoperable with other state-of-the-art designs.

To address this question, we study the differences between threshold Schnorr signature protocols and classify them based on their approach to the nonce agreement – a key component in signature computation. This classification allows us to investigate possible interoperability across classes and identify obstacles to in-class interoperability. In certain cases, cross-class interoperability can be achieved via an untrusted mediator that translates communication among different protocols.

Based on these insights, we design a new multi-signature scheme called SHINE, interoperable with several other Schnorr-based schemes and optimized for resource-constrained devices. We analyze the security of the scheme and evaluate its performance on six smartcards.

The rest of this chapter is structured as follows. After describing the relevant background in Section 5.2, we present the efficient implementation of FROST in Section 5.3. Section 5.4 starts with the interoperability study and compares different approaches to the nonce agreement. Building upon the lessons learned, we propose our scheme SHINE in Section 5.5 and draw conclusions in Section 5.6.

5.2 Background

This section introduces the notation used in this chapter, recalls Schnorr signatures, and presents relevant threshold schemes.

Notation A group description is a triplet $(\mathbb{G}, q, G)$, where q is a λ -bit prime, $\mathbb{G}$ is a cyclic group of order q , and G is a selected generator of $\mathbb{G}$. We use the additive notation for the group operation and denote group elements in upper-case. Conversely, we use lower-case to denote elements of $\mathbb{Z}_q$. A sampling of an element e from a non-empty set S is denoted as $e \leftarrow \$ S$. By $\mathcal{A}(x)$, we denote the set of outputs of probabilistic algorithm $\mathcal{A}$ given input x . We reserve t for the threshold and n for the total number of secret shares. Furthermore, with a secret s , we use the following notation:

- PRF_s – a pseudorandom function seeded with s ;
- KDF_s – a key derivation function seeded with s ;
- Enc_s – symmetric encryption with a key s ;
- Dec_s – symmetric decryption with a key s ;
- Com – a commitment function.

5.2.1 Schnorr signatures

The Schnorr signature scheme [74] is derived from the Schnorr identification scheme using the Fiat-Shamir transform [113] and relies on the hardness of the discrete logarithm problem. The scheme outputs efficiently computable and verifiable signatures of short length. It has been proven existentially unforgeable under the chosen message attack in the random oracle model [114]. Multiple definitions of the Schnorr signature schemes have been used. In this paper, we choose the one typically used in recent works [98, 100], as it supports efficient batch verification and prevents related-key attacks [115].

Definition 5.2.1 (Schnorr signature). Let $(\mathbb{G}, q, G)$ be a group description and $H : \mathbb{G}^2 \times \mathbb{Z}_q \rightarrow \mathbb{Z}_q$ be a hash function. A Schnorr signature of a message $m \in \mathbb{Z}_q$ verifiable with public key $X \in \mathbb{G}$ is a pair $(R, s) \in \mathbb{G} \times \mathbb{Z}_q$ satisfying the verification equation $sG = R + H(R, X, m)X$.

For a random nonce $r \in \mathbb{Z}_q$ and a private key $x \in \mathbb{Z}_q$ such that $xG = X$, a valid Schnorr signature of a message m is $(R, s) = (rG, r + H(R, X, m)x)$.

For a fixed $H(R, X, m)$, the signing equation is linear, which is convenient in constructing threshold Schnorr signature protocols. For example, a simple n -party protocol can be constructed as follows. First, all n parties agree on an aggregate nonce R , a linear combination of their individual contributions $R_i = r_iG$. Subsequently, they produce signature shares $s_i = r_i + H(R, X, m)x_i$, which are summed up to obtain the resulting signature $s = \sum_{i=1}^n s_i$, verifiable under the aggregate public key $X = \sum_{i=1}^n X_i$. This protocol has a few caveats, which cause it to be insecure in many use cases and are resolved by more complex designs. The two main security obstacles are related to the key aggregation and the nonce agreement.

The key aggregation is prone to *rogue-key attacks*, where the adversary computes her key as a function of the public keys of other parties and cancels out their contribution. To illustrate the problem, assume the attacker is the first party. She can compute her key as $X_1 = x'_1G - \sum_{i=2}^n X_i$ for some $x'_1 \in \mathbb{Z}_q$. When this rogue key is combined with the other keys, the resulting aggregate key corresponds to x'_1G , and the attacker can create signatures on behalf of the group.

Rogue-key attacks can be prevented by distributed key generation, which requires fresh key pairs like in Myst [81]. Alternatively, pre-existing keys can be reused when supplemented by proof of knowledge of their private key, e.g., MSDL [116]. Another method that supports key reuse is the non-interactive key aggregation method presented in MuSig [108], which avoids the attack by unpredictably altering the aggregate key whenever any of the inputs changes.

The other problem occurs in a concurrent setting. If sequential signing can be enforced, the protocol is secure, and the aggregate nonce does not even need to be computed by the signer, as is the case in CoSi [110]. However, if the signing instances with the same key can be executed concurrently (e.g., nonce contributions are shared in advance), the Drijvers et al.'s [117] attack can achieve signature forgery. The attack relies on solving an instance of the ROS problem [118], which can be solved in subexponential [119] or polynomial [120] time, depending on the number of concurrent sessions.

5.2.2 Current threshold signature protocols

This section described recent developments in the construction of threshold Schnorr signature protocols and mentions all protocols considered in the interoperability study.

CoSi [110] is a two-round n -party Schnorr signature protocol designed for high-speed signing by many parties organized into a tree structure. The scheme has been proven secure only for logarithmically many concurrent signing instances in a later work [117]. A similar scheme optimized for use with smartcards is Myst [81]. However, one of the optimizations the scheme uses (nonce caching) was found vulnerable to an attack by Drijvers et al. [117].

MuSig [108] was originally presented as a two-round protocol that was later found vulnerable by Drijvers et al. [117]. Earlier MSDL [116] used a preliminary commitment round that avoided the problem, and the same approach was also adopted to MuSig, resulting in a three-round concurrently-secure protocol.

MuSig-DN [121] was the first concurrently-secure two-round protocol resulting in standard Schnorr signatures which avoids the Drijvers et al.'s attack by generating the nonce deterministically. The nonce must be supplemented with costly non-interactive zero-knowledge proofs of its correct construction to achieve security.

MuSig2 [109] and DWMS [107] made advances in constructing secure two-round protocols with unlimited concurrency and, independently of each other, introduced a technique preventing the Drijvers et al.'s attack. This approach is much more efficient than deterministic nonce derivation with zero-knowledge proofs but it still presents a significant overhead, limiting its usefulness for constrained devices.

Crites et al. [111] combined MuSig2 with proofs of possession similar to MSDL to construct a Schnorr-based protocol called Speedy-MuSig. This combination brings faster key aggregation to the MuSig2 scheme, resulting in the fastest two-round concurrently-secure Schnorr n -party signature protocol.

FROST [16] is a threshold signature scheme that is secure for an arbitrary threshold $t \leq n$ and, as such, provides great flexibility to its applications. Its original version was also vulnerable to the Drijvers et al.'s attack, but a later version employed a variation of the technique used in MuSig2 [109] and DWMS [107] to avoid the issue.

Garillot et al. [122] presented another deterministic approach secure in the dishonest majority setting that has the benefit of not requiring additional randomness nor state. Its construction is conceptually similar to the MuSig-DN protocol, as it uses deterministic nonce derivation supplemented by non-interactive zero-knowledge proofs. The computation of the proof is more efficient than in MuSig-DN, but the proof size and thus communication requirements were significantly increased.

5.3 Efficient implementation of FROST

This section describes an efficient implementation of FROST, fully compatible with its standard [17]. The protocol consists of two rounds, the first of which can be securely precomputed, which is convenient for low-performance devices like smartcards or when communicating over slow networks. The second round, however, relies heavily on elliptic curve operations that need to be computed in dependence on the message and are computationally demanding. Our implementation is based on the JCMathLib library [15] with extensions as described in Chapter 4. Additionally, we proposed and incorporated smartcard-specific optimizations, significantly improving its performance without introducing any incompatibility. To the best of our knowledge, this is the first public open-source implementation of a threshold protocol on cryptographic smartcards supporting an arbitrary threshold.

5.3.1 Required operations

The specification [17] defines the scheme $\text{FROST}(E, H)$ parameterized by two values – an elliptic curve E and a hash function H , and describes five ciphersuites:

- $\text{FROST}(\text{Ed25519}, \text{SHA-512})$,
- $\text{FROST}(\text{Ed448}, \text{SHAKE256})$,
- $\text{FROST}(\text{ristretto255}, \text{SHA-512})$,
- $\text{FROST}(\text{P-256}, \text{SHA-256})$, and
- $\text{FROST}(\text{secp256k1}, \text{SHA-256})$.

We will discuss the required operations in general for $\text{FROST}(E, H)$ but highlight properties specific to particular instantiations when necessary.

Modular arithmetic Independently on a given parametrization, all instantiations of FROST need to perform modular arithmetic with at least 256-bit numbers. The modular arithmetic is used to construct the signature share of party i , defined as:

$$s_i = r_{0,i} + b_i r_{1,i} + \lambda_i e x_i \pmod{q}$$

where $r_{0,i}$ and $r_{1,i}$ represent the hiding and binding nonces generated by signer i ; the value b_i is its binding coefficient; the Lagrange coefficient λ_i corresponds to the signer's secret share x_i ; the signature challenge is denoted by e ; and q is the order of the elliptic curve group. From the expression, it is apparent that FROST implementations need to perform modular additions and multiplications. Furthermore, to compute λ_i the computation of modular subtraction and inversion is needed. Lastly, the computation of b_i and e involves hashing a bit string to a field element, requiring modular reductions in compliance with the specification.

Elliptic curves Depending on the given parametrization, an implementation of $\text{FROST}(E, H)$ needs to perform computations with points in the group of E . The most significant number of elliptic curve point operations is performed in the aggregation of a group commitment:

$$R = \sum_{i=1}^t R_{0,i} + \sum_{i=1}^t b_i R_{1,i}$$

where b_i is the same binding coefficient as in the signature share computation; $R_{0,i}$ and $R_{1,i}$ are hiding and binding commitments, corresponding to nonces generated by party i for its signature share. Apart from the group commitment aggregation, scalar multiplication is also needed for the computation of hiding and binding commitments by a signing party. Additionally, the specification [17] states that points should be encoded using SEC1 compressed encoding [92] or a similar compressed encoding in case of curves Ed25519, Ed448, and ristretto255, in which one of the point coordinates has to be reconstructed.

Hash functions FROST(E, H) uses the cryptographic hash function H in many contexts, separating each of them by a different domain separation tag. In total, the hash function is used in five parts of the implementation: three times to hash to a scalar in $\mathbb{Z}_q$ and two times as a hash to a byte string. The defined ciphersuites include three different hash functions: SHA-256, SHA-512, and SHAKE-256.

5.3.2 Implementation on smartcards

Based on the requirements, we decided to implement one of the ciphersuites that uses a curve in the short Weierstrass form without the need for transformations, to get performance closer to a native implementation, and SHA-256, as its support among available smartcards is more common and it is faster than SHA-512. Out of the two possible options FROST(P-256, SHA-256) and FROST(secp256k1, SHA-256), we decided to implement the latter. The switch to the other curve is only a matter of changing a few constants in the code. The resulting implementation¹ is compliant with the FROST specification [17] and was verified by the supplemented test vectors.

Optimizations

We first implemented the protocol exactly according to the specification [17], but for practical usage with smartcards, we included the following optimizations that maintain compatibility.

Decompressed points Since smartcards are always used in combination with a typically much more computationally powerful host device, the device may perform additional precomputations to decrease the computational load on the smartcard. This can be used in the case of point decompression. As discussed in the previous sections, point decompression requires a sequence of (for a smartcard) costly operations. We can rely on the host device to decompress points as they are being sent to the smartcard. This computation can be done without impacting security, as smartcard implementations should check the validity of points upon their loading into its `ECPublicKey` object.

1. <https://github.com/crocs-muni/JCFROST>.

Additionally, if the implementation were to use a ciphersuite that does not use an elliptic curve in the short Weierstrass form and thus needed to convert it, the host could perform the conversion along with the decompression, saving additional computation of the smartcard.

Constrained lambda computation Lagrange coefficient computation requires a number of modular multiplications linear in t , which is a significant factor for large signing groups. However, if the maximal number of parties is reasonably small, most of the intermediate multiplications can be performed without modular reductions (up to $n \leq 57$). Furthermore, with $n \leq 12$, the intermediate computations can fit within a 4-byte wide type, and with $n \leq 7$, within a 2-byte wide type. One additional subtraction needs to be performed if the result is negative, but it is a minimal price for the performance gain.

5.3.3 Performance evaluation

We measured the performance of the signing round of the implementation on three smartcards by NXP: JCOP4 J3R180, JCOP3 J3H145, and JCOP2 J2E145. The measurement was done using JCPProfilerNext [123], with each measurement repeated 100 times. All inputs to the protocol² were freshly sampled for each measurement and then fixed for the rest of the profiling to obtain precise results. We present performance results of the implementation with the described optimizations.

Table 5.1 shows the mean values of the measurements in dependence only on threshold t , as the computation is independent of the number of shares n . From the results, we can see that smartcards that support efficient computation with elliptic curves can produce a signature share within a few seconds for all measured values of t . In contrast, older smartcards with lower processing speeds that do not support these algorithms require more than 25 seconds to complete the computation even in the simplest case.

To identify how different parts of the computation contribute to the overall cost, we present Figure 5.1a (NXP JCOP4 J3R180) and Figure 5.1b (NXP JCOP3 J3H145). These figures display the computation time divided into its five components: binding factors computation,

2. Except for the message, which was fixed to the string "frost".

Table 5.1: Time (ms) required to compute signature share on the analyzed smartcards in dependence on threshold t .

t	JCOP4 J3R180	JCOP3 J3H145	JCOP2 J2E145
2	1290	2481	25724
3	1649	3219	38435
4	1940	4137	51180
5	2297	4933	63934
6	2597	5682	76597
7	2956	6481	89523
8	3257	7276	—
9	3544	8023	—
10	3902	8798	—
11	4196	9610	—
12	4556	10352	—

group commitment computation, lambda computation, challenge computation, and signature share computation.

The figures show that computations of the resulting signature share and challenge are not correlated with t , thus setting a lower bound on the computation time. It may appear that the computation of lambda (the Lagrange interpolation coefficient λ_i) is also uncorrelated with t , but that is only due to the used optimization that allows most of the computation to be performed only with the platform's native type.

Group commitment computation is performing on the native performance level on both J3R180 and J3H145 cards, as most of the computational load of point addition and multiplication is realized by the underlying hardware-accelerated algorithms. However, this computation is a major bottleneck for older cards that do not support efficient elliptic curve operations.

Besides the operations with elliptic curve points, binding factor computation is the most demanding component. The computation requires deriving a binding coefficient for each of the t parties by processing inputs to the protocol using H and then mapping the result to a scalar in $\mathbb{Z}_q$, requiring costly modular reduction. These operations cannot be preprocessed as they rely on fresh inputs, and the modular reduction cannot be avoided without diverging from the specification.

5. THRESHOLD SCHNORR SIGNATURE PROTOCOLS

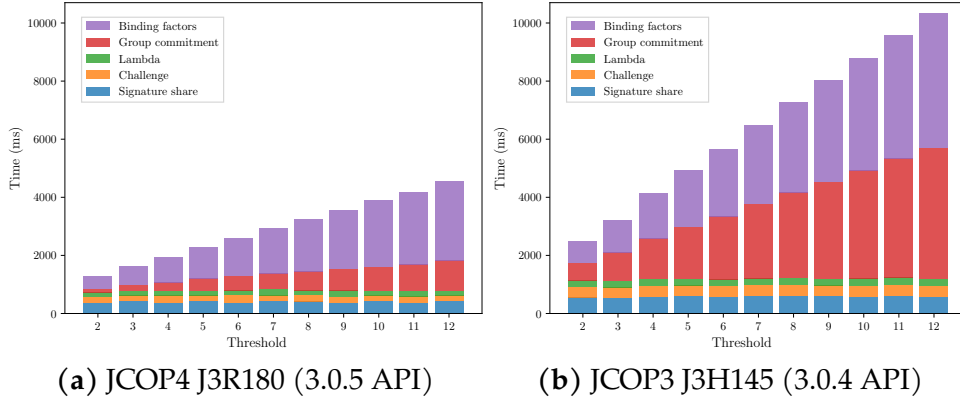


Figure 5.1: Time measurement (average of 100 runs, in milliseconds) of signing round components with different threshold values on smart-cards that support efficient elliptic curve operations.

The performance of hash computations could be improved by storing the internal state of the hashing object and reusing it, since many of the computations share the same prefix. However, this cannot be done with `MessageDigest` class provided by the JavaCard API.

5.4 Interoperability of threshold Schnorr protocols

The Schnorr-based protocols discussed in Section 5.2.2 exhibit different trade-offs. Some are optimized for a low number of communication rounds; others are better suited for limited devices where the computation is costly; some use only standard operations available on legacy systems or can utilize dedicated co-processors, and some need to use non-standard cryptographic primitives. As a result, none of the schemes is ideally suitable for all platforms.

In this section, we attempt to address the problem of protocol heterogeneity. We surveyed current threshold Schnorr signature designs and classified them based on their approach to the nonce agreement. With this classification, we specify what is required of the protocols from the same class to be compatible with each other. Furthermore, we inspect the differences among the classes and bridge them using an untrusted third party without any changes to the underlying schemes. If this mediation is possible, we call the schemes *interoperable*.

More precisely, we define *interoperability* as the ability of two or more threshold signature protocols to execute jointly via an untrusted mediator in a way that results in a valid signature on behalf of all of the parties and none of the parties can distinguish such an execution from the execution with its own instances.

With this definition, the security of interoperability can be reduced to the security of individual protocols. Since all the considered protocols were proven secure in the dishonest majority setting, their security does not rely on the actions of other participants. In particular, the mediator can be considered the adversary in the security proofs.

Nonce agreement is the main part in computing threshold Schnorr signatures. All signing parties must contribute to it their fresh nonce, which they later reflect in signing. After the nonce is known, the signatures can be computed non-interactively.

We have identified four main approaches to nonce agreement: 1) nonce exchange, 2) nonce commitment, 3) nonce delinearization, and 4) deterministic nonce derivation. These methods differ in the number of communication rounds, computational complexity, and security assumptions. The following subsections analyze the approaches and describe the mediator for each interoperable pair.

5.4.1 Nonce exchange

Nonce exchange (NE) features two communication rounds and is the simplest and most efficient approach. It is used by applications focusing on high performance [81, 110], which utilize that its security does not rely on a specific construction of the aggregate nonce. Thus the signers do not even have to compute the aggregate nonce themselves, allowing for a further decrease in the computation requirements.

The disadvantage of this approach is that it is secure only when executed with limited concurrency, i.e., sequentially or with a logarithmic number of concurrent sessions. Otherwise, a practical message forgery can be achieved by Drijvers et al.'s attack [117].

Principle Each signer i samples a random nonce $r_i \leftarrow \mathbb{Z}_q$, computes the corresponding element $R_i = r_i G$, and transmits this element. The elements of all signers are then summed up into the aggregate nonce $R = \sum_{i=1}^n R_i$ used in the signing.

Interoperability Since there are no constraints on the construction of the aggregate nonce, NE schemes are convenient for achieving interoperability with other nonce agreement approaches that are more restrictive but concurrently secure.

5.4.2 Nonce commitment

Nonce commitment (NC) is a three-round approach that has been used by MSDL [116] and MuSig [108]. It provides concurrent security with only a minimal computation overhead over NE. If the communication latency is not too high, this approach is also quite efficient and suitable for computationally-limited devices.

Principle The Drijvers et al.’s attack requires the attacker to be able to choose their nonce depending on the nonces of other parties³. This precondition can be broken by a preliminary communication round, in which each signer i first outputs a commitment to its nonce element $\text{Com}(R_i)$, and only after receiving commitments from all other parties can the nonce element R_i be revealed. The provided nonce elements need to be verified against the commitments, and if an inconsistency is discovered, the protocol must be aborted.

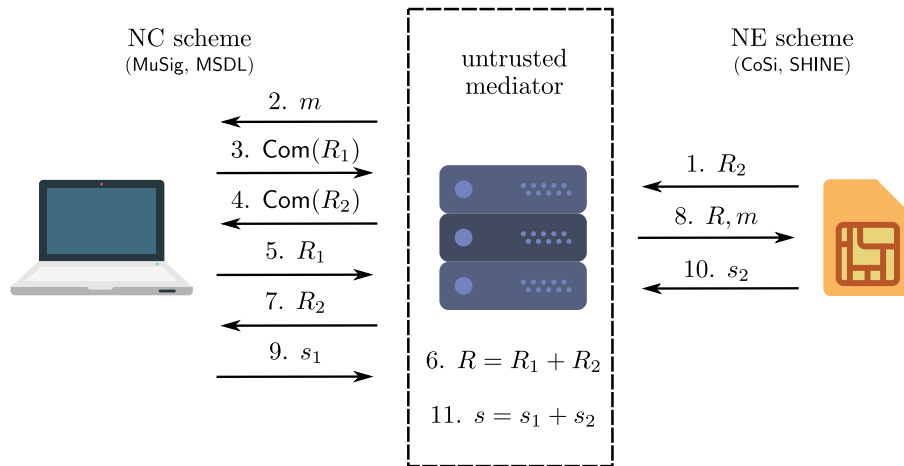


Figure 5.2: Interoperability mediation between NC and NE schemes.

3. Assuming other parts of the input are already fixed.

Interoperability The need for commitment limits interoperability among different instances of NC. To work together, the schemes have to use the same Com function, which is a consequence of the commitment properties, preventing it from being readjusted by other parties.

Nonetheless, NC is interoperable with NE-based schemes via a translation layer that simulates the commitment round on behalf of the NE schemes as follows (see Figure 5.2). First, the NE schemes share their R_i . The translation layer computes commitments for these elements with an appropriate Com function and simulates the commitment round with NC schemes. Afterward, the NC schemes and the translation layer (on behalf of NE schemes) reveal R_i that successfully verify against the commitments. Hence, no party aborts, and they all arrive at the same aggregate nonce.

5.4.3 Nonce delinearization

Nonce delinearization (ND) is the most recent approach to the nonce agreement that is secure under concurrent execution with just two communication rounds, the first of which can be preprocessed. The main downside of this approach is its high computational cost, as it requires each signer to generate multiple nonces and to perform multi-scalar multiplication in the second round. Nonetheless, the practical benefits in many applications outweigh the cost, and this technique has been used in the design of the latest schemes [16, 107, 109, 111].

Principle Each signer i uniformly samples ν nonces $r_{i,j}$ and reveals the corresponding elements $R_{i,j} = r_{i,j}G$. When the message to be signed is known, the nonce elements are used to compute the aggregate nonce

$$R = \sum_{i=1}^n \sum_{j=1}^{\nu} \beta_{i,j} R_{i,j},$$

where $\beta_{i,j}$ are *delinearization coefficients*. The delinearization coefficients are non-linearly dependent on all nonce elements and the message via a hash function, which causes the aggregate nonce to change unpredictably whenever any of the inputs changes, and thus thwarts the Drijvers et al.'s attack.

5. THRESHOLD SCHNORR SIGNATURE PROTOCOLS

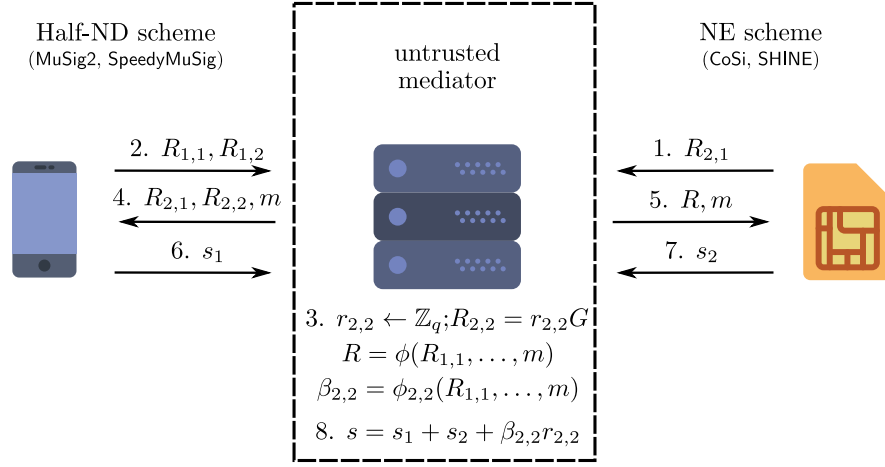


Figure 5.3: Interoperability mediation between half-ND and NE schemes.

Interoperability The requirement of specific nonce aggregation derived from pre-shared nonce elements and the message limits the interoperability with other instances of ND schemes, as the same coefficients $\beta_{i,j}$ and ν would need to be used. Schemes using, e.g., a different hash function in the coefficient computation do not arrive at the same aggregate nonce.

Interoperability with NE schemes is a bit more nuanced and cannot be achieved in general. Equations (5.1) and (5.2) show a signature by an NE scheme and an ND scheme, respectively ($\nu = 2$ for brevity).

$$s = r_i + ex_i \quad (5.1)$$

$$s = \beta_{i,1}r_{i,1} + \beta_{i,2}r_{i,2} + ex_i \quad (5.2)$$

The mediator cannot reconcile the difference between (5.1) and (5.2) because it cannot multiply the nonce r_i without also changing the ex_i component. However, if $\beta_{i,1} = 1$, the interoperability is achievable via the following mediation (see Figure 5.3).

First, all signers begin by sharing their nonce elements. The single nonce element provided by NE schemes is used as their first nonce element, and the mediator computes the other nonce elements instead of the signers. These simulated nonce elements can be sent to ND-based signers, who can now compute the aggregate nonce. The

aggregate nonce R is also computed by the mediator who provides it to NE-based signers, which reply with their signatures $s_i = r_i + ex_i$. The mediator then augments the signatures by the simulated nonces $s_i + \beta_{i,2}r_{i,2} + \dots + \beta_{i,v}r_{i,v}$, making them compatible with signatures of ND schemes. Finally, the signature shares can be combined into a valid signature without changing the underlying schemes.

Having $\beta_{i,1} = 1$ has been suggested as an optimization of MuSig2 [109] that became the default choice in a later revision of the scheme and was since then adopted by other works [16, 111]. We call this variation, where the first nonce is not multiplied by the coefficient, *half-nonce delinearization*. This variant is not a mere performance optimization, but it enables interoperability with NE schemes, as illustrated above. For this reason, we suggest preferring the designs of schemes with half-ND. This choice still allows for adding arbitrarily many nonces to tweak the problem⁴ while remaining interoperable with NE schemes and incurring no additional cost to them.

Achieving interoperability is not possible with NC schemes, as those schemes need to receive a commitment to the single nonce produced by each party before revealing their nonce. However, ND schemes use multiple nonces and cannot combine them before all other nonces are known.

5.4.4 Deterministic nonce

Deterministic nonce (DN) derivation is another approach to concurrently secure two-round scheme construction. DN derivation has been used in standard signature schemes, preventing attacks due to biased randomness in the nonce generation. However, it cannot be directly applied to a multi-party setting as a malicious party that diverges from the correct computation could force an honest signer to reuse a nonce, which would result in a key compromise. To deal with this problem, MuSig-DN [121] and a scheme by Garrilot et al. [122] use non-interactive zero-knowledge proofs. The disadvantage of this approach is its computational cost, which is the highest among the presented approaches.

4. A MuSig2 variant suggested four nonces.

Table 5.2: Interoperability of nonce agreement approaches. Icons ✓, ✗, and ~, denote whether achieving interoperability is possible always, never, or under certain preconditions, respectively.

	NE	NC	ND	DN
NE	✓	✓	~	✗
NC	✓	~	✗	✗
ND	~	✗	~	✗
DN	✗	✗	✗	~

Principle Each signer derives its nonce using a deterministic computation and constructs a non-interactive zero-knowledge proof of its correct construction. The proof is output along with the nonce element, and other protocol participants need to verify the proof. If the verification fails, the signing must be aborted.

Interoperability The need for a zero-knowledge proof severely limits the interoperability of the scheme. Different implementations of DN schemes require a consensus on the used derivation function and the proof construction to work together. Interoperability with schemes using a different approach to the nonce agreement is impossible, as it contradicts the requirement of deterministic nonce derivation.

5.4.5 Summary

Table 5.2 displays the interoperability matrix of the nonce agreement approaches. Schemes based on NE stand out among others as the most flexible due to their ability to accept an externally provided nonce without any knowledge of its construction and remain secure with a sequential execution. The interactions between the NE and ND schemes are interoperable when the half-ND method is used. Even though NE schemes are interoperable with NC schemes and half-ND schemes separately, they cannot be used together since NC schemes and ND schemes are not interoperable. DN is interoperable only with its instances.

Based on the study of interoperability, we propose to focus on three designs of Schnorr-based schemes:

1. The sequentially constrained NE schemes for computationally restricted devices which benefit the most from the efficiency;
2. the half-ND schemes for devices where the computation is not a limiting factor, and that could benefit from interoperability;
3. the DN schemes for the cases where the performance is not an issue, determinism is crucial, and a setup of homogenous instances is guaranteed.

We followed up on our first recommendation and used NE to design SHINE, an n -party Schnorr signature scheme optimized for cryptographic smartcards and similar resource-constrained devices, which is interoperable with other nonce agreement approaches. We present the design in the next section.

Lastly, it remains to be noted that theoretical interoperability does not automatically imply interoperability of corresponding implementations. For that, the implementations need to use compatible private key shares and produce compatible signatures – not only Schnorr signatures but the same instance of Schnorr signatures, e.g., Ed25519 [98] or BIP-Schnorr [100].

5.5 New signature protocol SHINE

This section describes SHINE (Smartcard Highly-Interoperable Nonce Encryption scheme), an n -party scheme optimized for computations on cryptographic smartcards while being interoperable with certain prior threshold Schnorr schemes. The design includes a central party that mediates communication among individual signers. We utilize this central party for the precomputation of inputs, data storage, and also for achieving interoperability, as described in the previous section.

SHINE can create a signature in two communication rounds, the first of which can be precomputed. The scheme uses a variant of NE that enables interoperability with all classes except DN. However, it also requires sequential execution to be secure, which we enforce by design. Additionally, to avoid randomness generation failure attacks and minimize storage requirements, we derive nonce using a secret pseudorandom function that depends on an internal counter.

In the following subsection, we describe the SHINE's distributed key generation. Next, we introduce the technique of nonce caching with encryption and discuss its differences from plain nonce caching. We describe the signing protocol in Section 5.5.3 and analyze its security in Section 5.5.4. The last section presents an implementation of SHINE on cryptographic smartcards and evaluates its performance.

5.5.1 Setup

Before SHINE can be used to sign messages, the signing group needs to be established. The signing group consists of a set of signers who generate their private key shares, compute the aggregate key, and initialize context information. In this process, we assume that the central party included all participants in the key aggregation, i.e., did not ignore or simulate messages by some parties. In practice, the validity of this assumption can be later verified by querying participants for their key contributions by secondary channels.

SHINE can adapt to key aggregation approaches of protocols with which it should be interoperable. As the default option, we have selected proofs of possession, as it allows efficient group key aggregation and low communication. Alternatively, MuSig key aggregation [108] or distributed key generation can be used instead.

After the key is aggregated, each party i finalizes the group establishment process by initializing its signing context. The signing context consists of a λ -bit secret p_i that is used for deterministic nonce and encryption key derivation and an increase-only counter c_i that tracks the index of the most recently used nonce. The former value is uniformly sampled, and the latter is initially set to zero. These values are later used in the signing protocol.

5.5.2 Nonce caching with encryption

Nonce caching is a technique used in Myst [81] by a setup of multiple smartcards. It optimizes the signing speed by generating nonces in advance, computing the corresponding elements, and storing them on a server. Thus, the costly scalar multiplication can be performed during downtime when there are no signing requests, and the precomputed nonce can be provided to signers when needed.

This approach was later shown to be vulnerable to the Drijvers et al.'s attack [117]. Since the properties of nonce caching are beneficial for resource-limited devices like smartcards, we designed a technique called *nonce caching with encryption*, or *nonce encryption* for short, which avoids the Drijvers et al.'s attack and still allows for nonce caching with some restrictions.

The vulnerability to Drijvers et al.'s attack occurs only if the scheme is used concurrently, e.g., an adversary can open multiple signing sessions in parallel or cache multiple nonces before they are used for signing. Nonce encryption leverages this property and enforces sequential execution to ensure, at most, a single cached nonce is revealed at a time while still being usable for signing.

Nonce encryption consists of two phases – Cache and Reveal (see Figure 5.4). The smartcard computes the nonce element during the caching phase and sends it encrypted⁵ using a fresh key to the central party. The reveal phase is then used to reveal the corresponding decryption key, thus revealing the cached nonce while invalidating previous nonces.

The Cache phase is initiated by the central party, which sends a signing instance identifier j . The smartcard i uses a pseudorandom function, keyed with a secret p_i (generated during the group establishment), to derive a nonce $r_{i,j}$ corresponding to the signing instance j . Then, the smartcard computes $R_{i,j} = r_{i,j}G$, the demanding operation that would be the bottleneck during signing.

In the standard nonce caching, $R_{i,j}$ would be simply transmitted to the central party; however, if this step was repeated, it would lead to the vulnerability to the Drijvers et al.'s attack. With nonce encryption, the nonce is not transmitted in plain but rather encrypted with symmetric key $k_{i,j}$ derived from the signing instance identifier j using a key derivation function known only to the signer i . The central party stores the received value for future use.

The Reveal phase uses the smartcard's internal counter c_i to track which nonce elements were already revealed. When tasked by the central party to send $k_{i,j}$ for a signing instance $j \geq c_i$, the smartcard

5. We use encryption only to securely store the cached element on the central party to minimize storage requirements on the signing device (as memory on smartcards is limited). There is no need to bind to the encrypted value.

derives $k_{i,j}$ and sends it back, but at the same time increases its inner counter $c_i = j$. With the knowledge of $k_{i,j}$, the central party can decrypt the cached nonce $R_{i,j}$, possibly combine it with nonces of other parties, and use it in a subsequent signing.

So far, the counter c_i did not limit the attacker in any way. It is used only during signing, in which the smartcard must not produce a signature for a signing instance $j < c_i$. This restriction ensures that only the signing instance $j = c_i$ can succeed since nonces for $j > c_i$ are not known yet. This already enforces the signing of non-decreasing sequences, and it only remains to avoid nonce reuse, which is achieved by incrementing the internal counter c_i when producing a signature.

Comparison to plain nonce caching

Just as in plain nonce caching, we avoid costly group operations during signing. In this subsection, we highlight the differences.

Storage-wise, nonce encryption still requires only constant memory on smartcards, but the central party cannot pre-aggregate nonces anymore, as only a single nonce is known at a time. Therefore, the space required by the central party grows linearly in n .

Communication-wise, nonce encryption seemingly needs one additional round to transmit the decryption key. However, this transmission can be automatically piggybacked with the previous signing operation, where the decryption key for $c_i + 1$ can be revealed, as c_i was already invalidated. Consequently, the number of rounds can remain the same; only the additional decryption key is transmitted.

Finally, computation-wise, symmetric encryption can be realized efficiently, minimizing the additional demands on smartcards. The only new concern is the aggregate nonce computation by the central party – which cannot be precomputed, as only a single decryption key is known at a time. In the case of burst signing requests, the overall solution would result in a performance decrease compared to the vulnerable nonce caching. However, assuming the central party is significantly more resourceful than the smartcards, the aggregation can be performed relatively quickly and is not a limiting factor.

Cache(j)	Reveal(j)	Sign(R_j, m, j)
$r_{i,j} = \text{PRF}_{p_i}(j)$	if $j \geq c_i$ then $c_i = j$ fi	if $j < c_i$ then abort fi
$R_{i,j} = r_{i,j}G$	$k_{i,j} = \text{KDF}_{p_i}(c_i)$	$c_i = j + 1$
$k_{i,j} = \text{KDF}_{p_i}(j)$	return $k_{i,j}$	$r_{i,j} = \text{PRF}_{p_i}(j)$
$E_{i,j} = \text{Enc}_{k_{i,j}}(R_{i,j})$		$s_{i,j} = r_{i,j} + H(R_j, X, m)x_i$
return $E_{i,j}$		return $s_{i,j}$

Figure 5.4: Definition of SHINE procedures.

5.5.3 Signing protocol

The last procedure in Figure 5.4 shows the signing algorithm. The central party initiates the signing of a message m by computing the aggregate nonce. If it already has all appropriate decryption keys, it sums the cached nonces as $R_j = \sum_{i=1}^n R_{i,j}$. Otherwise, the central party might need to exchange an additional Cache or Reveal message with some smartcards prior to the aggregation. When the aggregate nonce is computed, the central party sends a signature request to every participating smartcard with the given signing index j , the aggregate nonce R_j , and the message m .

When a smartcard receives a signing request, it first checks whether the signing index is greater or equal to its internal counter. If not, it aborts the protocol. Otherwise, the smartcard sets its internal counter $c_i = j + 1$ and continues with the signing. It derives its nonce $r_{i,j}$ and computes its signature $s_{i,j} = r_{i,j} + H(R_j, X, m)x_i$, which it outputs. Optionally, the decryption key for index $j + 1$ can be revealed together with the signature. Finally, the central party sums the signatures to obtain the resulting (R_j, s_j) .

5.5.4 Security analysis

In this section, we analyze the security of SHINE by reducing it to the one-more discrete logarithm problem [124] in the random oracle model [125]. The problem can be summarized as follows. The adversary has access to an oracle $\mathcal{O}^{\text{DLog}}$ providing them with up to q_d

discrete logarithms, while the goal is presenting discrete logarithms of $q_d + 1$ challenges received from the $\mathcal{O}^{\text{Chal}}$ oracle.

In the analysis, we use the following algorithms:

- GrGen that outputs a group description $(\mathbb{G}, q, G)$ for a given security parameter λ ,
- KeyGen that outputs a uniformly sampled key pair (x_1, x_1G) ,
- KeyAgg that aggregates key according to the selected approach,
- Verify that performs standard Schnorr signature verification.

$\text{EUF-CMA}_{\text{SHINE}}^A(\lambda)$	$\mathcal{O}^{\text{Reveal}}(j)$
$(\mathbb{G}, q, G) \leftarrow \text{GrGen}(1^\lambda)$ $p_1 \leftarrow \mathbb{Z}_q; c \leftarrow 0; Q \leftarrow \{\}$ $(x_1, X_1) \leftarrow \text{KeyGen}()$ $X_2, \dots, X_n \leftarrow \mathcal{A}(X_1)$ $X \leftarrow \text{KeyAgg}(X_1, \dots, X_n)$ $(m^*, \sigma^*) \leftarrow \mathcal{A}^{\mathcal{O}^{\text{Cache, Reveal, Sign, H}}}(X_1)$ return $m^* \notin Q \wedge \text{Verify}(X, m^*, \sigma^*)$	if $j \geq c$ then $c \leftarrow j$ fi $k_{1,j} \leftarrow \text{KDF}_{p_1}(c)$ return $k_{1,j}$
$\mathcal{O}^{\text{Cache}}(j)$	
$r_{1,j} \leftarrow \text{PRF}_{p_1}(j)$ $R_{1,j} \leftarrow r_{1,j}G$ $k_{1,j} \leftarrow \text{KDF}_{p_1}(j)$ $E_{1,j} \leftarrow \text{Enc}_{k_{1,j}}(R_{1,j})$ return $E_{1,j}$	if $j < c$ then abort fi $c \leftarrow j + 1$ $r_{1,j} \leftarrow \text{PRF}_{p_1}(j)$ $s_{1,j} \leftarrow r_{1,j} + H(R, X, m)x_1$ $Q \leftarrow Q \cup \{m\}$ return $s_{1,j}$

Figure 5.5: The SHINE unforgeability game. $\mathcal{O}^H$ is the random oracle.

Theorem 1. If a polynomial adversary $\mathcal{A}$ against the SHINE unforgeability game (Figure 5.5) wins with probability ε while making q_S

reveal oracle queries and $q_H - q_S$ random oracle queries, then the OMDL problem can be solved in polynomial time with probability ε' in the ROM with the KOSK assumption, such that:

$$\varepsilon' \geq \frac{\varepsilon}{8q_H q_S} - \frac{q_S^2 k_{\max}^2}{q},$$

where $k_{\max} = 8q_H / \varepsilon \cdot \ln(8n / \varepsilon)$.

Proof. The proof adapts the main idea of the reduction given by Drijvers et al. [117] for the restricted version of CoSi. Given the adversary $\mathcal{A}$ from the theorem statement, we construct an algorithm $\mathcal{B}$ that simulates the SHINE unforgeability game for $\mathcal{A}$, and succeeds when it does not abort and $\mathcal{A}$ succeeds. The algorithm $\mathcal{B}$ is constructed in a way compatible with the forking lemma, which we then apply to $\mathcal{B}$ to solve the OMDL problem.

The algorithm $\mathcal{B}$ is started with elements from $\mathbb{Z}_q$ for the random oracle outputs $(h_1, \dots, h_{q_H})$ and is given access to oracles $\mathcal{O}^{\text{Chal}}$ and $\mathcal{O}^{\text{DLog}}$. We sequentially order the $\mathcal{O}^{\text{Reveal}}$ and $\mathcal{O}^H$ oracle queries made by $\mathcal{A}$. If the k -th query is a random oracle query, $\mathcal{B}$ replies with h_k . If k -th query is a reveal query, we set $k_j = k$, where j is the input to the query. We also modify $\mathcal{O}^{\text{Chal}}$ to accept an input element from $\mathbb{Z}_q$. When the oracle is queried on a previously unseen element, it invokes its inner $\mathcal{O}^{\text{Chal}}$ oracle; otherwise, it outputs the previously output value. Without loss of generality, we assume that $\mathcal{A}$ makes all random oracle queries needed to verify the produced signatures.

First, $\mathcal{B}$ makes a minor modification of nonce caching in the simulation, allowing us to defer nonce sampling to the time when the corresponding decryption key should be revealed – as per SHINE’s sequential nature. The change is sampling a random string $E_{1,j}$ in the $\mathcal{O}^{\text{Cache}}$ oracle instead of computing and encrypting $R_{1,j}$. The value $R_{1,j}$ is computed only when the corresponding decryption key should be revealed, i.e., in the corresponding $\mathcal{O}^{\text{Reveal}}$ call, and the decryption key is chosen so that the decrypted value corresponds to the sampled $R_{1,j}$. For that, we need the encryption to be non-binding. This change is indistinguishable by $\mathcal{A}$, does not introduce additional aborts, and $\mathcal{B}$ succeeds whenever $\mathcal{A}$ succeeds.

Another change that $\mathcal{B}$ makes is setting an output of $\mathcal{O}^{\text{Chal}}$ as X_1 , instead of using the KeyGen algorithm. Because of this change, x_1 is

not known anymore, but we still need to be able to correctly simulate the signing queries, which we do as follows.

Before running $\mathcal{A}$, $\mathcal{B}$ tosses a biased coin coin_j that turns out 1 with probability $1/q_S$ for every signing instance $j \in \{1, \dots, q_S\}$. Whenever $\text{coin}_j = 1$, $\mathcal{B}$ samples $g_j \in \{1, \dots, q_H\}$. The simulation of $\mathcal{O}^{\text{Reveal}}$ and $\mathcal{O}^{\text{Sign}}$ oracles for session j then depends on coin_j .

If $\text{coin}_j = 0$, $\mathcal{B}$ invokes $\mathcal{O}^{\text{Chal}}(h_{k_j})$ to get the element $R_{1,j}$ when revealing the decryption key. To successfully answer a signing query, the simulator queries its $\mathcal{O}^{\text{DLog}}$ oracle with $R_{1,j} + H(R, X, m)X_1$.

If $\text{coin}_j = 1$, $\mathcal{B}$ samples a random $s_{1,j}$ from $\mathbb{Z}_q$ and computes $R_{1,j} = s_{1,j}G - h_{g_j}X$ when revealing the decryption key. The signing query might not always succeed in this case. If $H(R, X, m) \neq h_{g_j}$ for the input provided in the signing query, $\mathcal{B}$ aborts. Otherwise, it outputs $s_{1,j}$ in the corresponding $\mathcal{O}^{\text{Sign}}$ call.

Due to SHINE's design, at most one corresponding $\mathcal{O}^{\text{Sign}}$ query has not been made, nor has the session been invalidated by querying $\mathcal{O}^{\text{Sign}}$ with a higher index. If a session was open when the query for the forgery was made and $\text{coin}_j = 0$, $\mathcal{B}$ aborts.

When $\mathcal{B}$ does not abort, the changes are indistinguishable by $\mathcal{A}$, and $\mathcal{B}$ succeeds whenever $\mathcal{A}$ succeeds. $\mathcal{B}$ does not abort when the session index is guessed correctly, which happens with probability $1/q_S$, and the corresponding hash oracle query is also guessed correctly, which happens with probability $1/q_H$. Furthermore, $\mathcal{B}$ does not abort when no session was open when the fork occurred. From that, we get that the success probability of $\mathcal{B}$ is at least $\epsilon/q_S q_H$.

Now, we apply the Generalized Forking Lemma [126] to $\mathcal{B}$. By the construction of $\mathcal{B}$, no challenge that is still being usable for signing is known at the time of the fork; thus, the application of the forking lemma does not result in multiple $\mathcal{O}^{\text{DLog}}$ queries for the same challenge. If the forking of $\mathcal{B}$ succeeds and it outputs two forgeries s and s' for hashes h and h' , respectively, we use them to solve the OMDL problem in the following way. First, we compute the discrete logarithm of the group key X as $x = (s - s')/(h - h')$. In the KOSK model, we can compute the discrete logarithm of X_1 directly from x , as $\mathcal{A}$ has to output its private keys $(x_2, \dots, x_n)$, and x is a linear combination of $(x_1, \dots, x_n)$. To compute answers to the other challenges, we use x_1 to

solve linear equations in the form $r_{1,j} = s_{1,j} - H(R_j, X, m)x_1$; if some outputs of $\mathcal{O}^{\text{Chal}}$ were not used, we apply $\mathcal{O}^{\text{DLog}}$ on them.

The probability of solving the OMDL is the probability that the forking succeeded and no collisions occurred among any of the q_S values that $\mathcal{B}$ submits to its $\mathcal{O}^{\text{Chal}}$ in any of its runs. Thus, we have

$$\epsilon' \geq \frac{\epsilon}{8q_H q_S} - \frac{q_S^2 k_{\max}^2}{q}.$$

□

We needed the KOSK model only at the end to extract the private key x_1 ; the proof can be extended to MuSig-like and proof of possession group key aggregation by extracting the key correspondingly.

5.5.5 Performance evaluation

We implemented SHINE for the JavaCard platform using the extended version of the JCMATHLib library [15] described in Chapter 4. We tested and evaluated our implementation on six different smartcards:

- | | |
|---------------------------|-----------------------|
| (1) G&D SmartCafe 6.0, | (4) NXP JCOP2 J2E145, |
| (2) G&D SmartCafe 7.0, | (5) NXP JCOP3 J3H145, |
| (3) Infineon Secora ID S, | (6) NXP JCOP4 J3R180. |

We measured the time required to compute each phase of the protocol and their counterparts in a variant without nonce caching to be able to assess its impact. We repeated each measurement 100 times and averaged the results. Table 5.3 presents a summary of the results.

The implementation achieves an average signing speed of around 700 ms, independent of the number of involved parties. The signing slowdown caused by key derivation for piggybacking did not exceed 45 ms for any of the cards and thus increased signing latency by 6% at most. Encryption operation added to the nonce caching resulted in less than 38 ms slowdown, and at most 23% of the time required to perform the nonce computation.

The differences in time required to compute the nonce are caused by different native algorithm support. Smartcards (3), (5), and (6) supported ALG_EC_SVDP_DH_PLAIN_XY algorithm, which made the nonce

Table 5.3: Time required to compute parts of SHINE on different smartcards (in milliseconds).

Smartcard	(1)	(2)	(3)	(4)	(5)	(6)
compute	2764	1962	98	2826	194	60
compute + cache	2801	1984	109	2854	218	74
difference	38	23	11	28	24	14
percentage	1	1	11	1	12	23
sign	768	637	688	802	737	458
sign + reveal	813	660	707	842	756	472
difference	45	23	19	40	19	15
percentage	6	4	3	5	3	3
without caching	3532	2599	786	3627	931	519
with caching	813	660	707	842	756	472
difference	2719	1938	80	2785	175	46
percentage	77	75	10	77	19	9

computation close to the native performance of the hardware, while on other cards, additional transformations had to be made. This computation is the precomputed part; thus, it influences the speedup over signing without caching the most.

We estimate that implementing the signing part using native low-level operations instead of slower computation via JCMathLib would achieve performance comparable to ECDSA on a given platform, e.g., around 200 ms for common smartcards [127]. Nonce caching with encryption would provide an overall speedup of around 50% in such implementations.

Our implementation of SHINE uses a central party, which mediates communication among different devices and serves as storage for the cached nonces. It contains NE, NC, and ND schemes, which can be used by themselves or jointly with the SHINE applet, demonstrating its interoperability. Both implementations are released under a permissive open-source license⁶.

6. <https://github.com/crocs-muni/SHINE/>,
<https://github.com/crocs-muni/SHINE-mediator/>.

5.6 Conclusion

In this chapter, we have shown that it is possible to efficiently execute a state-of-the-art threshold Schnorr signature protocol FROST [16] on the latest smartcards, even using only the restrictive interface that JavaCard API provides. We achieved this by repurposing algorithms exposed by the API that are accelerated on the hardware level to compute operations required by the protocol. Our implementation is the first open-source threshold protocol implementation for JavaCard smartcards, which supports threshold values lower than the number of issued shares and complies with the protocol standard [17].

While the performance results on the latest smartcards were practical, measurements on models not supporting as efficient elliptic curve operations were notably worse. Because of this, we studied the possibility of designing a specialized threshold Schnorr protocol for the smartcard environment that would maintain interoperability with state-of-the-art designs. We classified the existing protocols based on their approach to the nonce agreement, a key component in signature computation. We identified four classes of nonce agreement approaches: nonce exchange (NE), nonce commitment (NC), nonce delinearization (ND), and deterministic nonce derivation (DN).

NE-based protocols are secure only with limited concurrency but are the most efficient and interoperable. For these reasons, we suggest that protocols of this class should be considered for the use cases where computation is costly and sequential execution is acceptable.

Protocols based on DN are the most rigid, the least interoperable, and the most computationally demanding. However, in exchange for these downsides, they gain the benefits of stateless execution and no need for additional randomness. Therefore, these protocols should be considered for scenarios where computation is not a limiting factor, the sources of randomness are limited, and a setup of homogeneous implementations can be guaranteed.

ND protocols lie in the middle between NE and DN regarding their computational complexity and interoperability. In the half-ND variant, these protocols are interoperable with NE-based protocols without compromising security. We thus propose that the half-ND protocols are considered in the common scenario featuring fast devices like

smartphones, which could benefit from interoperability with more constrained devices while being secure under concurrent execution.

The class of NC-based schemes is the only one that currently does not display any apparent additional utility. It provides concurrent security that is achievable even for computationally restricted devices. However, such devices could rely on the more efficient NE, allowing them to be interoperable with half-ND schemes.

Based on these findings, we designed the scheme SHINE to benefit from interoperability while being efficiently executable on smartcards. The scheme is based on NE and is complemented by a novel approach to nonce caching, avoiding the previous attack by Drijvers et al. [117].

We implemented SHINE as an applet for the JavaCard platform and evaluated its performance on six smartcard models. The experiments empirically confirmed the performance improvement of nonce caching on computationally restricted devices over a variant without caching. Furthermore, we provided an implementation of the central party that practically demonstrates the interoperability of SHINE with NE, NC, and half-ND schemes.

6 Two-party ECDSA computation approaches

This chapter is based on our conference paper, which has been accepted for publication at the ACNS conference [A5].

- [A5] A. Dufka, P. Laud, and P. Švenda, “Two-party ECDSA with JavaCard-based smartcards,” in *Applied Cryptography and Network Security: 23rd International Conference on Applied Cryptography and Network Security*, to appear, 2025, pp. 1–18.
-

6.1 Introduction

ECDSA [75] is the most widely used type of elliptic-curve-based signature at present. It is adopted by major internet protocols like TLS [128], SSH [129], GPG [130], and Bitcoin [105]; and its computation is frequently supported by current secure hardware elements [A1, A2].

Despite its ubiquity and hardware support, enabling threshold ECDSA computation on resource-constrained devices like smartcards remains a significant challenge. While recent studies have demonstrated practical performance of other types of threshold signatures on smartcards [A4, A7, 81], no open-source implementation has yet succeeded in achieving similar results for threshold ECDSA.

In this chapter, we explore the approaches for efficient computation of two-party ECDSA on JavaCard-based smartcards. Two-party protocols, as a special instance of threshold protocols, enable faster computation while still remaining suitable for many practical use cases, typically involving authorization by two independent components.

The main reason why computing ECDSA [75] in a distributed way is much more complex, compared to, e.g., Schnorr signatures [74], is that its computation involves the multiplication of two secretly shared values. Performing this computation securely involves protocols for computing oblivious linear evaluation (OLE) [131], commonly constructed using oblivious transfer or partially homomorphic encryption¹, requiring at least tens of kilobytes in communication or computation of several public-key operations.

1. Using class groups or accompanied with costly zero-knowledge range proofs.

In the literature, we have identified a protocol specific to the two-party setting by Lindell [20] that does not require as costly computation and may be suitable for implementation on smartcards. The protocol relies on partially homomorphic encryption of inputs using Paillier’s cryptosystem and does not require costly zero-knowledge range proofs, unlike other (but also more flexible) protocols based on the same construction. The most costly component of the protocol, i.e., decryption of a Paillier’s ciphertext, can be accelerated using the modular exponentiation co-processor available on smartcards.

Even though Lindell’s protocol is likely the most favorable two-party protocol for smartcard implementation, its implementation via public API requires several seconds of computation, making it impractical for some use cases. Therefore, we also investigated approaches utilizing the preprocessing model with an optional trusted preprocessing party. We considered two levels of preprocessing: 1) computing multiplication triples and 2) computing presignatures.

With the first approach, the preprocessing party only assists with the most computationally intensive part of ECDSA protocols – multiplying secretly shared values. This is achieved by constructing multiplication triples, which can be used to compute the multiplication efficiently, even though they are independent of the multiplied values, i.e., do not require knowledge of any secret in the preprocessing.

The other approach is more intrusive as it requires the secrets during preprocessing. The key share is preprocessed by a threshold ECDSA protocol that results in presignatures that can be adjusted by a message and used to construct a signature share non-interactively.

We proposed techniques for efficient computation of all three approaches (Lindell’s two-party scheme, a protocol with preprocessed multiplication triples, and a protocol with preprocessed presignatures) on smartcards by adjusting their communication patterns to the two-party setting, utilizing resources of the more powerful host, and avoiding certain costly operations. We implemented all three protocols for the JavaCard platform using the extended JCMATHLib library [15] described in Chapter 4 and evaluated their performance and trade-offs. The source code is released under an open-source license².

2. <https://github.com/crocs-muni/JC2pECDSA/>,
<https://github.com/crocs-muni/JCPreECDSA/>.

6.2 Background

This section describes the building blocks used in the presented protocols and establishes the notation used in this chapter.

Notation Throughout this chapter, we use G to refer to the group of points of an elliptic curve, over which the ECDSA protocol is defined, and denote its elements in upper-case. Conversely, lower-case letters are used to denote elements of its group of scalars $\mathbb{Z}_r$. The x-coordinate of a point $P \in G$ is denoted by subscript as P_x . Uniform sampling of an element e from a non-empty set S is denoted as $e \leftarrow S$.

6.2.1 Additive and multiplicative secret sharing

Secret sharing is a central concept in secure multi-party computation. It allows for splitting a secret value into multiple shares in a way that less than a certain number of shares yield no information about the secret. However, knowledge of more shares enables its reconstruction.

In this chapter, we will primarily rely on additive secret sharing. An additive sharing of a value $x \in \mathbb{Z}_r$ is a set of values $\{x_1, \dots, x_n\}$ such that $\sum_{i=1}^n x_i = x$, denoted as $[x]$. Analogously, an additive sharing of an elliptic curve point $X \in G$, i.e., $\sum_{i=1}^n X_i = X$, is denoted as $[X]$.

Protocols based on secret sharing need to perform computations with secret-shared inputs and obtain secret-shared results. With additive secret sharing, it is possible to compute addition, negation, and multiplication by a constant non-interactively: $[a + b] = [a] + [b]$, $[-a] = -[a]$, $[k + a] = k + [a]$, $[k \cdot a] = k \cdot [a]$, where $[a]$ and $[b]$ are secret-shared values and k is a constant. However, multiplying secret-shared values requires interaction.

Apart from additive sharing, one of the protocols in this work uses multiplicative secret sharing. It is defined analogously to the additive sharing; the only difference is that instead of adding the shares to reconstruct a secret, they are multiplied, i.e., $x = \prod_{i=1}^n x_i$.

6.2.2 Preprocessing model

The preprocessing model is an established paradigm in the field of secure multi-party computation, which allows for splitting a func-

tion evaluation into two separate phases called *offline* and *online*. The former is computationally demanding but can be computed in advance before inputs to the function evaluation are known, and the latter is a lightweight computation that depends on the inputs and preprocessing outputs, and evaluates the function.

The multi-party evaluation of arithmetic functions is bottlenecked by the computation of multiplication of secret-shared values, which can be efficiently realized in the preprocessing model using Beaver's trick [132]. Beaver's trick assumes that the involved parties hold additive secret shares of values a, b, c , such that $[c] = [a \cdot b]$. Given such a triple, two secret-shared values $[x]$ and $[y]$ can be multiplied as follows. First, $[x - a]$ and $[y - b]$ is computed locally and opened as d and e . Then, a secret-sharing of a product of x and y can be computed with local operations as $[x \cdot y] = [c] + d[b] + e[a] + ed$. Such triples can be generated by a trusted dealer or in a distributed way using a framework like SPDZ [133, 134].

To guard against malicious parties, the multiplication triples need to be complemented with authentication values that prove the validity of the opened values. A common approach of achieving this is through the use of information-theoretic MACs, e.g., the SPDZ-style MAC [133]. With SPDZ-style MAC, all involved parties hold additive shares of a static MAC key Δ , and each secret-shared value $[x]$ authenticated by Δ is complemented by a secret-sharing of its authentication value $[\hat{x}] = [x \cdot \Delta]$. When opening an authenticated secret-share $[x]$, it is first opened as usual, then $[\bar{x}] = [\hat{x}] - x \cdot [\Delta]$ is computed and committed to by each party, subsequently the value is revealed and checked whether $\sum_{i=1}^n \bar{x}_i = 0$. To preserve correctness of MACs throughout a computation, all operations with secret shares need to be mimicked with their MACs.

In this text, we use the term *authenticated triple* to describe a tuple of six secret-shared elements $[\mathbf{t}] = ([a], [b], [c], [x], [y], [z])$ such that $([a], [b], [c])$ is a multiplication triple, i.e., $[c] = [a \cdot b]$, and $[x], [y], [z]$ are authentication values corresponding to $[a], [b], [c]$, respectively. We use the superscript notation to refer to particular elements of the tuple, e.g., $[\mathbf{t}^a] = [a]$. Similarly, we use $\mathbf{t}_i^a$ when referring to the i -th share of the tuple.

6.2.3 Paillier's cryptosystem

Paillier's cryptosystem [135] is a partially homomorphic public-key encryption scheme based on the decisional composite residuosity assumption. The homomorphic properties allow for the addition of two ciphertexts and for the multiplication of a ciphertext by a constant. These capabilities are useful in applications both from generic secure multi-party computation and threshold cryptography.

A key pair in the Paillier's cryptosystem consists of a private key (λ, μ) and the corresponding public key (n, g) . The number n is a product of two large primes, p and q , which define the group $\mathbb{Z}_{n^2}^*$ and g is its selected generator. The private key components are derived from p and q as $\lambda = (p-1)(q-1)$ and $\mu = \left(\frac{(g^\lambda \bmod n^2) - 1}{n} \right)^{-1} \bmod n$.

To encrypt a message $m \in \mathbb{Z}_n$, the encryption algorithm E samples a random value $r \in \mathbb{Z}_n$ coprime to n and computes the ciphertext $c = g^m \cdot r^n \bmod n^2$. The decryption algorithm D recovers the message m' from ciphertext c by computing $m' = \frac{(c^\lambda \bmod n^2) - 1}{n} \cdot \mu \bmod n$.

The homomorphic addition of two ciphertexts, c_1 and c_2 , is performed by multiplication modulo n^2 . Similarly, the multiplication of a ciphertext by a constant is achieved by exponentiation modulo n^2 .

6.2.4 Proof of discrete logarithm knowledge

The proof of discrete logarithm knowledge due to Schnorr [74] is a protocol using which a prover can prove to a verifier in zero-knowledge that it knows a discrete logarithm of a given group element. The protocol is typically used in a non-interactive variant constructed via the Fiat-Shamir transformation [113]. We denote the computation of the non-interactive variant as $\text{DLPoK}(x, P)$, where x is the discrete logarithm and P is the base point.

To prove the knowledge of a discrete logarithm $x \in \mathbb{Z}_r$ of an element $X = xG$. The prover samples a random nonce $k \in \mathbb{Z}_r$, computes the corresponding element $R = kG$, and commits to it using a hash function $e = H(R)$. This value is used to compute the proof $s = k - ex \bmod q$, and the proof is $\pi = (e, s)$. To verify such proof, the verifier computes $R' = sG + eX$ and checks whether $H(R') = e$.

6.3 Three approaches for two-party ECDSA on smartcards

Based on our literature survey and knowledge of the capabilities of JavaCard-based smartcards, we presume that Lindell's two-party protocol is the most suitable for entire computation on smartcards thanks to its optimized design and an option to use hardware-accelerated operations supported by the JavaCard platform. Still, the cost of protocol execution will likely be too prohibitive for most practical applications.

For these reasons, we also investigated two approaches suitable for the preprocessing model, where a trusted party can be utilized during a smartcard's setup. The first of these approaches relies on multiplication triples, secret sharings of correlated random values that can be used to multiply two secretly shared values efficiently while being independent of the multiplied values. However, preprocessing on this level still involves a significant amount of on-card computation, limiting its practicality.

In comparison, the second approach reduces the on-card computation to the bare minimum by utilizing presignatures. Presignatures are specific additive secret sharings of ECDSA signatures, allowing for very efficient construction of signatures but requiring the knowledge of the key share during preprocessing.

In the following subsections, we describe how we applied all three approaches for two-party ECDSA computation to a setup consisting of a smartcard connected to a more computationally-powerful host.

6.3.1 Lindell's two-party protocol

Lindell's two-party ECDSA protocol [20] seems to be the most suitable among the protocols we have surveyed for full implementation on smartcards since it needs only a few kilobytes in transmission, and many of its costly computations can be mapped to accelerated operations provided by the JavaCard API.

The protocol is based on Paillier's homomorphic encryption [135], which can be accelerated using smartcard's modular exponentiation coprocessor; therefore, its most costly component can be computed on the platform much more efficiently than if it would be computed purely in software, giving it an unusual advantage over other approaches.

The reason why Lindell's protocol is more suitable for resource-constrained devices compared to other protocols based on Paillier's encryption is that the protocol does not require the computation of zero-knowledge range proofs over Paillier's ciphertexts during signing. Instead, in the two-party setting, the party that would otherwise need to verify the range proofs can already try to compose and verify the signature. This verification can also be computed efficiently using a function call provided by the JavaCard API.

As the protocol's computation is asymmetric, i.e., the computations the parties need to do differ, it can be utilized to minimize the overall computation time by appropriately assigning the roles in the protocol. Naïvely, based on the communication pattern, the fact that the communication needs to be initiated by the host and that the host needs to receive the signature, it would seem reasonable to initiate the protocol from the host's side. However, that is inefficient since the second party needs to perform several operations with Paillier's ciphertexts, whereas the initiating party only does one decryption, which can be additionally accelerated using the Chinese remainder theorem [69, 135].

Therefore, we shifted the communication in the protocol, resulting in two additional messages being sent but significantly improving the overall performance due to the computation-communication trade-off on smartcards. The protocol is depicted in Figure 6.1.

In the protocol, both parties are initialized with multiplicative shares of private key $x = x_1 \cdot x_2$, their joint public key X , Paillier's modulus n and generator g . Additionally, the host knows a Paillier's encryption of the smartcard's private key share $E(x_2)$ and the message m ; whereas, the smartcard knows the Paillier's decryption key (λ, μ) .

The communication is initiated by the host, who sends the message m to the smartcard. Upon receiving the message, the smartcard initiates the protocol by sampling its multiplicative share of a nonce k_2 , computes the corresponding point $R_2 = k_2G$ and a proof of its discrete logarithm knowledge π_2 , and commits to both the point and the proof using a hash function. The commitment is sent to the host.

In response to the commitment, the host samples its own multiplicative share of the nonce k_1 , computes the corresponding point $R_1 = k_1G$ and a proof of its discrete logarithm knowledge π_1 . The

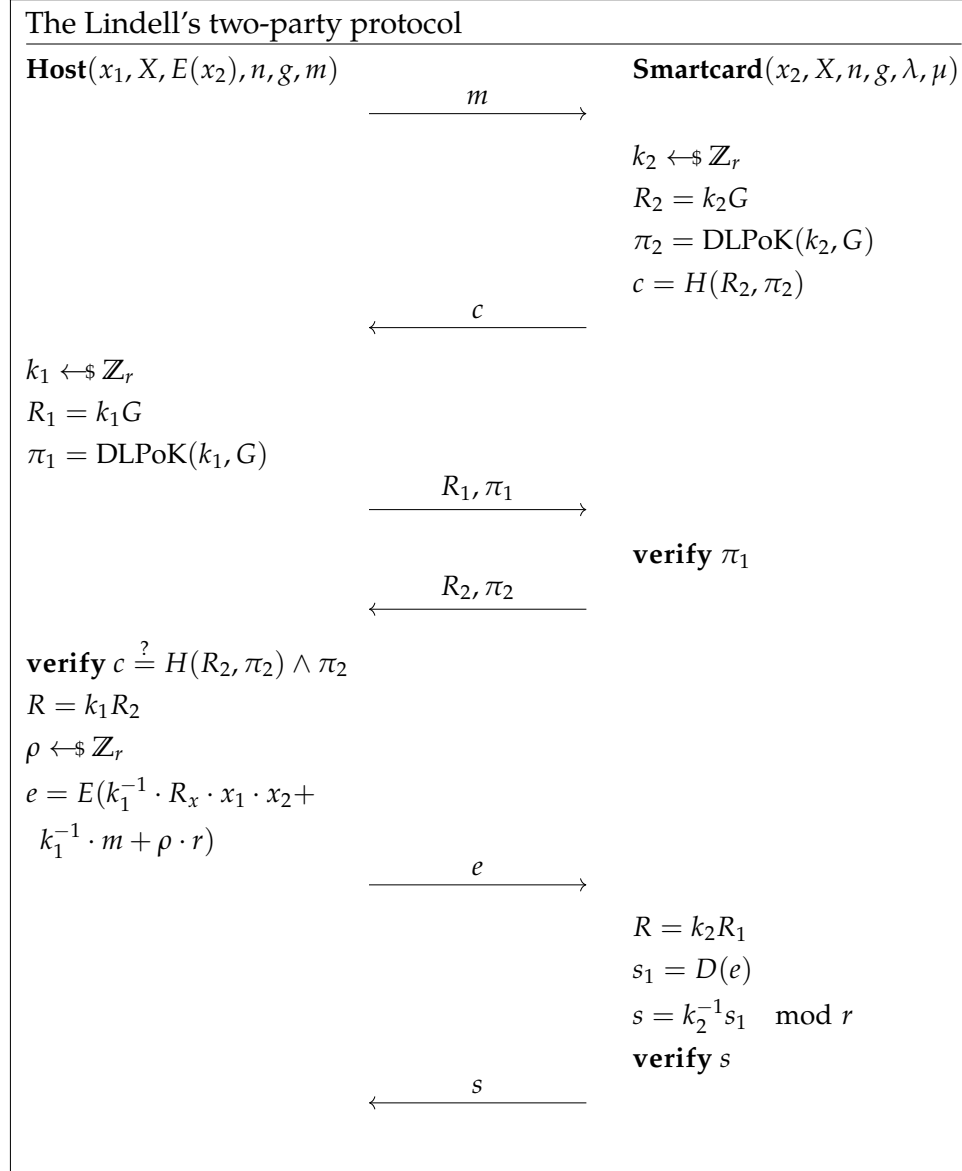


Figure 6.1: The Lindell's two-party protocol [20] adapted to a setup with a smartcard and a more computationally powerful host.

point and the proof are sent to the smartcard in plain. The smartcard verifies the proof and opens its commitment.

The host then verifies that the commitment was correct, that the proof is correct, and proceeds with computing its encrypted signature share in the homomorphic context using the smartcard's encrypted key share $E(x_2)$. The resulting encrypted share is sent to the smartcard. The smartcard decrypts the signature share, multiplies it with an inverse of its nonce share, and verifies that the result is a valid signature of the message m under the public key X . If it is, it releases the signature.

If any of the verifications performed during protocol fails, the protocol is aborted. We instantiated the zero-knowledge proof of discrete logarithm knowledge with the standard Schnorr's proof [74] (as described in Section 6.2.4). For the commitment, we use SHA256, the same hash function as is used in ECDSA.

6.3.2 Two-party protocol with multiplication triples

The approach based on multiplication triples aims to accelerate the most costly operation within threshold ECDSA protocols – multiplication of secret-shared values – but keep the rest of the protocol computation on-card. Since the multiplication triples are independent of the values they aid in multiplying, their preprocessing can be done without any knowledge of the smartcard's secrets.

These triples can be constructed by a single centralized party to whom all involved parties trust or in a distributed manner using a generic multi-party computation framework, e.g., the SPDZ [133, 134]. The triples also need to carry their corresponding authentication values to achieve security in the malicious setting (see Section 6.2.2).

We applied the multiplication triples to a generic threshold ECDSA protocol by Dalskov et al. [82], adjusted its message exchange to minimize the number of messages for the two-party setting, and interleaved value openings with the protocol. The result is a 6-message protocol depicted in Figure 6.2.

The protocol uses the triples to compute secret sharings of $[k^{-1}]$, $[k^{-1}x]$, and $[k]G = [R]$, which can be then non-interactively composed into partial signatures $[s] = H(m)[k^{-1}] + R_x[k^{-1}x]$ that can be opened and verified using the standard ECDSA verification algorithm.

6. TWO-PARTY ECDSA COMPUTATION APPROACHES

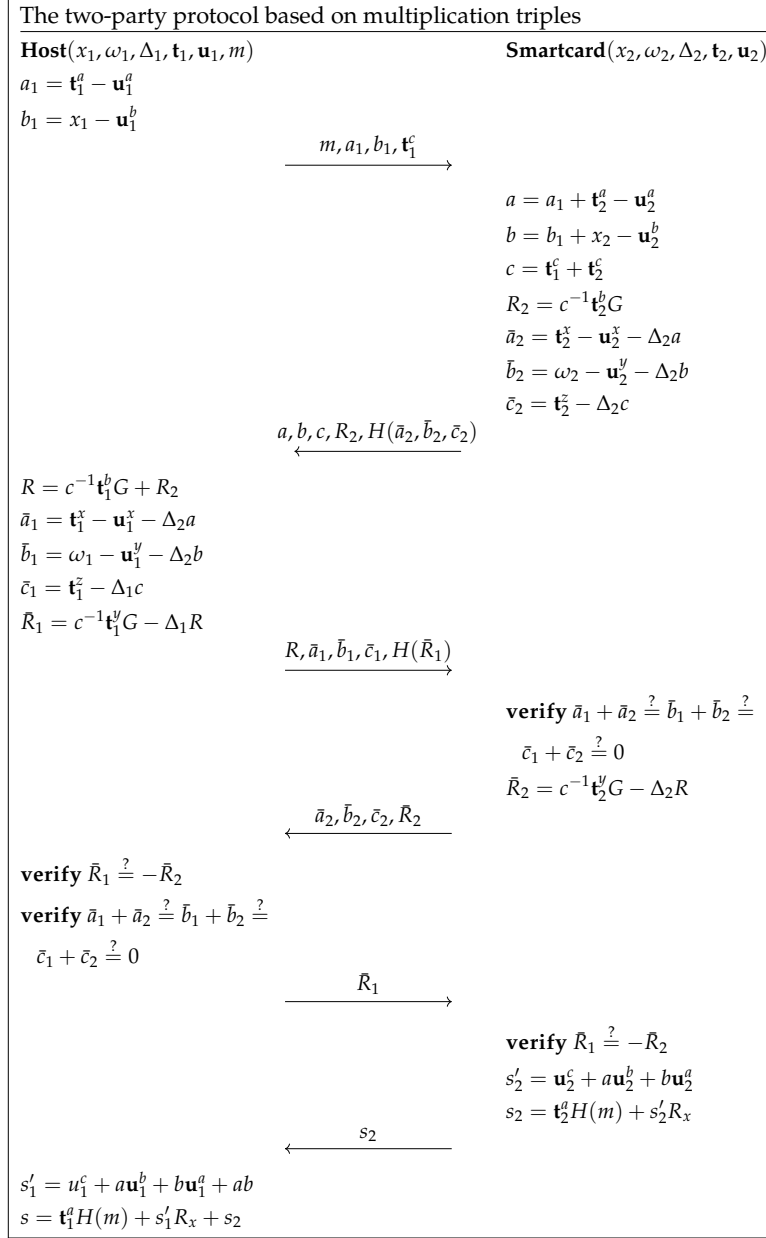


Figure 6.2: The Dalskov et al. [82] threshold ECDSA protocol adjusted to the two-party computation with SPDZ triples. All operations are performed modulo r .

In the protocol, both parties are initialized with shares of private key $[x]$, private key's authentication value $[\omega]$, static MAC key $[\Delta]$, and two authenticated multiplication triples $[\mathbf{t}]$ and $[\mathbf{u}]$. Additionally, the host knows the message m .

First, the host sends the message m and initiates the opening of $[c] = [\mathbf{t}^c]$, $[a] = [\mathbf{t}^a - \mathbf{u}^a]$, $[b] = [x - \mathbf{u}^b]$. The smartcard continues with the opening and computes the corresponding authentication shares whose commitment is sent back. Since the smartcard already knows the opened c , it is able to compute its share of the nonce point $[R] = c^{-1}[\mathbf{t}^b]G$ and initiate its opening. Note that $c^{-1}[\mathbf{t}^b]$ is a secret sharing of the nonce $[k]$.

When the host receives the response, it continues with the computation of its share of $[R]$ and the authentication values for $[a]$, $[b]$, $[c]$, $[R]$. It sends back the share of $[R]$, authentication values for $[a]$, $[b]$, $[c]$ directly, and a commitment to the authentication value of $[R]$. We used a minor optimization by skipping the commitment to the authentication values of $[a]$, $[b]$, $[c]$ at this point. This optimization has no security impact, since it corresponds to the rushing behavior in the protocol, as commitments of other parties were already received by the sender.

With this information, the smartcard is already able to check the correctness of authentication values of the opened values a, b, c and computes its authentication value of $[R]$, which may be directly revealed by utilizing the same optimization. The previously committed authentication values of $[a]$, $[b]$, $[c]$ are also revealed to the host.

Now, the host checks the correctness of all opened values and reveals its authentication value of $[R]$, which is checked by the smartcard. The smartcard then proceeds to compute its partial signature s_2 and sends it directly to the host, who completes and verifies the signature.

6.3.3 Presignatures

The most efficient way of constructing threshold ECDSA signatures with a smartcard is to rely on presignatures. Presignatures are additive secret sharings of $[k^{-1}]$ and $[z] = [k^{-1} \cdot x]$ together with an x-coordinate of the point $R = [k]G$. Given such values, a sharing of a signature s of message m can be computed locally as $[s] = [k^{-1}]H(m) + [z]$, opened, and verified using the ECDSA verification procedure.

Beyond the optimal efficiency, presignatures also provide the benefit of being able to support a signing set of an arbitrary size with the same computation in the online phase, i.e., they can be used with all possible t -out-of- n threshold setups. However, the t involved parties need to be set during preprocessing.

There are some security considerations with using presignatures, as an attacker could condition the signed message on R . However, Groth and Shoup [83] have shown that it can be secure with appropriate countermeasures in the generic group model [136] under a new assumption on the used hash function.

Presignatures can be constructed very efficiently by a centralized party given access to all parties' key shares or in a distributed way by running a threshold ECDSA protocol that supports presignatures, e.g., the protocol by Doerner et al. [87]. The smartcard can either participate fully in such precomputation, which would, however, require a very long computation, or offload its key to a trusted preprocessing party that acts on its behalf in the protocol.

Alternatively, it is possible to combine the two discussed preprocessing approaches to achieve practical performance with a reasonable security trade-off. The preprocessed multiplication triples, which are independent of the used secrets, can be used to compute a threshold ECDSA protocol, resulting in presignatures on a smartcard. The presignatures can then be stored for later use with minimal latency, and their construction does not require revealing any secrets.

6.4 Implementations on smartcards

We utilize the extended JCMATHLib library [81], described in Chapter 4, to implement all the protocols discussed in the previous section.

6.4.1 Paillier's decryption

Decryption of a Paillier's ciphertext (as described in Section 6.2.3) involves several computational steps: modular exponentiation with modulus n^2 , subtraction of one from an n^2 -bit integer, division of an n^2 -bit integer by n , and finally multiplication of two n -bit integers

modulo n . The security of the scheme depends on the bit-size of n ; at present, this value should be of at least 4096 bits.

Modular exponentiation accelerators in current smartcards support at most 4096-bit moduli [A2], which may seem insufficient to implement the protocol on current security level, due to the need of exponentiation mod n^2 . However, with the knowledge of factors of $n = p \cdot q$, this limitation can be bypassed using CRT [69, 135].

With CRT-based decryption [135], the plaintext is first computed separately modulo p and q . Each of these computations involves exponentiation only modulo p^2 or q^2 , which have at most the same bit size as n . Typically, these two results would then need to be combined. However, when used in the context of Lindell's protocol with a 256-bit curve and 4096-bit n , valid plaintexts are always lower than the factors (p and q). Therefore, the combination is unnecessary. Consequently, the computation needs to be performed only modulo one of the factors, decreasing the computation cost by more than half.

However, before the computations mod p^2 can be performed, the ciphertext e needs to be reduced. This can be done by splitting the ciphertext as $e = e_h \cdot 2^{4096} + e_l$ using operations on e 's bit representation. Then, the reduced ciphertext can be computed as $e \bmod p^2 = e_h \cdot t + e_l \bmod p^2$, where $t = 2^{4096} \bmod p^2$ can be precomputed.

The remaining operation, integer division, can be accelerated with the existing techniques by modular multiplication with an inverse of the divisor modulo a larger prime. For this computation, we used the largest 4096-bit prime.

6.4.2 Preprocessing with constrained memory

Apart from low performance, smartcards also have a very limited persistent storage, ranging from tens of kilobytes to a few hundreds on currently available models [A2]. This limited capacity poses a challenge for the practicality of preprocessing approaches, which require a large number of presignatures or multiplication triples to be stored on the smartcard during the initial setup, while the devices can store only around a thousand of preprocessed outputs.

To alleviate this issue, we leverage the host device that always interacts with the smartcard and has access to significantly more memory. The host device can store the preprocessed values on behalf of the

smartcard and provide them to the smartcard whenever a protocol is initiated. However, for this approach to remain secure, it is crucial that the preprocessed values are kept secret from the host and that their origin (the preprocessing party) can be verified.

This is typically achieved using authenticated encryption. However, to this day, the support of authenticated encryption algorithms on JavaCard-based smartcards is rare [A2], therefore we rely on the standard encrypt-then-MAC [137] approach with AES [68] in the CBC mode [138] for confidentiality and CBC-MAC [139] for authenticity.

With the proposed approach, the smartcard is preloaded with a symmetric encryption and MAC keys generated by the preprocessing party during the setup phase. The party generates virtually unlimited number of preprocessed outputs and wraps them by encrypting and MACing them using the keys. The wrapped values can be provided directly to the host device or even stored on a cloud storage. Later, when provided with a wrapped value, the smartcard will be able to verify its authenticity and decrypt it, both using the preloaded keys.

Another important security constraint that needs to be enforced is the freshness of the used values. If a smartcard were to reuse a presignature or a multiplication triple, it would result in breaking its security assumptions and may reveal its private key share. In order to limit the memory requirements for keeping information about the used values, we sequentially number the precomputed values and maintain a monotonic counter in the smartcard's memory. If a precomputed value with a number lower than the counter's value is received, the smartcard rejects it. Otherwise, it sets its counter to the received number. This way, only a single integer is sufficient for keeping the track of the used outputs.

To further optimize memory use, we eliminate the need for storing an additional counter in the wrapped data by exploiting the initialization vector. The IV values used with AES-CBC do not need to be random, they only need to be unique for a given encryption key. This can be exploited to carry the index of preprocessed values without requiring any further data to be stored.

6.5 Performance evaluation

We measured the performance of all three approaches on the NXP JCOP4 J3R180 smartcard³. Each measurement was repeated 100 times, and the results were averaged. The performance of the signing phase of all the protocols is summarized in Table 6.1. Note that the implementations rely heavily on the JCMATHLib library, resulting in significantly lower performance compared to what could be achievable with low-level proprietary access to the platform. Nevertheless, this implementation still provides a reasonable upper bound on the computation time, and the resulting implementation can be made public, facilitating the reproducibility of our research results.

The implementation of Lindell’s [20] protocol with 4096-bit modulus required in total 5932 ms of on-card computation. The generation of the nonce, proving its discrete logarithm knowledge, and committing to the values, all performed during the first round took on average 627 ms. The second round required 186 ms to verify the host’s proof. The most expensive was the last round, which involves Paillier’s ciphertext decryption, composition of the resulting signature, and its verification, resulting in the remaining 5119 ms.

In comparison, the approaches that incorporate some form of pre-computation achieved significantly better results. The adjusted variant of two-party protocol by Dalskov et al. [82], which utilizes precomputed multiplication triples, was able to compute a two-party ECDSA signature on the smartcard within 1800 ms.

As expected, the best performance was observed in the approach using precomputed presignatures. This method required only 203 ms to produce a partial signature, as it involves only a hash function invocation, a modular multiplication, and an addition. Beyond its performance, this approach offers great flexibility since the same partial-signature computation can be used for any signing group (of arbitrary size and threshold). The smartcard only needs an unused presignature corresponding to a given subset of signing parties.

3. The best-performing smartcard with 4096-bit modular exponentiation support from the survey in Chapter 3.

Table 6.1: Comparison of the implemented protocols' properties. The performance column represents the time in milliseconds needed to run the computation on NXP JCOP4 J3R180 smartcard. The experiments were repeated 100 times with their results averaged.

Approach	Setups	Trust required	Performance
Lindell's protocol [20]	2-out-of-2	low	5932 ms
Multiplication triples [82]	2-out-of-2	medium	1800 ms
Presignatures [83]	t -out-of- n	high ⁴	203 ms

6.6 Conclusion

The currently available smartcards and their public interfaces are not well-suited for full two-party ECDSA computation since it involves a multiplication of secret-shared values, usually constructed using partially homomorphic encryption with zero-knowledge range proofs or using oblivious transfers. The former approach requires too many computations of unsupported⁵ operations on smartcards, while the latter imposes prohibitively large communication overhead.

To estimate the cost of such computation, we implemented what is likely the most efficient protocol for constructing two-party ECDSA signatures, as proposed by Lindell [20]. This protocol utilizes Paillier's cryptosystem but avoids the need for costly zero-knowledge proofs during the signing process, distinguishing it from other Paillier-based protocols. Additionally, the decryption step in Paillier's cryptosystem can be partially accelerated using the co-processors available on smartcards. The results show that even though the computation can be accelerated, the overall performance is still relatively low, as its execution takes approximately 6 seconds on the smartcard we used. The primary bottleneck of the computation remains to be the modular multiplication (of 4096-bit numbers), which is in the JCMathLib library accelerated by two invocations of the modular exponentiation co-processor, causing significant overhead.

4. If fully precomputed by a trusted third party.

5. Operations that are only partially acceleratable with current interfaces.

Although our results demonstrate that the practicality of full two-party ECDSA computation (without preprocessing) on current smartcards is limited, it becomes feasible in the preprocessing model. In this model, protocols can be split into a costly offline phase and a lightweight online phase. The offline phase can be computed during the setup of a smartcard by a trusted preprocessing party, decreasing the computation demands on the smartcard.

One level of preprocessing involves the use of multiplication triples, which enables the execution of our two-party instantiation of the selected generic protocol [82] within two seconds. Since multiplication triples are completely independent of the multiplied values, their precomputation can be offloaded to a trusted party without exporting any secrets from the smartcard.

Another level of preprocessing involves presignatures – special correlations adjustable by the signed message to create partial signatures. Partial signatures can be combined non-interactively, resulting in a signature creation in around 200 ms. However, unlike the previous approach, this method requires the private key during preprocessing, thus the key would need to be shared with the preprocessing party.

A balanced solution with a practical trade-off between security and efficiency might involve combining both preprocessing approaches. In this hybrid approach, the smartcard would use multiplication triples precomputed by a trusted third party in a protocol computing presignatures, which could then be stored for later use. This way, the private key does not need to be exported, yet a presignature can be constructed in less than two seconds.

While our work highlights the current limitations of two-party ECDSA computation on smartcards, these challenges could be resolved with enhanced algorithmic support. Manufacturers could expose the interface for accelerated modular multiplication and integrate dedicated circuits for Paillier’s cryptosystem, enabling trustless and practical two-party ECDSA computation on smartcards.

7 Distributing computation of an e-cash system

This chapter is based on our publication presented at TRUSTBUS [A6].

- [A6] A. Dufka, J. Janků, and P. Švenda, “Trust-minimizing BDHKE-based e-cash mint using secure hardware and distributed computation,” in *21st International Workshop on Trust, Privacy and Security in the Digital Society, Proceedings of the 19th International Conference on Availability, Reliability and Security*, Munich, Germany, 2024, pp. 1–10.
-

7.1 Introduction

Decades-old designs of untraceable electronic cash [140] are resurfacing as a second-layer solution to the anonymity and scalability issues in modern, blockchain-based cryptocurrencies, allowing for fully untraceable and cheap transactions with instant confirmation.

Instead of relying on the integrity of a public ledger to solve the double-spending problem, an e-cash system relies on a centralized server, called *mint*, that controls the issuance and redemption of its e-cash tokens. The ownership of a token corresponds to the knowledge of a mint’s signature on a unique number that has not yet been redeemed. Once the signature is presented to the mint, it adds the signed number to its database of already used tokens, and the corresponding token may no longer be redeemed. Until a token is redeemed, it can be transacted among users in the system. To achieve untraceability of transactions, issuance and redemption of a token should be unlinkable, which is realized using blind signatures [140].

A major disadvantage of e-cash systems is their complete reliance on a trusted mint which has full control over the supply of e-cash tokens. If the mint misbehaves, it may issue an arbitrary number of new tokens. The misbehavior can be detected by using various auditing techniques, but they are only reactive measures, and in the meantime, the underlying resources could be drained.

In contrast, a more preventative measure is to minimize the risk of token forgery by reducing the trust in the mint. We consider two

approaches to this trust minimization: 1) utilizing secure hardware elements for performing security-critical operations and 2) distributing the mint capabilities among multiple independent devices.

Secure hardware elements are designed to be tamper-proof; thus, they should be hard to compromise, even in the hands of untrustworthy parties. Furthermore, as the devices are typically programmed to perform just the security-critical operations and no other, they feature only a minimal attack surface, making it harder for an attacker to find and exploit its vulnerabilities even if present.

A common issue of secure hardware elements, especially smartcards, is that their computational resources and interfaces are quite limited. The devices are typically equipped with specialized cryptographic co-processors to accelerate a limited set of predefined operations, which can be performed efficiently, but other computations cannot benefit from them. For this reason, we surveyed building blocks of e-cash systems, focusing on the suitability of their implementation in resource-constrained devices.

The survey revealed that two building blocks are particularly suitable for the implementation on smartcards – pairing-free blind signatures, e.g., [141, 142, 143, 144], and a non-signature blind protocol called BDHKE [145]. The former includes a whole class of blind signature protocols that do not rely on bilinear pairings, and the latter is a blind protocol that does not support public verifiability yet provides the functionality needed in an e-cash system. Since BDHKE is the most efficient construction from the mint’s side and is also compatible with the blind BLS protocol [146], a popular building block of e-cash systems, we focused on it for further analysis.

The second approach – distributed computation – allows for minimizing the trust put in the mint by splitting its capabilities among multiple independent devices in a way that all of them need to agree on an operation before it can be completed. We propose a new protocol for securely realizing the BDHKE functionality distributed among n parties, aiming at the highest security configuration that remains secure as long as there is a single non-corrupted party. We analyzed the protocol’s security by reducing the problem of a token forgery to the one-more gap Diffie-Hellman problem [147] in the random oracle model [125].

Since both approaches are orthogonal, combining them yields complementary benefits and makes a mint's misbehavior even harder. Additionally, the combination provides an option to use hardware produced by different manufacturers to address the potential threat of supply chain attacks in the form of hardware backdoors [81].

We implemented the BDHKE protocol on JavaCard-based smartcards both in the single-party and distributed variants and proposed an optimization allowing a further increase in performance by offloading certain computations to the host device. We evaluated the performance of the implementation on a physical smartcard, demonstrating the practicality of the solution. The source code is publicly available and released under a permissive license¹.

Related Work

The outcome of this work is related to an earlier work by Mavroudis et al. [81], who used a set of smartcards to perform joint signing, asymmetric decryption, and randomness generation. Their goal was to achieve secure computation even in the presence of backdoored components by relying on hardware produced by different manufacturers. We extend this idea to e-cash mints.

The distributed computation of blind protocols has been widely analyzed, with several designs for many constructions like blind Schnorr-based signatures [144], blind BLS [148], or blind RSA [149, 150]. Nevertheless, to the best of our knowledge, no prior work has focused on the distributed computation of BDHKE. Compared to the other distributed blind protocols, our protocol relies on the one-more gap Diffie-Hellman hardness assumption, does not require a pairing-friendly curve, and features short outputs consisting of a single group element.

Tomescu et al. [151] proposed a complex system involving a distributed e-cash mint based on Pointcheval-Sanders signatures [152] that can withstand corruption up to a third of servers by utilizing threshold cryptography and Byzantine fault tolerance protocol. Baudet et al. [153] proposed a performant and scalable Byzantine fault tolerant protocol for electronic payments that involves partially-trusted sharded authorities. Rial and Piotrowska [154] designed a distributed

1. <https://github.com/crocs-muni/JCMint>.

e-cash system that enables offline transactions and features threshold issuance of the tokens. Apart from academic publications, Fedimint² is a recently evolving project that utilizes blind BLS signatures [146] to distribute computation and decrease trust in the mint. Compared to these solutions, we aim to achieve compatibility with resource-constrained devices.

7.2 Background

In this section, we describe the notation used in this chapter and define digital signatures and proofs of discrete logarithm equality.

Notation A group description is a triplet $(\mathbb{G}, p, G)$, where $\mathbb{G}$ is a group of prime order p , and G is its selected generator. A group description can be generated using a probabilistic polynomial-time (PPT) algorithm GrGen that on input 1^λ outputs a group description with λ -bit prime p . We use the additive notation for the group operation and denote group elements in upper-case. Conversely, we use lower-case to denote $\mathbb{Z}_p$ elements. Uniform sampling of an element e from a non-empty set S is denoted as $e \leftarrow_{\$} S$. By $\mathcal{A}(x)$, we denote the set of outputs of probabilistic algorithm $\mathcal{A}$ given input x . We reserve n for the number of protocol participants.

7.2.1 Digital signature

Digital signature DS is a tuple of probabilistic polynomial-time algorithms $(\text{DS.Setup}, \text{DS.KeyGen}, \text{DS.Sign}, \text{DS.Verify})$ such that:

- $\text{DS.Setup}(1^\lambda) \rightarrow \text{par}$: On input a security parameter λ outputs a set of public parameters par which are implicitly provided to all other algorithms.
- $\text{DS.KeyGen}() \rightarrow (\text{sk}, \text{pk})$: Generates and outputs a key pair (sk, pk) , where sk is the private key and pk is the public key.
- $\text{DS.Sign}(\text{sk}, m) \rightarrow \sigma$: On input a private key sk and a message m , signs the message with the private key and outputs signature σ .

2. <https://fedimint.org/>.

- $\text{DS.Verify}(\text{pk}, m, \sigma) \rightarrow b$: On input a public key pk , a message m , and a signature σ , output a bit b indicating whether the signature is valid with respect to the message and the public key.

A digital signature should satisfy *correctness* and existential *unforgeability* against the chosen message attack [155].

7.2.2 Proof of discrete logarithm equality

Non-interactive zero-knowledge proof of discrete logarithm equality, denoted DLEQ, is a pair of probabilistic polynomial-time algorithms (DLEQ.Prove , DLEQ.Verify) such that:

- $\text{DLEQ.Prove}(d, X, P) \rightarrow \pi$: On input scalar d and elements X, P , compute elements $Y = dX, Q = dP$ and output a zero-knowledge proof π of the relationship $\log_X Y = \log_P Q$.
- $\text{DLEQ.Verify}(\pi, X, Y, P, Q) \rightarrow b$: On input proof π and elements X, Y, P, Q output a bit b indicating whether the proof is valid with respect to the elements.

A non-interactive zero-knowledge proof of discrete logarithm equality should satisfy *completeness*, *soundness*, and *zero-knowledge* [155].

We will instantiate DLEQ with Chaum-Pedersen proof [156], which we briefly recall. The proof system works with a group description $(\mathbb{G}, p, G)$ and uses a hash function $H_\pi : \mathbb{G}^6 \rightarrow \mathbb{Z}_p$. The $\text{DLEQ.Prove}(d, X, P)$ first samples $r \leftarrow \$ \mathbb{Z}_p$ and computes $Y = dX, Q = dP, A = rX, B = rP$, queries $c = H_\pi(X, Y, P, Q, A, B)$, computes $s = r - cd \pmod{p}$, and sets $\pi = (c, s)$. The $\text{DLEQ.Verify}(\pi, X, Y, P, Q)$ decomposes $(c, s) = \pi$, computes $A = sX + cY, B = sP + cQ$, and outputs 1 if $c = H_\pi(X, Y, P, Q, A, B)$, otherwise 0.

Such a DLEQ instantiation allows, in the random oracle model [125], for proof forgery in simulations, which will be useful in reduction proof in Section 7.4.4. To forge a proof for inputs X, Y, P, Q , the simulator samples $s, c \leftarrow \$ \mathbb{Z}_p^2$, computes $A = sX + cY$ and $B = sP + cQ$ and programs the oracle to output c for $H_\pi(X, Y, P, Q, A, B)$.

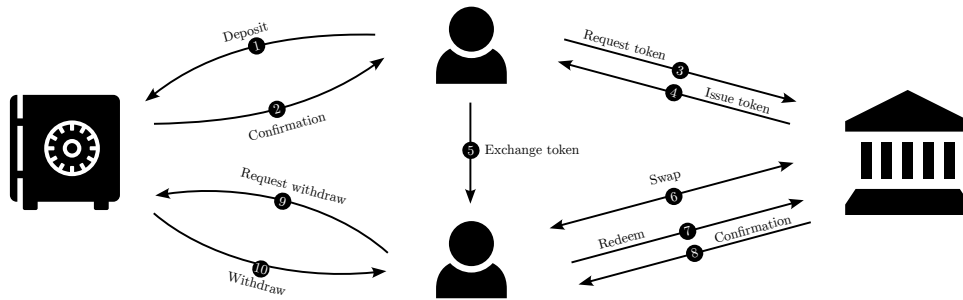


Figure 7.1: An example of a mint backed by an external custody service. A user wanting to enter the system first deposits money **1** to the custody service. In response, the custody service signs **2** the user's request for issuing an appropriate number of e-cash tokens. The user then sends **3** this signature along with the request to the mint, which will verify it and issue the e-cash tokens **4**. Subsequently, the tokens can be transferred among users of the system **5**, who validate the tokens with the mint and swap them **6** for fresh ones. Once a user wishes to withdraw the tokens, she requests a redemption by the mint **7**. The mint verifies the tokens and confirms the success by issuing a signature **8**. The user then provides **9** this signature to the custody service, which in response releases **10** an appropriate amount of the backing asset to the user.

7.3 Electronic cash concepts

The concept of untraceable electronic cash was originally introduced by Chaum, providing a practical application of blind signatures [140, 157]. The system involves a trusted party issuing and redeeming tokens against its own local ledger, containing a set of redeemed tokens to prevent double-spending.

Mint implementations typically provide an interface for three basic operations: 1) issuing a new token, 2) redeeming a token, and 3) swapping an old token for a fresh one. The first two operations are self-explanatory. The third operation is needed to commit transactions between users. During a transaction, the payee should immediately swap the received token for a fresh one; otherwise, the payer, knowing the same information, could still redeem the token before the payee. For this reason, transactions require online interaction with the mint.

Beyond these operations, other application-specific operations can be provided, but they can be constructed from these basic ones, so we do not discuss them further.

Token issuing and redeeming is typically performed in connection to a certain action performed by an external component, e.g., depositing or withdrawing the underlying collateral. To enable integration with such components without binding to a particular one, we consider a signature-based interface communicating these actions securely.

The interface assumes that the public keys of the mint and the external component are exchanged upon their initialization. With the interface, the external party must authorize each token issuance by creating a signature for a token request. The mint later verifies the signature, and if it is valid, a new token is created. Similarly, after the mint completes a redemption, it creates a signature confirming the successful redemption. This signature can be verified by the external party to ensure the validity of the redemption. For an illustration of such an interaction on an example usage with a custodian holding the collateral, see Figure 7.1.

7.3.1 Blind protocol

E-cash systems have been instantiated with various blind protocols, some of which rely only on private verifiability [145]. Therefore, we include this notion and define a blind protocol as follows.

A blind protocol BP is a tuple of probabilistic polynomial-time algorithms $(\text{BP.Setup}, \text{BP.KeyGen}, \{\text{BP.Sign}_j\}_{j=1}^r, \text{BP.Verify})$, parameterized by the number of exchanged messages r , such that:

- $\text{BP.Setup}(1^\lambda) \rightarrow \text{par}$: On input a security parameter λ outputs a set of public parameters par which are implicitly provided to all other algorithms.
- $\text{BP.KeyGen}() \rightarrow (\text{sk}, \text{pk})$: Generates and outputs a key pair (sk, pk) , where sk is the private key and pk is the public key.
- $\text{BP.Sign}_j(\text{st}, \text{in}) \rightarrow (\text{st}', \text{out})$: On input a state st and input in perform a single step of the blind signing protocol and output new state st' and output out . The state st' is always stored by the party computing the BP.Sign_j procedure and input to its next

invocation BP.Sign_{j+2} , while the output out is provided to the other party which uses it as its input in computing BP.Sign_{j+1} . The initial state of the user/issuer contains pk/sk , respectively. The user always computes the last round BP.Sign_r , whose output $\text{out} = (m, T)$ contains the message m and the corresponding token T .

- $\text{BP.Verify}(\text{sk}, m, T) \rightarrow b$: On input a *private* key sk , a message m , and a token T , output a bit b indicating whether the token is valid with respect to the message and the key.

A blind protocol should satisfy *one-more unforgeability*, just like a blind signature [114, 140], a weakened variant of *blindness* assuming an honest-but-curious signer due to lacking public verifiability, and a modified correctness allowing for verification with the private key: for every $m \in \{0, 1\}^*$ and $(\text{pk}, \text{sk}) \leftarrow \$ \text{BP.KeyGen}()$, the BP.Sign_j message exchange results in T such that $\Pr[\text{BP.Verify}(\text{sk}, m, T) = 1] = 1$.

We have surveyed blind protocols commonly used in e-cash construction with the aim of identifying the ones suitable for computation on resource-constrained devices. The original blind signature used in the e-cash proposal was based on the RSA algorithm [149, 150]. However, its outputs are inferior in size and efficiency to comparable elliptic-curve-based schemes, which can have up to 8x smaller outputs at the same 128-bit security level.

Currently, the most compact and feature-rich blind protocols are constructed using pairing-friendly elliptic curves [146, 152, 158, 159], allowing for very efficient signing. However, their verification is significantly more costly, as it involves pairing function computation, which is not feasible for resource-constrained devices.

A middle ground between these categories is offered by pairing-free elliptic-curve-based blind signatures [141, 142, 143, 144], which produce outputs with around double the size of pairing-based schemes, but with verification time of the same order of magnitude as signing. Unfortunately, due to the threat of the ROS attack [141], the schemes involve a more complex design and rely on a proof in the algebraic group model [160]. Still, these schemes are likely a viable option for resource-constrained devices.

Another alternative suitable for resource-constrained devices is blind protocols that lack public verifiability. These protocols can some-

times be constructed by transferring a pairing-based blind signature into the pairing-free setting and verifying the output by its reconstruction. An example of such a scheme is Blind Diffie-Hellman Key Exchange (BDHKE) [145], although it was not originally constructed this way. The verification of this scheme is as efficient as signing, which is even more efficient than that of pairing-free blind signatures. For these reasons, we focus on BDHKE in the rest of this work and expound on it in the following subsection.

Blind Diffie-Hellman Key Exchange

The BDHKE protocol was originally mentioned on a cypherpunk mailing list by David Wagner [145]. It was proposed as an alternative for the blind signing protocol based on RSA used in the original e-cash proposal, and it has been recently utilized in a project called Cashu³, developing an e-cash system backed by Bitcoin [105].

The protocol is defined in Figure 7.2 as an instance of a blind protocol. To get a token using BDHKE, a user generates a random message m of λ bits, which is then hashed to a group element using $H_G : \{0, 1\}^* \rightarrow \mathbb{G}$ and blinded by adding a blinding factor rG , where r is a random scalar from $\mathbb{Z}_p$. The blinded point C is then sent to the signer, who multiplies it by its private key k and returns the resulting blinded token T' . Once the user receives the response, she can unblind it and get the token as $T = T' - rK$, where K is the public key of the signer. To verify the unblinded token later, the signer needs to be provided (m, T) and then checks whether $kH_G(m) = T$ holds.

The protocol is reminiscent of blind BLS [146], and it is virtually the same, only lacking public verifiability, as it does not utilize a group with computable bilinear pairings [76]. The only party that can verify the output is the party that controls the private key.

The non-verifiability can be resolved by extending the protocol, as Jarecki et al. [147] have shown. They have proposed a construction similar to BDHKE to which they added public verifiability using a non-interactive zero-knowledge proof of discrete logarithm equality [156]. This solves the problem only for the token recipient, as the verification

3. <https://cashu.space/>.

Setup(1^λ)	KeyGen()	Sign ₁ (st, in)
$(\mathbb{G}, p, G) \leftarrow \text{GrGen}(1^\lambda)$	$k \leftarrow \$ \mathbb{Z}_p$	$m \leftarrow \$ \{0, 1\}^\lambda$
Select $H_G : \{0, 1\}^* \rightarrow \mathbb{G}$	$K \leftarrow kG$	$r \leftarrow \$ \mathbb{Z}_p$
$\text{par} \leftarrow (\mathbb{G}, p, G, H_G, \lambda)$	return (k, K)	$C \leftarrow H_G(m) + rG$
return par		return $((\text{st}, m, r), C)$
Sign ₂ (st, in)	Sign ₃ (st, in)	Verify(k, m, T)
$k \leftarrow \text{st}$	$(K, m, r) \leftarrow \text{st}$	$V \leftarrow kH_G(m)$
$C \leftarrow \text{in}$	$T' \leftarrow \text{in}$	return $V = T$
$T' \leftarrow kC$	$T \leftarrow T' - rK$	
return $(\perp, T')$	return $(\perp, (m, T))$	

Figure 7.2: Definition of BDHKE blind protocol.

requires the blinding factor, but allows to keep the protocol blind even in the malicious setting.

7.3.2 Mint model

We model mint as a stateful system $\mathcal{M}$ parameterized by a digital signature scheme DS and a blind protocol BP. The system is initialized with its private key k , redeem confirmation private key k' , external party's public key E , and an empty ledger L . The system provides three operations:

- $\text{Issue}(C, \sigma) \rightarrow T$: On input a blinded challenge C and its signature σ , $\mathcal{M}$ checks if $\text{DS.Verify}(E, C, \sigma) = 1$, and proceeds with the BP.Sign_j message exchange to produce a token T .
- $\text{Redeem}(m, T) \rightarrow \sigma$: On input a message m and a signed token T , $\mathcal{M}$ checks that $m \notin L$, updates $L \leftarrow L \cup \{m\}$, and if $\text{BP.Verify}(k, m, T) = 1$, outputs $\text{DS.Sign}(k', m)$.
- $\text{Swap}(m, T, C) \rightarrow T'$: On input a message m , a signed token T , and a challenge C , checks that $m \notin L$, updates $L \leftarrow L \cup \{m\}$,

and if $\text{BP.Verify}(k, m, T) = 1$, produces a new token T' using the BP.Sign_j message exchange.

The operations can consist of multiple sub-operations that are invoked in a sequence before obtaining the output. In case a verification within the operations fails, or the sub-operations are invoked in an incorrect order, no output may be given. DS, BP, and the public keys of k and k' are publicly available.

7.4 Distributing mint functionality

The risk of compromise of an e-cash system can be decreased by distributing its mint functionality among multiple independent partial mints in a way that unless all of them agree with an operation, the operation cannot be successfully completed. Such a setting requires the private key to be split among the partial mints using a secure secret sharing and the produced shares to be used by the parties in a secure multi-party computation protocol without reconstructing the complete private key in a single place.

We consider a setup of n partial mints $\mathcal{M}_i$, where each is initialized with its private key k_i , public keys of all partial mints $K_1, \dots, K_n$ such that the mint key is defined as $K = \sum_{i=1}^n K_i$, analogously for the redeeming key k' and the corresponding public keys, and a public key of an external component E . Additionally, the partial mints can communicate with each other via a mediator that can arbitrarily alter or drop messages (and thus cause a denial of service). Beyond that, the partial mints behave exactly as regular mints; only DS and BP are instantiated with an n -party protocol.

7.4.1 Distributing BDHKE

We propose a new protocol for secure distributed computation of BDHKE, called dBDHKE, in a way suitable for resource-constrained devices. Interestingly, the proposed solution can also be used to distribute the blind BLS protocol [146] to devices that hold key shares but do not support pairing function computation.

The distributed variant of Sign operation is from the mint's side (Sign_2) same as the single-party one, since its outputs can be aggre-

gated non-interactively, just like in the BLS scheme [148]. The aggregation is an additional task that needs to be performed by the party performing the protocol in Sign_3 , who receives partial tokens from the partial mints $T'_1, \dots, T'_n$ and aggregates them as $T' = \sum_{i=1}^n T'_i$ before their unblinding.

Distributing the Verify computation is more complex, as produced tokens cannot be directly verified without having access to the complete private key $k = \sum_{i=1}^n k_i$. Therefore, we propose a two-round protocol during which the token is recreated and complemented by non-interactive zero-knowledge proofs of its correct construction. Using these proofs, partial mints can verify that the provided token was valid. However, since the token has been recreated during this process, it may no longer be accepted in the future, *independently* of the verification result. The application utilizing the protocol needs to enforce this behavior, which is achieved in the mint case by adding the token to the ledger.

The verification protocol splits the Verify operation into two procedures ($\text{Verify}_1, \text{Verify}_2$), defined in Figure 7.3. In the first procedure, verifiers (partial mints) are initialized with their state containing their private key k_i and public keys K_j of all parties, and input consisting of a message m and a token T to verify. Using its key share k_i , each verifier computes a verifying token V_i and a DLEQ proof π_i of its correct construction. The procedure outputs a state for the next round, consisting of the verified token and public keys of other parties, and a message to transmit, consisting of the verifying token V_i and proof π_i . The messages are then transmitted to other protocol participants.

The second procedure of each party is invoked with its state from the previous round and messages of all parties as input. The procedure verifies all received proofs and checks that the reconstructed point corresponds to the token that is being verified. The result of the verification is output.

Correctness

The token issued with the distributed protocol is a valid token verifiable with the private key $k = \sum_{i=1}^n k_i$ as shown in the following

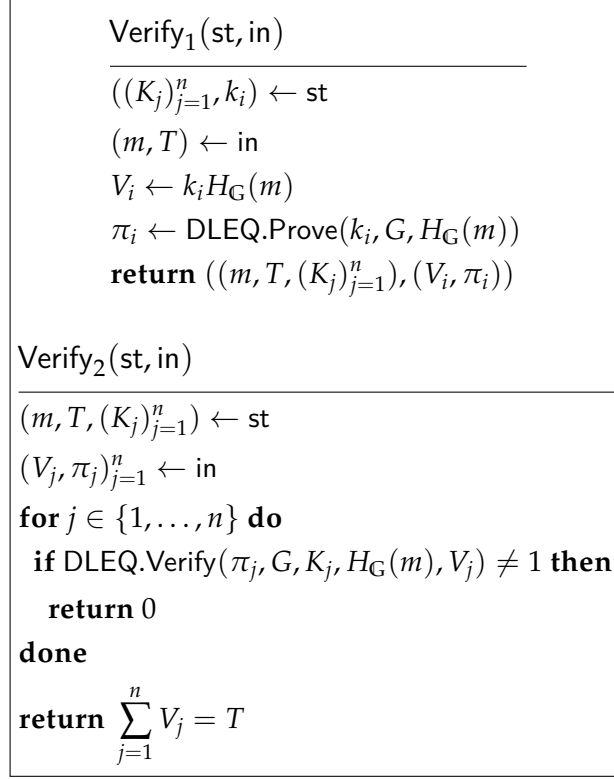


Figure 7.3: Definition of dBDHKE verification procedures.

equation:

$$\begin{aligned}
 T &= T' - rK = \sum_{i=1}^n T'_i - rK = \sum_{i=1}^n k_i(H_G(m) + rG) - rK \\
 &= k(H_G(m) + rG) - rK = kH_G(m) + rK - rK = kH_G(m).
 \end{aligned}$$

The distributed verification is correct since for a valid token T of message m , $T = \sum_{i=1}^n V_i = \sum_{i=1}^n k_i H_G(m) = kH_G(m)$, which satisfies the verification equation.

7.4.2 Distributing DS

To fully distribute the trust of all mint operations, even the DS.Sign during Redeem operation needs to be computed using a n -party signature

protocol. Many such protocols exist, e.g., [109, 146]. Focusing on the compatibility with resource-constrained devices, we recommend the Schnorr n -signature scheme by Nick et al. [109], as it can be efficiently constructed and verified using smartcards. The protocol consists of two communication rounds, which can be interleaved with the verification rounds to eliminate the need for extra communication. In case the verification on smartcards is not needed, BLS signatures [146] may be a better solution, as they are more efficient to compute and can be aggregated non-interactively.

7.4.3 DOS mitigation

Since dBDHKE requires a token to be added to the ledger during verification, independently of its validity, an attacker may attempt to spam the system with invalid tokens and fill up the available memory. This is particularly troublesome for devices with limited storage, which may need to rely on various storage offloading techniques, significantly degrading the system's performance.

This problem can be addressed in an account-based system by constraining the user's number of verification attempts within a time window or by sanctioning users repeatedly attempting to verify invalid tokens. Although such systems do not enable direct tracing of users' transactions, their metadata may reveal partial information about users' balances unless additional precautions are taken.

A more private system would utilize a group-based authentication, for example, based on group signatures [161], which only proves that the accessing user is one of the eligible users but does not allow identifying which one. However, this only prevents attacks by outsiders, as eligible users cannot be distinguished from each other.

In an entirely anonymous setting, the problem can be addressed by disincentivizing spamming by increasing the cost of making a request, similarly to the concept of HashCash [162]. For a distributed BDHKE-based mint, the technique can be applied by requiring the submitted token (output of H_G) to have a defined prefix. Given such a requirement, users would need to search for an input that hashes to a value with the prefix before submitting a request. A mint then could efficiently verify whether a token satisfies the requirement by recomputing the hash before proceeding with the protocol.

7.4.4 Security properties

In this section we show that the dBDHKE protocol with DLEQ instantiated using Chaum-Pedersen proof [156] satisfies the security against *one-more forgery*, as defined in Figure 7.5, assuming the hardness of the one-more gap Diffie-Hellman problem [147] in the random oracle model [125]. Privacy-related properties are inherited from the base scheme since users' interactions remain the same.

In the attacker model, we assume that the initial generation and distribution of private keys and corresponding public keys were performed correctly and consistently. Then, the attacker was given private keys of all but one honest party $\mathcal{M}_1$ to simulate the corruption of $n - 1$ parties. The attacker has also an oracle access to $\mathcal{M}_1$'s procedures Sign_2 , Verify_1 , and Verify_2 , and also random oracles H_G and H_π .

Hardness assumption

The one-more gap Diffie-Hellman problem [147] is defined in Figure 7.4. A challenger provides an adversary with a group description $(\mathbb{G}, p, G)$ and a public key $K = kG$, where k is known only to the challenger. Additionally, the adversary has access to three oracles: Chal , which outputs a randomly sampled challenge C from $\mathbb{G}$, sDH , which on input C outputs kC , and sDDH , which on input C, T from $\mathbb{G}$ outputs 1 if and only if $kC = T$, i.e., (C, K, T) is a valid DDH triple. The adversary wins if it outputs more distinct valid solutions kC_i to challenges than is the number of queries made to the sDH oracle.

Reduction

Given a PPT adversary $\mathcal{A}$ that with non-negligible probability wins the OMUF game, we construct a PPT adversary $\mathcal{B}$ who wins the OMGDH game with non-negligible probability.

$\mathcal{B}$ is initialized with a group description $(\mathbb{G}, p, G)$, the challenger's public key K_1 , and Chal , sDH and sDDH oracles. $\mathcal{B}$ initializes $\mathcal{A}$ using the same group description, K_1 as the honest party's public key, and private keys of all other $n - 1$ parties $k_2, \dots, k_n$ which $\mathcal{B}$ generated. $\mathcal{B}$ answers $\mathcal{A}$'s oracle queries as follows:

OMGDH ^A (λ)	Chal()
$(\mathbb{G}, p, G) \leftarrow \text{GrGen}(1^\lambda)$	$c \leftarrow c + 1$
$k \leftarrow_{\$} \mathbb{Z}_p; K \leftarrow kG$	$C_c \leftarrow_{\$} \mathbb{G}$
$c \leftarrow 0; l \leftarrow 0$	return C_c
$(\hat{C}_j, U_j)_{j=1}^c \leftarrow_{\$} \mathcal{A}^{\text{Chal}, \text{sDH}, \text{sDDH}}(\mathbb{G}, p, G, K)$	$\text{sDH}(C)$
for $j \in \{1, \dots, c\}$ do	$l \leftarrow l + 1$
if $\hat{C}_j \neq C_j \vee U_j \neq kC_j$ then	return kC
return 0	$\text{sDDH}(C, T)$
done	return $kC = T$
return $l < c$	

Figure 7.4: The (static) one-more gap Diffie-Hellman game.

- $H_{\mathbb{G}}(m)$: $\mathcal{B}$ checks whether the oracle has been queried with the same input before. If not, it invokes its $\text{Chal}()$ oracle and its output C is programmed into the $H_{\mathbb{G}}$ oracle. Then, the programmed value is output.
- $H_{\pi}(X, Y, P, Q, A, B)$: $\mathcal{B}$ checks whether a query with the same input was submitted to the oracle before. If not, a fresh $c \leftarrow_{\$} \mathbb{Z}_p$ is sampled and programmed into the H_{π} oracle. Then, the programmed value is output.
- $\mathcal{M}_1.\text{Sign}_2(C)$: $\mathcal{B}$ increments its counter l , relays the request to its $\text{sDH}(C)$ oracle, and returns the output k_1C .
- $\mathcal{M}_1.\text{Verify}_1(m, T)$: $\mathcal{B}$ uses its sDDH oracle to check whether $V'_1 = T - \sum_{j=2}^n k_j H_{\mathbb{G}}(m)$ is a valid token with respect to m and the challenger's key K_1 . In case it is, the verifying point V_1 is set to the extracted point V'_1 . Otherwise, $\mathcal{B}$ queries the sDH oracle for it. The DLEQ proof π_1 is forged by programming H_{π} , as described in Section 7.2.2. In case the value for given inputs is already set, $\mathcal{B}$ aborts. Finally, (V_1, π_1) is output.
- $\mathcal{M}_1.\text{Verify}_2(\text{proofs})$: $\mathcal{B}$ checks that a verification is currently in progress and proceeds according to the OMUF game. In case

$\text{OMUF}_{\text{dBDHKE}}^{\mathcal{A}}(\lambda)$	$\mathcal{M}_1.\text{Verify}_1(m, T)$
$(\mathbb{G}, p, G, H_G, \lambda) \leftarrow \text{Setup}(1^\lambda)$	if $m \in L$ then return $\perp$
Select $H_\pi : \mathbb{G}^6 \rightarrow \mathbb{Z}_p$	$L \leftarrow L \cup \{m\}$
$(k_i, K_i)_{i=1}^n \leftarrow \text{KeyGen}^n()$	$\text{st} \leftarrow ((K_j)_{j=1}^n, k_1)$
$L \leftarrow \{\}; l \leftarrow 0$	$\text{in} \leftarrow (m, T)$
$\mathcal{A}^{H_G, H_\pi, \mathcal{M}_1}(K_1, (k_j)_{j=2}^n)$	$\text{st}', \text{out} \leftarrow \text{Verify}_1(\text{st}, \text{in})$
return $l < 0$	return out
$\mathcal{M}_1.\text{Sign}_2(C)$	$\mathcal{M}_1.\text{Verify}_2(\text{proofs})$
$l \leftarrow l + 1$	if $\text{st}' = \perp$ then return $\perp$
$(\text{st}, T') \leftarrow \text{Sign}_2(k_1, C)$	$b \leftarrow \text{Verify}_2(\text{st}', \text{proofs})$
return T'	$\text{st}' \leftarrow \perp$
	if b then $l \leftarrow l - 1$
	return b

Figure 7.5: The dBDHKE one-more unforgeability game.

a verification of the token with sDDH oracle failed, yet Verify_2 output 1, abort.

Throughout $\mathcal{A}$'s execution, l corresponds to the difference between the number of queries made to the sDH oracle and the number of challenge solutions known by $\mathcal{B}$, as will be detailed in the following paragraphs. Consequently, if l turns negative and $\mathcal{B}$ does not abort, $\mathcal{B}$ is able to win the OMGDH game.

For each query to $\mathcal{M}_1.\text{Sign}_2$, the l value is incremented since sDH was queried, and we have to assume that the output is not a solution to a challenge, as $\mathcal{A}$ could have used blinding.

When $\mathcal{A}$ invokes the verification sequence with a valid token, $\mathcal{B}$ is able to extract $k_1 H_G(m)$ without performing an sDH query, as it can rely on the verification of the token using the sDDH oracle. Since the token was valid, l may be eventually decremented, but $\mathcal{B}$ can already answer the corresponding challenge.

When $\mathcal{A}$ invokes the verification sequence with an invalid token, $\mathcal{B}$ has to query the sDH oracle to provide the correct answer, but since

the token is invalid, the verification will fail with overwhelming probability due to the soundness of the DLEQ system. In that case, l will not be decremented, and since m was already included in L , the reconstructed token is no longer acceptable for verification, and $\mathcal{B}$ learned the solution to the corresponding challenge.

If $\mathcal{A}$ attempts to invoke the verification with a message that is already present in L , it will immediately return without making any further queries or changes to l .

$\mathcal{A}$'s view in the simulation is indistinguishable from a real execution. When $\mathcal{A}$ terminates, $\mathcal{B}$ looks up all challenges C_i from the Chal oracle, which may have been either included in L , or unused. For the former, $\mathcal{B}$ looks up the corresponding solutions U_i obtained when simulating the $\mathcal{M}_1.\text{Verify}_1$ procedure. For the latter, $\mathcal{B}$ queries its sDH oracle. Finally, $\mathcal{B}$ outputs all pairs (C_i, U_i) . If the output of $\mathcal{A}$ is a valid solution to the OMUF game, then the output of $\mathcal{B}$ is a valid solution to the OMGDH problem.

Consequently, $\mathcal{B}$ wins the OMGDH whenever $\mathcal{A}$ wins the OMUF game and $\mathcal{B}$ does not abort. $\mathcal{B}$ aborts when it fails to program the H_π oracle with a DLEQ forgery, which occurs with probability at most q_π/p , where q_π is the number of queries made to the H_π oracle. The only other instance when $\mathcal{B}$ aborts is when there is an inconsistency between at least one of the DLEQ proofs and the sDDH oracle, each of which can occur with the probability of the soundness error ϵ_π of the DLEQ system, which is negligible. Therefore, both failure events occur only with negligible probability.

Avoiding trusted setup

In case the trusted setup is undesirable, the protocol can be modified to accommodate distributed key generation at the cost of an additional communication round.

Without knowing the keys $k_2, \dots, k_n$, $\mathcal{B}$ would not be able to simulate answers for valid tokens during $\mathcal{M}_1.\text{Verify}_1$ queries. In order to restore this ability, $\mathcal{B}$ would need to obtain the verification points of all parties before outputting its answer. This can be achieved by extending the protocol by an additional commitment round, during which $\mathcal{B}$ learns the verification points of other parties before having to submit its answer.

7.5 Implementation on smartcards

In order to obtain the maximal benefits of using tamper-proof devices, the devices need to run the entire mint functionality as defined in Section 7.3.2, involving all three operations Issue, Redeem, and Swap, including the state management.

We implemented the BDHKE protocol both in the single-party and the distributed variant for current JavaCard-based smartcards. The implementation uses the extended JCMathLib library described in Chapter 4. Such implementation provides lower performance than could be achievable with low-level access to the platform, but it is portable among JavaCard smartcards from different manufacturers, and its source code can be fully public⁴.

We assume that smartcards are provisioned in a secure environment, i.e., initially loaded with an authentic applet implementation and provided with its key and additional setup information, but afterward can be used even in an adversarial environment.

7.5.1 Single-party case

The single-party BDHKE protocol from the signer's side consists of two operations: scalar multiplication and hashing to a curve. Scalar multiplication with hardware-accelerated implementation is commonly supported by the latest JavaCard smartcards [A2], so this operation can be computed close to the platform's native performance. However, hashing to a curve is more costly, as neither the algorithm nor its underlying operations are available in the JavaCard API.

There are many hash-to-curve algorithms, but a relatively straightforward one (and used by the Cashu project) is the following: compute the SHA256 hash of the input, concatenate a counter value to the hash output, and hash it again. The result is taken as an x-coordinate of a point. Afterward, the corresponding y-coordinate is computed. Since the probability of such a point being valid is only around 50%, it can fail. In that case, the counter is incremented, and the process is repeated until a valid point is found.

4. <https://github.com/crocs-muni/JCMint>.

This whole process can be implemented on a smartcard, but it is rather costly, as computing the y-coordinate requires several modular operations. These operations are relatively slow, especially when using modular arithmetic that is only partially hardware-accelerated. Although we implemented this operation fully on a smartcard, we also propose a computation offloading approach in the following section, which allows for a vastly improved performance.

Offloading hash-to-curve computation

Since smartcards are typically used with a much more computationally powerful host device, they can offload costly public computations to the host device as long as the results can be efficiently verified.

The SHA256 computation can be performed efficiently with the dedicated JavaCard API algorithm [A2] and the costly part is the reconstruction of a point, which may need to be repeated several times due to the trial and error attempts. To avoid this, the host device can compute the output of the hash-to-curve operation and provide it to a smartcard. The smartcard can efficiently verify that the provided output is a valid point and that its x-coordinate is a hash of the input.

However, a problem with this approach is that it only works if a valid point is found in the first iteration. Otherwise, the card would not be able to verify whether it received the first point that matched, which could result in an incorrect point being provided. If that would be the case, an attacker could convince the smartcard that a point has a different pre-image, causing double-spending.

The issue can be avoided by putting a constraint on the message that users use for their tokens. Messages that require more than just a single iteration will be considered invalid. This shifts more computation to users, as they have to search⁵ for such messages, but as they have more powerful devices, the delay should be acceptable.

7.5.2 Extension to multi-party setting

The multi-party setting adds the need for computation and verification of a zero-knowledge proofs of discrete-logarithm equality, and the addition of points to check whether the verifying points match

5. Every attempt has around 50% chance of success.

the received token. The support for point addition is already available in the latest JavaCard-based smartcards [A2], but both the proof computation and verification need to be implemented.

The DLEQ functionality instantiated with the Chaum-Pedersen construction [156] requires scalar multiplications, point additions, a hash function evaluation, and modular arithmetic. All these operations can be computed using the JavaCard API and the extended JCMathLib library [15].

During the first round of the verification protocol, only a single proof needs to be constructed. However, in the second round, all proofs of the other parties need to be verified, incurring an overhead linear in the number of parties. This computation becomes rather restrictive with larger sets of signers, as it involves costly operations.

7.6 Performance evaluation

We measured the performance of our JavaCard implementation on NXP JCOP4 J3R180 smartcard⁶ with a 256-bit curve, both in the single-party and the distributed variant with up to 5 parties, and also with and without hash-to-curve computation offloading. We measured separately the time required to issue and verify a new token using the (d)BDHKE protocol. The sum of those operations corresponds to the mint's Swap operation. All the measurements were repeated one hundred times and averaged. The results from the smartcard are presented in Table 7.1 and visualized in Figure 7.6.

Due to the used measurement approach, the results include communication overhead involving sending an APDU packet to the smartcard and receiving the response, requiring 6-108 ms, depending on the amount of transmitted data. On the contrary, they do not include possible delays caused by the mediator computation nor computation of other parties which can run in parallel.

The implementation turned out to be particularly efficient in the single-party setting with enabled optimizations, allowing for a token swap in just 174 ms. This is close to the hardware's native performance level, as none of the involved operations require the use of reconstructed operations from JCMathLib. In case the constraint on valid

6. The best-performing smartcard from the survey in Chapter 3.

Table 7.1: The average computation time (ms) of signing a token (Sign_2) and verifying it (Verify_1 , Verify_2) in dependence on the number of involved parties with and without hash-to-curve computation offloading (opt). In the single-party setting, verification can be done in a single round; therefore Verify_2 is empty. Measured on NXP JCOP4 J3R180 with a 256-bit elliptic curve.

n	1	2	3	4	5
Sign_2	52	52	52	52	52
Verify_1 (opt)	122	342	342	339	341
Verify_1	784	1002	1026	1028	995
Verify_2	—	287	535	787	1035
Total (opt)	174	681	930	1178	1428
Total	836	1340	1613	1866	2082

points from Section 7.5.1 cannot be applied, the hash-to-curve computation cannot be offloaded and must be computed on a card. This can cause multiple point reconstruction attempts, each requiring around 232 ms, leading to high variation and computation time of 836 ms.

In the distributed setting, swapping a token involves both dBDHKE verification rounds and also issuing a new token after successful verification. This computation requires relying on the JCMATHLib library independently on the hash-to-curve offloading. With the offloading enabled, a 2-party setup can swap a token in 681 ms and each additional party adds around 250 ms, being still under 1.5 s for 5 parties. Without the offloading, the average computation time is increased by around 700 ms independently of the number of parties.

The achieved results are acceptable for mints performing a single-digit number of transactions per minute. Furthermore, the throughput can be increased in solutions using multiple token denominations by adding smartcards dedicated to different denominations.

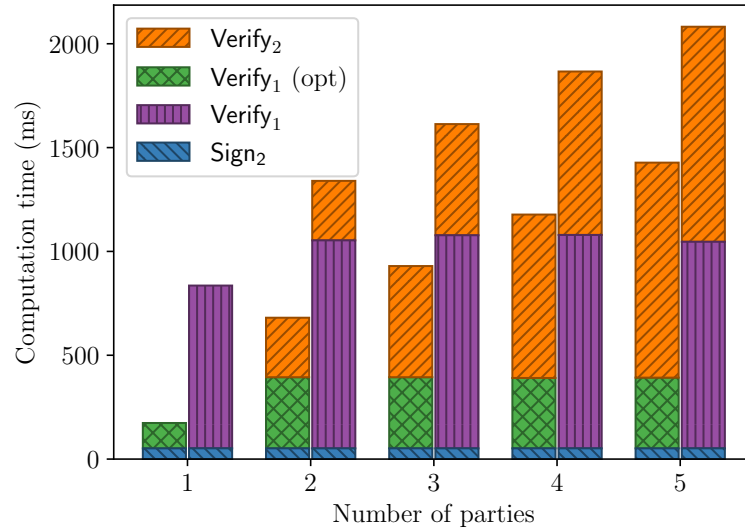


Figure 7.6: The time required to perform a token swap in dependence on the number of parties. Left columns show time with offloaded hash-to-curve computation and right columns without. Measured on NXP JCOP4 J3R180 smartcard with a 256-bit elliptic curve.

Table 7.2: The number of operations needed to perform in different parts of the protocol depending on the number of parties n . The last row shows the number of operations required in the verification of single-party BDHKE.

	$\mathbb{G}$ mult	$\mathbb{G}$ add	$\mathbb{Z}_p$ mult	$\mathbb{Z}_p$ add	H2C
Sign ₂	1	0	0	0	0
Verify ₁	3	0	1	1	1
Verify ₂	$4n - 4$	$3n - 3$	0	0	0
Verify	1	0	0	0	1

Still, the hardware is capable of achieving better results using a low-level access. We counted the number of mathematical operations used in the protocol (presented in Table 7.2) and estimated the potential native performance of the protocol based on the number of scalar multiplications ($\mathbb{G}$ mult), as it is by a large factor the most costly

operation in the protocol. The JavaCard API exposes this operation directly; therefore, we can measure how fast it can perform. By extrapolating from this value, we estimate that in the two-party setting, the protocol should be computable on the platform at around 400 ms, and each additional party should add around 200 ms.

7.7 Conclusion

In this work, we have proposed an approach for trust-minimization of e-cash mints based on BDHKE, a very efficient yet not publicly verifiable blind protocol. The approach involves distributing the mint's operations including the blind protocol computation among independent tamper-proof devices holding secret shares of the mint's key.

Although distributed signing with BDHKE is straightforward, due to the tokens being aggregatable non-interactively, the verification is more complex, as BDHKE token verification relies on the private key. Consequently, parties holding only a share of the key cannot verify it alone. To resolve this issue, we proposed an n -party protocol during which the token is recreated and complemented with a zero-knowledge proof of its validity, facilitating the verification.

The protocol is also of separate interest, as it is compatible with the BLS scheme and thus can enable the verification of n -party BLS signatures on resource-constrained devices. This yields a unique option to use publicly verifiable BLS-based n -party mints while relying on the more efficient dBDHKE-style verification on the mints' side.

We have implemented the protocol for the JavaCard platform and measured its performance on a physical smartcard NXP JCOP4 J3R180. The results show that BDHKE is the most practical in the single-party setting. However, even a two-party variant can swap a token in less than 700 ms, making the results practical for mints performing a single-digit number of token swaps per minute.

8 Conclusion

This dissertation thesis focused on two main research goals centered around the execution of threshold cryptography protocols on resource-constrained security-certified devices. The first goal was to analyze the capabilities, performance, and security of the two largest ecosystems of security-certified devices with a focus on their suitability for threshold cryptography. The second goal was to design new approaches for practical threshold computation of major elliptic-curve-based signature schemes on such devices. Although the results were mostly positive, major limitations of current devices have been uncovered. The following paragraphs discuss the obtained results, consequences of presented limitations, and envision future research directions.

Based on our analyses of ecosystems of JavaCard-based smartcards and Trusted Platform Modules, both types of devices typically have sufficient hardware resources for performing practical computations of operations underlying threshold elliptic-curve-based protocols. However, the restricted public interfaces to these operations impose significant limitations on the performance achievable by protocol implementations not natively supported by the platform. On the one hand, the design decision to limit the public interface is understandable from a security standpoint, as it minimizes the attack surface and decreases the likelihood of creating vulnerable implementations. On the other hand, it limits innovation only to parties with enough resources to influence the development of these platforms.

While given access only to these restricted interfaces, we extended known techniques repurposing the interfaces to enable practical computation of threshold protocol implementations on JavaCard-based smartcards. Although these techniques likely do not generalize beyond computation on such smartcards, they are crucial for achieving practical performance on this dominant platform. The other techniques we designed for the computation in cooperation with a more powerful host device (secure computation offloading, preprocessing, and interoperable protocols) do generalize and should provide performance gain even with full access to the underlying hardware. We provide open-source implementations of threshold protocols for the

8. CONCLUSION

computation of three major elliptic-curve-based signature variants on JavaCard-based smartcards, all achieving practical performance.

This thesis focused primarily on classical cryptosystems, in particular, those based on elliptic curve cryptography, which are currently in wide use. However, recent advancements highlight a growing shift towards post-quantum cryptography, marked by the standardization of the first such primitives by NIST [163, 164, 165]. This transition introduces significant challenges, particularly for threshold cryptography [166] and resource-constrained security-certified hardware. Post-quantum schemes often feature more complex designs and impose significantly higher demands on data processing and transmission, presenting hurdles for practical implementation.

Initial progress has already been made in both directions. Samsung has begun a security certification process for their dedicated co-processor designed for post-quantum algorithms [167]. Several protocols realizing threshold computation of post-quantum signatures have been published [166, 168, 169, 170]. However, the extent to which newly designed hardware will accommodate the computation of threshold post-quantum protocols remains uncertain and represents a worthwhile area for further study.

A parallel can be drawn with the evolution of the JavaCard interface, which became more flexible over time. A similar trajectory may emerge with post-quantum accelerators: initial designs might lack support for operations needed in threshold post-quantum computation, with such capabilities introduced only in later revisions. To preempt these delays, threshold protocols should be part of the hardware design considerations from the outset, enabling their seamless integration alongside the base schemes.

Future research in this direction could investigate the suitability of new post-quantum co-processor designs for accelerating the computation of published threshold post-quantum protocols and propose generic interfaces that could facilitate efficient computation of a broader range of protocols. Such work could provide timely and important findings, influencing the development of the hardware before it commits to too rigid architectures.

Appendix

A Achieved results

This chapter summarizes all my academic publications and teaching-related work undertaken during my doctoral studies. For publications, I express my contribution according to the CRediT taxonomy [171] and approximate the overall contribution as a percentage.

A.1 Academic publications included in the thesis

Publication 1: TPMScan: A wide-scale study of security-relevant properties of TPM 2.0 chips

In this publication, we studied the ecosystem of Trusted Platform Modules. We analyzed trends in their adoption in CC and FIPS 140 certification documents. We developed a tool for collecting data from real TPMs, using which we collected data samples from 78 unique TPM firmware, which resulted in the detection of new security vulnerabilities in ECC algorithm implementations. The most severe discovered vulnerability allows for extracting a private key from just nine signatures created with it. We also used the obtained data to evaluate the suitability of TPMs for the computation of advanced protocols.

- *Published at:* CHES 2024 – Conference on Cryptographic Hardware and Embedded Systems (CORE A).
- *My contribution:* I extended the tool for the collection of TPM data by ECC functionality. I conducted the analysis of the collected ECC data that, among other findings, led to the detection of biases in ECC signatures, resulting in the discovery of the 9-signature vulnerability. I also performed the analysis of device capabilities and supported operations.
- *CRediT:* Methodology, Investigation, Software, Visualization, Writing – draft, review, and editing.
- *Percentage:* 27% (total number of authors: 7).

Bibliographic reference: P. Švenda, A. Dufka, M. Brož, R. Lacko, T. Jaroš, D. Zaťovič, and J. Pospíšil, “TPMScan: A wide-scale study of security-relevant properties of TPM 2.0 chips,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2024, no. 2, pp. 714–734, 2024. doi: 10.46586/tches.v2024.i2.714–734.

Publication 2: JCAIlgTest: Robust identification metadata for certified smartcards

In this publication, we studied the ecosystem of JavaCard-based smartcards, focusing on the analysis of real devices. We developed a tool for the collection of smartcard characteristics, including capabilities, performance, and outputs of cryptographic algorithms. Using the tool, we collected data samples from over a hundred smartcards. Data analysis provided insight into the support and performance of current JavaCard-based smartcards. Moreover, we proposed extended security certification artifacts that would enable smartcard model verification by end-users.

- *Published at:* SECRIPT 2022 – 19th International Conference on Security and Cryptography (CORE B).
- *My contribution:* I helped to extend the data collection tool, prepared visualizations, updated and validated findings, and reviewed and edited parts of the text.
- *CRediT:* Investigation, Software, Validation, Visualization, Writing – review and editing.
- *Percentage:* 20% (total number of authors: 4).

Bibliographic reference: P. Švenda, R. Kvašňovský, I. Nagy, and A. Dufka, “JCAIlgTest: Robust identification metadata for certified smartcards,” in *Proceedings of the 19th International Conference on Security and Cryptography - Volume 1: SECRIPT*, S. D. C. di Vimercati, Ed., INSTICC, SciTePress, 2022, pp. 597–604, ISBN: 978-989-758-590-6. DOI: 10.5220/0000163500003283.

Publication 3: The adoption rate of JavaCard features by certified products and open-source projects

We followed up on the work undertaken in our earlier publication [A2] and continued with the study of the JavaCard ecosystem, this time from the perspective of certification documents and open-source applet implementations. First, we analyzed available certification documents to identify trends in newly certified devices. We complemented the discovered trends with insights from the open-source JavaCard implementations. Finally, we analyzed the practical portability of the applets by attempting their installation on three different smartcards and also studied the resources they require.

- *Published at:* CARDIS 2023 – 22nd Smart Card Research and Advanced Application Conference (CORE B).
- *My contribution:* I conducted the analysis of certification documents, prepared the related section and visualizations, and helped with reviewing and editing the rest of the text.
- *CRediT:* Data curation, Investigation, Visualization, Writing – draft, editing, and review.
- *Percentage:* 30% (total number of authors: 3).

Bibliographic reference: L. Zaoral, A. Dufka, and P. Švenda, “The adoption rate of JavaCard features by certified products and open-source projects,” in *Proceedings of the 22nd Smart Card Research and Advanced Application Conference*, S. Bhasin and T. Roche, Eds., Cham: Springer, 2023, pp. 169–189, ISBN: 978-3-031-54409-5. DOI: 10.1007/978-3-031-54409-5_9.

Publication 4: Enabling efficient threshold signature computation via JavaCard API

In this publication, we mapped the operations required in the implementation of threshold elliptic-curve-based protocols and discussed how they can be efficiently computed on current JavaCard-based smartcards while relying only on public JavaCard API. We proposed new techniques enabling further acceleration and integrated them within an open-source JavaCard library [15]. We used the improved library to implement FROST [16], a state-of-the-art threshold signature scheme, making it the first open-source smartcard implementation of a threshold protocol supporting arbitrary threshold. The implementation is fully compatible with the protocol standard [17], and we also proposed additional smartcard-specific optimizations. We demonstrate the practicality of the approach by performance evaluation on three smartcards.

- *Published at:* ARES 2023 – 18th International Conference on Availability, Reliability and Security (CORE B, nominated for the best paper award).
- *My contribution:* The concept of this publication was my original idea. I executed all major parts of its preparation, including protocol implementation, proposing optimizations, conducting

A. ACHIEVED RESULTS

performance measurements, evaluating results, and writing the original draft.

- *CRediT*: Conceptualization, Methodology, Investigation, Software, Visualization, Writing – draft, editing, and review.
- *Percentage*: 90% (total number of authors: 2).

Bibliographic reference: A. Dufka and P. Švenda, “Enabling efficient threshold signature computation via Java Card API,” in *Proceedings of the 18th International Conference on Availability, Reliability and Security*, ser. ARES’23, Benevento, Italy: Association for Computing Machinery, 2023, pp. 1–10, ISBN: 9798400707728. doi: 10.1145/3600160.3600180.

Publication 5: SHINE: Resilience via practical interoperability of multi-party Schnorr signature schemes

In this publication, we proposed the concept of threshold protocol interoperability – the ability of multiple threshold protocols to execute jointly in a way that results in a valid result on behalf of all involved parties. We surveyed state-of-the-art threshold Schnorr signature schemes and categorized them into four classes. We study the possibilities of interoperability among different classes and propose an option for its realization via an untrusted mediator in certain cases. Based on these findings, we propose a new multi-signature protocol designed for resource-constrained devices that is interoperable with popular protocols. We analyzed the security of the protocol by a reduction to the one-more discrete logarithm problem in the random oracle model and evaluated its performance on five different smartcards.

- *Published at*: SECUREPT 2022 – 19th International Conference on Security and Cryptography (CORE B).
- *My contribution*: The concept of this publication was my original idea, which was further developed in discussions with my co-authors. I proposed the classification of threshold Schnorr signature protocols, analyzed their interoperability, proposed the interoperability mediation, implemented software presented in the work, and conducted the performance measurement. The security analysis of the protocol was conducted together with the second co-author. I prepared the original draft and participated in its further refinement with my co-authors.

- *CRediT*: Conceptualization, Formal analysis, Methodology, Investigation, Software, Writing – draft, editing, and review.
- *Percentage*: 60% (total number of authors: 3).

Bibliographic reference: A. Dufka, V. Sedláček, and P. Švenda, “SHINE: Resilience via practical interoperability of multi-party Schnorr signature schemes,” in *Proceedings of the 19th International Conference on Security and Cryptography - Volume 1: SECRYPT*, S. D. C. di Vimercati, Ed., INSTICC, SciTePress, 2022, pp. 305–316, ISBN: 978-989-758-590-6. DOI: 10.5220/0011145600003283.

Publication 6: Two-party ECDSA with JavaCard-based smartcards

In this publication, we studied approaches to two-party ECDSA computation and proposed three approaches for their computation on current smartcards with different trade-offs. The first approach is based on a two-party protocol by Lindell [20], which is computable on smartcards thanks to their modular exponentiation co-processor but still relatively slow. The remaining two approaches utilize the pre-processing model with an optional trusted preprocessing party. We implemented all three approaches for the JavaCard platform while considering the hardware constraints and evaluated their performance on a physical smartcard to assess their practicality.

- *Published at*: ACNS 2025 – 23rd International Conference on Applied Cryptography and Network Security (CORE B).
- *My contribution*: The concept of this publication originated from discussions with my co-authors. I was responsible for protocol investigation, smartcard implementation, performance evaluation, and draft writing.
- *CRediT*: Conceptualization, Methodology, Investigation, Software, Writing – draft, editing, and review.
- *Percentage*: 60% (total number of authors: 3).

Bibliographic reference: A. Dufka, P. Laud, and P. Švenda, “Two-party ECDSA with JavaCard-based smartcards,” in *Applied Cryptography and Network Security: 23rd International Conference on Applied Cryptography and Network Security*, to appear, 2025, pp. 1–18.

Publication 7: Trust-minimizing BDHKE-based e-cash mint using secure hardware and distributed computation

In this publication, we distributed the functionality of an e-cash mint based on the BDHKE protocol in a way suitable for computation on resource-constrained devices. We analyzed the security of the distributed protocol by a reduction to the one-more gap Diffie-Hellman problem in the random oracle model. We implemented the protocol on JavaCard-based smartcards and measured its performance to assess its practicality. Furthermore, the proposed protocol is compatible with BLS signatures, making their distributed verification feasible even for devices that are not capable of computing the pairing function, provided they hold the corresponding private key shares.

- *Published at:* TRUSTBUS 2024 – 21st International Workshop on Trust, Privacy and Security in the Digital Society (CORE B).
- *My contribution:* This publication was conceptualized during discussions with my co-authors. I proposed the protocol, conducted its initial security analysis, implemented it, and evaluated its performance. The security analysis was refined together with my second co-author. I prepared the original draft, which was refined further by my co-authors.
- *CRedit:* Conceptualization, Formal analysis, Methodology, Investigation, Software, Writing – draft, editing, review.
- *Percentage:* 60% (total number of authors: 3).

Bibliographic reference: A. Dufka, J. Janků, and P. Švenda, “Trust-minimizing BDHKE-based e-cash mint using secure hardware and distributed computation,” in *21st International Workshop on Trust, Privacy and Security in the Digital Society, Proceedings of the 19th International Conference on Availability, Reliability and Security*, Munich, Germany, 2024, pp. 1–10.

A.2 Academic publications not included in the thesis

Publication 8: Skye: An expanding PRF based fast KDF and its applications

In this publication, we proposed a new KDF (Key Derivation Function) based on an expanding PRF (Pseudo-Random Function) designed to increase the performance of symmetric ratcheting within the Signal protocol and similar applications. We implemented the KDF and integrated it within the Signal code base to evaluate its performance benefits.

- *Published at:* AsiaCCS 2024 – 19th ACM Asia Conference on Computer and Communications Security (CORE A).
- *My contribution:* I prepared the prototype implementation of the KDF and used it to evaluate its performance within the Signal code base.
- *CRedit:* Methodology, Investigation, Software, Writing – editing and review.
- *Percentage:* 20% (total number of authors: 5).

Bibliographic reference: A. S. Bhati, A. Dufka, E. Andreeva, A. Roy, and B. Preneel, “Skye: An expanding PRF based fast KDF and its applications,” in *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security*, ser. ASIA CCS ’24, Singapore, Singapore: Association for Computing Machinery, 2024, pp. 1082–1098, ISBN: 9798400704826. DOI: 10.1145/3634737.3637673.

Publication 9: DiSSECT: Distinguisher of standard and simulated elliptic curves via traits

In this publication, we analyzed the properties of standard elliptic curves and statistically compared their properties to an extensive collection of newly generated curves, with the goal of detecting possible differences between the groups.

- *Published at:* AFRICACRYPT 2022 – 13th International Conference on Cryptology (not ranked by CORE).
- *My contribution:* I prepared a database for storing and processing elliptic curve representations and extended the capabilities

A. ACHIEVED RESULTS

of the system for the computation of elliptic curve traits. Based on discussions with co-authors, I proposed and developed the methodology for the automated detection of outliers.

- *CRediT*: Data curation, Methodology, Investigation, Software, Writing – editing and review.
- *Percentage*: 15% (total number of authors: 5).

Bibliographic reference: V. Sedláček, V. Suchánek, A. Dufka, M. Sýs, and V. Matyáš, “DisSECT: Distinguisher of standard and simulated elliptic curves via traits,” in *Progress in Cryptology - AFRICACRYPT 2022*, L. Batina and J. Daemen, Eds., Cham: Springer Nature Switzerland, 2022, pp. 493–517, ISBN: 978-3-031-17433-9.

Publication 10: Machine learning fingerprinting methods in cyber security domain: Which one to use?

In this publication, we compared the processing time, the memory requirements, and the achieved performance of four machine learning methods on the task of passive identification of operating systems from network traffic.

- *Published at*: IWCMC 2018 – 14th International Wireless Communications & Mobile Computing Conference (CORE B).
- *My contribution*: The work is based on my Bachelor’s thesis. I designed the methodology, conducted the experiments, evaluated the results, and contributed to the text.
- *CRediT*: Methodology, Investigation, Software, Writing – draft, editing, and review.
- *Percentage*: 45% (total number of authors: 3).

Bibliographic reference: M. Lastovička, A. Dufka, and J. Komárková, “Machine learning fingerprinting methods in cyber security domain: Which one to use?” In *14th International Wireless Communications & Mobile Computing Conference (IWCMC)*, 2018, pp. 542–547. doi: 10.1109/IWCMC.2018.8450406.

A.3 Teaching and theses supervision

- Seminar teaching:
 - PV080 – Information security and cryptography
 - IB111 – Foundations of programming
 - PB071 – Principles of low-level programming
- Course assignment preparation:
 - PV204 – Security technologies
 - PA193 – Secure coding principles
 - PV079 – Applied cryptography
 - IA174 – Fundamentals of cryptography
- Theses supervision:
 - Detection of Android Malware Using Machine Learning, Matúš Šikyňa, Bachelor's thesis, 2021.
 - Clustering of Mobile Devices Based on Their Security State, Dáša Pawlasová, Bachelor's thesis, 2022.
 - Fuzzing of the OpenSC Project, Veronika Hanulíková, 2022, Bachelor's thesis, awarded Dean's award for outstanding final thesis.
 - Schnorr Multi-Signatures for Secure Devices with Restricted Interfaces, Jakub Janků, 2023, Bachelor's thesis, awarded Dean's award for outstanding final thesis.
 - Policy-based Validation of Bitcoin Transactions on Cryptographic Smartcards, Jan Sekanina, Bachelor's thesis, 2023.
 - Analysis and Use of Standard Cryptographic Interfaces, Kristián Mika, Master's thesis, 2024.

Bibliography

Authored publications

- [A1] P. Švenda, A. Dufka, M. Brož, R. Lacko, T. Jaroš, D. Zaťovič, and J. Pospíšil, “TPMScan: A wide-scale study of security-relevant properties of TPM 2.0 chips,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2024, no. 2, pp. 714–734, 2024. doi: 10.46586/tches.v2024.i2.714-734.
- [A2] P. Švenda, R. Kvašňovský, I. Nagy, and A. Dufka, “JCAIlgTest: Robust identification metadata for certified smartcards,” in *Proceedings of the 19th International Conference on Security and Cryptography - Volume 1: SECRYPT*, S. D. C. di Vimercati, Ed., INSTICC, SciTePress, 2022, pp. 597–604, ISBN: 978-989-758-590-6. doi: 10.5220/0000163500003283.
- [A3] L. Zaoral, A. Dufka, and P. Švenda, “The adoption rate of JavaCard features by certified products and open-source projects,” in *Proceedings of the 22nd Smart Card Research and Advanced Application Conference*, S. Bhasin and T. Roche, Eds., Cham: Springer, 2023, pp. 169–189, ISBN: 978-3-031-54409-5. doi: 10.1007/978-3-031-54409-5_9.
- [A4] A. Dufka and P. Švenda, “Enabling efficient threshold signature computation via Java Card API,” in *Proceedings of the 18th International Conference on Availability, Reliability and Security*, ser. ARES '23, Benevento, Italy: Association for Computing Machinery, 2023, pp. 1–10, ISBN: 9798400707728. doi: 10.1145/3600160.3600180.
- [A5] A. Dufka, P. Laud, and P. Švenda, “Two-party ECDSA with JavaCard-based smartcards,” in *Applied Cryptography and Network Security: 23rd International Conference on Applied Cryptography and Network Security*, to appear, 2025, pp. 1–18.
- [A6] A. Dufka, J. Janků, and P. Švenda, “Trust-minimizing BDHKE-based e-cash mint using secure hardware and distributed computation,” in *21st International Workshop on Trust, Privacy and Security in the Digital Society, Proceedings of the 19th International Conference on Availability, Reliability and Security*, Munich, Germany, 2024, pp. 1–10.

- [A7] A. Dufka, V. Sedláček, and P. Švenda, “SHINE: Resilience via practical interoperability of multi-party Schnorr signature schemes,” in *Proceedings of the 19th International Conference on Security and Cryptography - Volume 1: SECRYPT*, S. D. C. di Vimercati, Ed., INSTICC, SciTePress, 2022, pp. 305–316, ISBN: 978-989-758-590-6. DOI: 10.5220/0011145600003283.
- [A8] A. S. Bhati, A. Dufka, E. Andreeva, A. Roy, and B. Preneel, “Skye: An expanding PRF based fast KDF and its applications,” in *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security*, ser. ASIA CCS ’24, Singapore, Singapore: Association for Computing Machinery, 2024, pp. 1082–1098, ISBN: 9798400704826. DOI: 10.1145/3634737.3637673.
- [A9] V. Sedláček, V. Suchánek, A. Dufka, M. Sýs, and V. Matyáš, “DiSSECT: Distinguisher of standard and simulated elliptic curves via traits,” in *Progress in Cryptology - AFRICACRYPT 2022*, L. Batina and J. Daemen, Eds., Cham: Springer Nature Switzerland, 2022, pp. 493–517, ISBN: 978-3-031-17433-9.
- [A10] M. Lastovička, A. Dufka, and J. Komárková, “Machine learning fingerprinting methods in cyber security domain: Which one to use?” In *14th International Wireless Communications & Mobile Computing Conference (IWCMC)*, 2018, pp. 542–547. DOI: 10.1109/IWCMC.2018.8450406.

External references

- [1] J. A. Halderman, S. D. Schoen, N. Heninger, W. Clarkson, W. Paul, J. A. Calandrino, A. J. Feldman, J. Appelbaum, and E. W. Felten, “Lest we remember: Cold-boot attacks on encryption keys,” *Communications of the ACM*, vol. 52, no. 5, pp. 91–98, 2009.
- [2] P. Kocher, J. Jaffe, and B. Jun, “Differential power analysis,” in *Advances in Cryptology—CRYPTO’99*, Springer, 1999, pp. 388–397.
- [3] M. Nemec, M. Sýs, P. Švenda, D. Klinec, and V. Matyáš, “The return of coppersmith’s attack: Practical factorization of widely used RSA moduli,” in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS 2017*,

- Dallas, TX, USA, October 30 - November 03, 2017, B. M. Thuraisingham, D. Evans, T. Malkin, and D. Xu, Eds., ser. CCS '17, Dallas, Texas, USA: ACM, 2017, pp. 1631–1648, ISBN: 978-1-4503-4946-8. doi: 10.1145/3133956.3133969.
- [4] D. Moghimi, B. Sunar, T. Eisenbarth, and N. Heninger, “TPM-FAIL: TPM meets timing and lattice attacks,” in *29th USENIX Security Symposium, USENIX Security 2020, August 12-14, 2020*, S. Capkun and F. Roesner, Eds., USENIX Association, 2020, pp. 2057–2073.
- [5] J. Jančár, V. Sedláček, P. Švenda, and M. Sýs, “Minerva: The curse of ECDSA nonces; systematic analysis of lattice attacks on noisy leakage of bit-length of ECDSA nonces,” *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2020, no. 4, pp. 281–308, 2020. doi: 10.13154/tches.v2020.i4.281-308.
- [6] T. Roche, *EUCLEAK*, Cryptology ePrint Archive, 2024/1380, 2024. [Online]. Available: <https://eprint.iacr.org/2024/1380>.
- [7] Y. Desmedt and Y. Frankel, “Threshold cryptosystems,” in *Conference on the Theory and Application of Cryptology (CRYPTO)*, Springer, 1989, pp. 307–315.
- [8] Y. G. Desmedt, “Threshold cryptography,” *European Transactions on Telecommunications*, vol. 5, no. 4, pp. 449–458, 1994.
- [9] A. Shamir, “How to share a secret,” *Communications of the ACM*, vol. 22, no. 11, pp. 612–613, 1979.
- [10] NIST. “First call for multi-party threshold schemes.” (2023), [Online]. Available: <https://csrc.nist.gov/publications/detail/nistir/8214c/draft> (visited on Oct. 18, 2024).
- [11] Common Criteria. “Common Criteria for information technology security evaluation,” [Online]. Available: <https://www.commoncriteriaportal.org/> (visited on Oct. 18, 2024).
- [12] NIST, “FIPS 140-2: Security requirements for cryptographic modules,” National Institute for Standards and Technology, Standard, May 2001. doi: 10.6028/NIST.FIPS.140-2.
- [13] M. Hedabou and Y. S. Abdulsalam, “Efficient and secure implementation of BLS multisignature scheme on TPM,” in *IEEE Conference on Intelligence and Security Informatics (ISI'20)*, IEEE, 2020, pp. 1–6.

BIBLIOGRAPHY

- [14] J. Janků, *Schnorr multi-signatures for secure devices with restricted interfaces*, Bachelor's thesis, Masaryk University, 2023. [Online]. Available: <https://is.muni.cz/th/p8u2x/> (visited on Oct. 18, 2024).
- [15] V. Mavroudis and P. Švenda, "Jcmathlib: Wrapper cryptographic library for transparent and certifiable javacard applets," in *2020 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, IEEE, Genoa, Italy: IEEE, 2020, pp. 89–96. doi: 10.1109/EuroSPW51379.2020.00022.
- [16] C. Komlo and I. Goldberg, "Frost: Flexible round-optimized schnorr threshold signatures," in *Selected Areas in Cryptography*, O. Dunkelman, M. J. Jacobson Jr., and C. O'Flynn, Eds., Cham: Springer International Publishing, 2021, pp. 34–65, ISBN: 978-3-030-81652-0.
- [17] D. Connolly, C. Komlo, I. Goldberg, and C. A. Wood, *The flexible round-optimized Schnorr threshold (FROST) protocol for two-round schnorr signatures*, RFC 9591, Jun. 2024. doi: 10.17487/RFC9591.
- [18] L. Chen and J. Li, "Flexible and scalable digital signatures in TPM 2.0," in *Proceedings of the 2013 ACM SIGSAC conference on Computer & communications security*, 2013, pp. 37–48.
- [19] Oracle. "Java Card 3.0.5 platform specification." (2015), [Online]. Available: <https://docs.oracle.com/javacard/3.0.5/index.html> (visited on Oct. 18, 2024).
- [20] Y. Lindell, "Fast secure two-party ecdsa signing," in *Advances in Cryptology—CRYPTO 2017: 37th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 20–24, 2017, Proceedings, Part II 37*, Springer, 2017, pp. 613–644.
- [21] TCG. "ISO/IEC 11889-1:2009." (2009), [Online]. Available: <https://www.iso.org/standard/50970.html> (visited on Oct. 18, 2024).
- [22] TCG. "TPM 2.0 a brief introduction." (2019), [Online]. Available: https://trustedcomputinggroup.org/wp-content/uploads/2019_TCG_TPM2_BriefOverview_DR02web.pdf (visited on Oct. 18, 2024).
- [23] H. Birkholz, D. Thaler, M. Richardson, N. Smith, and W. Pan, *Remote ATtestation procedureS (RATS) Architecture*, RFC 9334, Jan. 2023. doi: 10.17487/RFC9334.

- [24] Y. Kim and E. Kim, "HTPM: Hybrid implementation of trusted platform module," in *Proceedings of the 1st ACM Workshop on Workshop on Cyber-Security Arms Race*, ser. CYSARM'19, London, United Kingdom, 2019, pp. 3–10, ISBN: 9781450368407. doi: 10.1145/3338511.3357348.
- [25] TCPA. "Trusted Computing Platform Specifications v1.0." (2001), [Online]. Available: https://web.archive.org/web/20020806140425/http://www.trustedcomputing.org/docs/tcpa_final.pdf (visited on Oct. 18, 2024).
- [26] TCG, *Trusted Platform Module Library Specification, Family "2.0", Level 00, Revision 01.59*, 2019.
- [27] L. Chen and R. Urian, "Algorithm agility – discussion on TPM 2.0 ECC functionalities," in *Security Standardisation Research: Third International Conference, Proceedings 3*, Springer, 2016, pp. 141–159.
- [28] C. Paquin and G. Zaverucha, "U-Prove cryptographic specification v1.1," *Technical Report, Microsoft Corporation*, 2011.
- [29] L. Chen and R. Urian, "DAA-A: Direct anonymous attestation with attributes," in *8th International Conference on Trust and Trustworthy Computing, Proceedings 8*, Springer, 2015, pp. 228–245.
- [30] TCG. "TCG TPM v2.0 provisioning guidance." (2017), [Online]. Available: <https://trustedcomputinggroup.org/wp-content/uploads/TCG-TPM-v2.0-Provisioning-Guidance-Published-v1r1.pdf> (visited on Oct. 18, 2024).
- [31] TCG. "TCG certification programs," [Online]. Available: <https://trustedcomputinggroup.org/membership/certification/> (visited on Oct. 18, 2024).
- [32] TCG. "TPM Certified Products," [Online]. Available: <https://trustedcomputinggroup.org/membership/certification/tpm-certified-products/> (visited on Oct. 18, 2024).
- [33] P. Švenda, M. Nemec, P. Sekan, R. Kvašňovský, D. Formánek, D. Komárek, and V. Matyáš, "The million-key question – investigating the origins of RSA public keys," in *The 25th USENIX Security Symposium (UsenixSec'2016)*, USENIX, 2016, pp. 893–910, ISBN: 978-1-931971-32-4.

- [34] C. Sun, T. Espitau, M. Tibouchi, and M. Abe, "Guessing bits: Improved lattice attacks on (EC)DSA with nonce leakage," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2022, no. 1, pp. 391–413, Nov. 2021. doi: 10.46586/tches.v2022.i1.391-413.
- [35] J. Hearn, "Does the common criteria paradigm have a future?" *IEEE Security & Privacy Magazine*, vol. 2, no. 1, pp. 64–65, Jan. 2004, ISSN: 1540-7993. doi: 10.1109/MSECP.2004.1264857.
- [36] S. Murdoch, M. Bond, and R. J. Anderson, "How Certification Systems Fail: Lessons from the Ware Report," *IEEE Security & Privacy Magazine*, pp. 1–1, 2012, ISSN: 1540-7993. doi: 10.1109/MSP.2012.89.
- [37] B. Beckert, D. Grahl, and S. Grebing, "Mind the gap: Formal verification and the common criteria (discussion paper)," in *VERIFY@IJCAR*, 2010.
- [38] J. Haney, M. Theofanos, Y. Acar, and S. S. Prettyman, "Organizational views of NIST cryptographic standards and testing and validation programs," National Institute of Standards and Technology, Gaithersburg, MD, Tech. Rep. NIST IR 8241, Dec. 2018, NIST IR 8241. doi: 10.6028/NIST.IR.8241.
- [39] S. P. Kaluvuri, M. Bezzi, and Y. Roudier, "A quantitative analysis of Common Criteria certification practice," in *Trust, Privacy, and Security in Digital Business*, ser. LNCS, C. Eckert, S. K. Katsikas, and G. Pernul, Eds., vol. 8647, Series Title: Lecture Notes in Computer Science, Cham: Springer International Publishing, 2014, pp. 132–143. doi: 10.1007/978-3-319-09770-1_12.
- [40] Jeremy Boone, *TPM Genie: Interposer attacks against the trusted platform module serial bus*, https://github.com/nccgroup/TPMGenie/blob/master/docs/NCC_Group_Jeremy_Boone_TPM_Genie_Whitepaper.pdf, 2018.
- [41] H. N. Jacob, C. Werling, R. Buhren, and J.-P. Seifert, "faultTPM: Exposing AMD fTPMs' deepest secrets," in *2023 IEEE 8th European Symposium on Security and Privacy (EuroSP'23)*, 2023, pp. 1128–1142. doi: 10.1109/EuroSP57164.2023.00069.
- [42] A. Janovský, J. Jančár, P. Švenda, Ł. Chmielewski, J. Michalík, and V. Matyáš, "Sec-certs: Examining the security certification practice for better vulnerability mitigation," *arXiv:2311.17603*,

2023. [Online]. Available: <http://arxiv.org/abs/2311.17603> (visited on Nov. 30, 2023).
- [43] L. Bassham, A. Rukhin, J. Soto, J. Nechvatal, M. Smid, S. Leigh, M. Levenson, M. Vangel, N. Heckert, and D. Banks, *A statistical test suite for random and pseudorandom number generators for cryptographic applications*, Sep. 2010. [Online]. Available: https://tsapps.nist.gov/publication/get_pdf.cfm?pub_id=906762.
- [44] R. G. Brown, D. Eddelbuettel, and D. Bauer, "Dieharder," *Duke University Physics Department Durham, NC*, pp. 27 708–0305, 2018.
- [45] P. Lécuyer and R. Simard, "TestU01: AC library for empirical testing of random number generators," *ACM Transactions on Mathematical Software (TOMS)*, vol. 33, no. 4, pp. 1–40, 2007.
- [46] M. Šýs, D. Klinec, K. Kubíček, and P. Švenda, "Booltest: The fast randomness testing strategy based on boolean functions with application to DES, 3-DES, MD5, MD6 and SHA-256," in *E-Business and Telecommunications: ICETE 2017, Revised Selected Paper 14*, Springer, 2019, pp. 123–149.
- [47] A. K. Lenstra, H. W. Lenstra, and L. Lovász, "Factoring polynomials with rational coefficients," *Mathematische annalen*, vol. 261, no. ARTICLE, pp. 515–534, 1982.
- [48] D. Boneh and R. Venkatesan, "Hardness of computing the most significant bits of secret keys in Diffie-Hellman and related schemes," in *Annual International Cryptology Conference*, Springer, 1996, pp. 129–142.
- [49] A. Janovský, M. Nemec, P. Švenda, P. Sekan, and V. Matyáš, "Biased RSA private keys: Origin attribution of GCD-factorable keys," in *ESORICS'20*, Springer, 2020. doi: 10.1007/978-3-030-59013-0_25.
- [50] TCG. "PC client platform TPM profile (PTP) specification." (2020), [Online]. Available: <https://trustedcomputinggroup.org/resource/pc-client-platform-tpm-profile-ptp-specification/> (visited on Oct. 18, 2024).
- [51] Eurosmart. "2022 and 2023 secure elements' global market estimates," [Online]. Available: https://www.eurosmart.com/wp-content/uploads/2023/04/2022_12_MnT_pressrelease.pdf (visited on Oct. 18, 2024).

- [52] ISO, "ISO/IEC 7816-4:2013 - identification cards – integrated circuit cards – part 4: Organization, security and commands for interchange," International Organization for Standardization, Apr. 2013. [Online]. Available: <https://www.iso.org/standard/54550.html> (visited on Oct. 18, 2024).
- [53] Oracle. "Java Card technology," [Online]. Available: <https://www.oracle.com/java/java-card/> (visited on Oct. 18, 2024).
- [54] Gemalto. "IDPrime .NET," [Online]. Available: https://web.archive.org/web/20140201081249/http://www.gemalto.com/products/dotnet_card/ (visited on Oct. 18, 2024).
- [55] MultOS. "MultOS card." (2024), [Online]. Available: <https://multos.com/> (visited on Oct. 18, 2024).
- [56] Z. Chen, *Java Card Technology for Smart Cards: Architecture and Programmer's Guide*. USA: Addison-Wesley Longman Publishing Co., Inc., 2000, ISBN: 0201703297.
- [57] F. Besson, T. Jensen, and P. Vittet, "Sawjacard: A static analysis tool for certifying Java Card applications," in *International Static Analysis Symposium*, Springer, 2014, pp. 51–67.
- [58] M. Farhadi and J.-L. Lanet, "Chronicle of a Java Card death," *Journal of Computer Virology and Hacking Techniques*, vol. 13, no. 2, pp. 109–123, 2017.
- [59] J. Lancia and G. Bouffard, "Fuzzing and overflows in Java Card smart cards," in *SSTIC Conference, Rennes, France*, 2016.
- [60] B. Laugier and T. Razafindralambo, "Misuse of frame creation to exploit stack underflow attacks on java card," in *International Conference on Smart Card Research and Advanced Applications*, Springer, 2015, pp. 89–104.
- [61] S. Volokitin and E. Poll, "Logical attacks on secured containers of the java card platform," in *International Conference on Smart Card Research and Advanced Applications*, Springer, 2016, pp. 122–136.
- [62] G. Barbu, H. Thiebeauld, and V. Guerin, "Attacks on java card 3.0 combining fault and logical attacks," *Smart Card Research and Advanced Application*, pp. 148–163, 2010.
- [63] M. A. Kasmi, M. Azizi, and J.-L. Lanet, "Side channel analysis techniques towards a methodology for reverse engineering of Java Card byte-code," in *Information Assurance and Security*

- (IAS), *2015 11th International Conference on*, IEEE, 2015, pp. 104–110.
- [64] D. Vermoen, M. Witteman, and G. Gaydadjiev, “Reverse engineering java card applets using power analysis,” *Information Security Theory and Practices. Smart Cards, Mobile and Ubiquitous Computing Systems*, pp. 138–149, 2007.
- [65] J. Hajny, L. Malina, Z. Martinasek, and O. Tethal, “Performance evaluation of primitives for privacy-enhancing cryptography on current smart-cards and smart-phones,” in *Data Privacy Management and Autonomous Spontaneous Security*, Springer, 2014, pp. 17–33.
- [66] G. Bernabé and N. Clarke, “Study of RSA performance in Java Cards,” *Advances in Communications, Computing, Networks and Security Volume 10*, p. 45, 2013.
- [67] S. Bouzefrane, J. Cordry, and P. Paradinas, “A methodology for testing Java Card performance,” in *CFSE’08 Conférence Française en Systèmes d’Exploitation, Suisse*, 2008.
- [68] V. Rijmen and J. Daemen, “Advanced encryption standard,” *Proceedings of federal information processing standards publications, national institute of standards and technology*, vol. 19, p. 22, 2001.
- [69] D. Pei, A. Salomaa, and C. Ding, *Chinese remainder theorem: applications in computing, coding, cryptography*. World Scientific, 1996.
- [70] Oracle. “Java card platform 3.2 specification release notes,” [Online]. Available: <https://docs.oracle.com/en/java/javacard/3.2/specnotes/index.html> (visited on Oct. 18, 2024).
- [71] R. Pawlak, M. Monperrus, N. Petitprez, C. Noguera, and L. Seinturier, “Spoon: A library for implementing analyses and transformations of Java source code,” *Software: Practice and Experience*, vol. 46, pp. 1155–1179, 2015. DOI: 10.1002/spe.2346.
- [72] M. Paljak. “GlobalPlatformPro,” [Online]. Available: <https://github.com/martinpaljak/GlobalPlatformPro> (visited on Oct. 18, 2024).
- [73] Licel Corporation. “jCardSim | Java card runtime environment simulator,” [Online]. Available: <https://jcardsim.org/> (visited on Oct. 18, 2024).

BIBLIOGRAPHY

- [74] C.-P. Schnorr, "Efficient signature generation by smart cards," *Journal of cryptology*, vol. 4, pp. 161–174, 1991.
- [75] D. Johnson, A. Menezes, and S. Vanstone, "The elliptic curve digital signature algorithm (ecdsa)," *International Journal of Information Security*, vol. 1, no. 1, pp. 36–63, 2001.
- [76] D. Boneh, B. Lynn, and H. Shacham, "Short signatures from the weil pairing," in *International conference on the theory and application of cryptology and information security*, Springer, 2001, pp. 514–532.
- [77] H. Tews and B. Jacobs, "Performance issues of selective disclosure and blinded issuing protocols on java card," in *Information Security Theory and Practice. Smart Devices, Pervasive Systems, and Ubiquitous Networks*, O. Markowitch, A. Bilas, J.-H. Hoepman, C. J. Mitchell, and J.-J. Quisquater, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 2009, pp. 95–111, ISBN: 978-3-642-03944-7.
- [78] P. Bichsel, J. Camenisch, T. Groß, and V. Shoup, "Anonymous credentials on a standard java card," in *Proceedings of the 16th ACM Conference on Computer and Communications Security*, ACM, Chicago, Illinois, USA, 2009, pp. 600–610, ISBN: 9781605588940. DOI: 10.1145/1653662.1653734.
- [79] M. Sterckx, B. Gierlichs, B. Preneel, and I. Verbauwhede, "Efficient implementation of anonymous credentials on java card smart cards," in *2009 First IEEE International Workshop on Information Forensics and Security (WIFS)*, London, UK: IEEE, 2009, pp. 106–110. DOI: 10.1109/WIFS.2009.5386474.
- [80] P. Vullers, "Efficient implementations of attribute-based credentials on smart cards," Ph.D. dissertation, Radboud University Nijmegen, 2014.
- [81] V. Mavroudis, A. Cerulli, P. Švenda, D. Cvrček, D. Klinec, and G. Danezis, "A touch of evil: High-assurance cryptographic hardware from untrusted components," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '17, Dallas, Texas, USA: ACM, 2017, pp. 1583–1600, ISBN: 9781450349468. DOI: 10.1145/3133956.3133961.
- [82] A. Dalskov, C. Orlandi, M. Keller, K. Shrishak, and H. Shulman, "Securing DNSSEC keys via threshold ECDSA from generic MPC," in *Computer Security—ESORICS 2020: 25th Eu-*

- ropean Symposium on Research in Computer Security, ESORICS 2020, Guildford, UK, September 14–18, 2020, *Proceedings, Part II* 25, Springer, 2020, pp. 654–673.
- [83] J. Groth and V. Shoup, “On the security of ecdsa with additive key derivation and presignatures,” in *Annual International Conference on the Theory and Applications of Cryptographic Techniques (EUROCRYPT)*, Springer, 2022, pp. 365–396.
 - [84] Smart ID. “The smart way to identify yourself.” (2017), [Online]. Available: <https://www.smart-id.com/about-smart-id/> (visited on Oct. 18, 2024).
 - [85] ZenGo Ltd. “The smart way to identify yourself.” (2022), [Online]. Available: <https://zengo.com/mpc-wallet/> (visited on Oct. 18, 2024).
 - [86] Y. Lindell. “How smart cryptography makes coinbase more secure.” (2022), [Online]. Available: <https://www.coinbase.com/blog/how-smart-cryptography-makes-coinbase-more-secure> (visited on Oct. 18, 2024).
 - [87] J. Doerner, Y. Kondi, E. Lee, *et al.*, “Threshold ECDSA in three rounds,” *Cryptology ePrint Archive*, 2023.
 - [88] J. Camenisch, M. Drijvers, and J. Hajny, “Scalable revocation scheme for anonymous credentials based on n-times unlinkable proofs,” in *Proceedings of the 2016 ACM on Workshop on Privacy in the Electronic Society*, 2016, pp. 123–133.
 - [89] F. Balli, F. B. Durak, and S. Vaudenay, “BioID: A privacy-friendly identity document,” in *Security and Trust Management: 15th International Workshop, STM 2019, Luxembourg City, Luxembourg, September 26–27, 2019, Proceedings*, Springer, Luxembourg, Luxembourg: Springer-Verlag, 2019, pp. 53–70, ISBN: 978-3-030-31510-8. DOI: 10.1007/978-3-030-31511-5_4.
 - [90] R. Casanova-Marqués, P. Dzurenda, and J. Hajny, “Implementation of revocable keyed-verification anonymous credentials on java card,” in *Proceedings of the 17th International Conference on Availability, Reliability and Security*, ser. ARES ’22, Vienna, Austria: ACM, 2022, pp. 1–8, ISBN: 9781450396707. DOI: 10.1145/3538969.3543798.
 - [91] J. Doerner, Y. Kondi, E. Lee, and A. Shelat, “Threshold ECDSA from ECDSA assumptions: The multiparty case,” in *2019 IEEE*

- Symposium on Security and Privacy (SP)*, San Francisco, CA, USA: IEEE, 2019, pp. 1051–1066. doi: 10.1109/SP.2019.00024.
- [92] D. R. L. Brown, *Elliptic curve cryptography, standards for efficient cryptography group, ver. 2*, 2009. [Online]. Available: <https://www.secg.org/sec1-v2.pdf> (visited on Oct. 18, 2024).
- [93] BSI, *TR-03110 technical guideline advanced security mechanisms for machine readable travel documents and eidas token*, 2016. [Online]. Available: <https://www.bsi.bund.de/dok/TR-03110-en> (visited on Oct. 18, 2024).
- [94] L. C. Washington, *Elliptic Curves: Number Theory and Cryptography, Second Edition*, 2nd ed. USA: Chapman & Hall/CRC, 2008, ISBN: 9781420071467.
- [95] R. Struik, “Alternative elliptic curve representations,” Internet Engineering Task Force, Internet-Draft draft-ietf-lwig-curve-representations-23, Jan. 2022, (WIP), 150 pp. [Online]. Available: <https://datatracker.ietf.org/doc/draft-ietf-lwig-curve-representations/23/> (visited on Oct. 18, 2024).
- [96] D. Oswald. “Repository jc_curve25519.” (2015), [Online]. Available: https://github.com/david-oswald/jc%5C_curve25519 (visited on Oct. 18, 2024).
- [97] Oracle. “Java card platform 3.1 specification release notes,” [Online]. Available: <https://docs.oracle.com/en/java/javacard/3.1/specnotes/index.html> (visited on Oct. 18, 2024).
- [98] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, and B.-Y. Yang, “High-speed high-security signatures,” *Journal of cryptographic engineering*, vol. 2, no. 2, pp. 77–89, 2012.
- [99] Coinkite Inc. “Dual secure elements.” (2022), [Online]. Available: <https://github.com/Coldcard/firmware/blob/master/docs/mk4-secure-elements.md> (visited on Oct. 18, 2024).
- [100] P. Wuille, J. Nick, and A. Towns. “Schnorr signatures for secp-256k1.” (2020), [Online]. Available: <https://github.com/bitcoin/bips/blob/master/bip-0340.mediawiki> (visited on Oct. 18, 2024).
- [101] J. Poon and T. Dryja, *The Bitcoin Lightning network: Scalable off-chain instant payments*, 2016. [Online]. Available: <https://lightning.network/>

- [//lightning.network/lightning-network-paper.pdf](https://lightning.network/lightning-network-paper.pdf) (visited on Oct. 18, 2024).
- [102] Á. Ficsór, I. A. Seres, Y. Kogman, and L. Ontivero, *WabiSabi: Centrally coordinated coinjoins with variable amounts*, 2021. [Online]. Available: <https://github.com/zkSNACKs/WabiSabi/releases/latest/download/WabiSabi.pdf> (visited on Oct. 18, 2024).
- [103] C. P. Schnorr, *Method for identifying subscribers and for generating and verifying electronic signatures in a data exchange system*, US Patent 4,995,082, 1991.
- [104] IANIX. "Things that use Ed25519," [Online]. Available: <https://ianix.com/pub/ed25519-deployment.html> (visited on Oct. 18, 2024).
- [105] S. Nakamoto, *Bitcoin: A peer-to-peer electronic cash system*, 2008. [Online]. Available: <https://bitcoin.org/bitcoin.pdf> (visited on Oct. 18, 2024).
- [106] P. Wuille, J. Nick, and A. Towns. "Taproot: Segwit version 1 spending rules." 2024-10-18. (2020), [Online]. Available: <https://github.com/bitcoin/bips/blob/master/bip-0341.mediawiki>.
- [107] H. K. Alper and J. Burdges, "Two-round trip schnorr multi-signatures via delinearized witnesses," in *Annual International Cryptology Conference*, Springer, 2021.
- [108] G. Maxwell, A. Poelstra, Y. Seurin, and P. Wuille, "Simple schnorr multi-signatures with applications to bitcoin," *Designs, Codes and Cryptography*, vol. 87, no. 9, pp. 2139–2164, 2019.
- [109] J. Nick, T. Ruffing, and Y. Seurin, "Musig2: Simple two-round schnorr multi-signatures," in *Annual International Cryptology Conference*, Springer, 2021, pp. 189–221.
- [110] E. Syta, I. Tamas, D. Visser, D. I. Wolinsky, P. Jovanovic, L. Gasser, N. Gailly, I. Khoffi, and B. Ford, "Keeping authorities 'honest or bust' with decentralized witness cosigning," in *IEEE Symposium on Security and Privacy*, 2016, pp. 526–545.
- [111] E. Crites, C. Komlo, and M. Maller, "How to prove schnorr assuming schnorr: Security of multi-and threshold signatures," *Cryptology ePrint Archive*, 2021.

BIBLIOGRAPHY

- [112] J. Nick, T. Ruffing, and E. Jin. “MuSig2 for BIP340-compatible multi-signatures.” (2020), [Online]. Available: <https://github.com/bitcoin/bips/blob/master/bip-0327.mediawiki> (visited on Oct. 18, 2024).
- [113] A. Fiat and A. Shamir, “How to prove yourself: Practical solutions to identification and signature problems,” in *Conference on the theory and application of cryptographic techniques*, Springer, 1986, pp. 186–194.
- [114] D. Pointcheval and J. Stern, “Security arguments for digital signatures and blind signatures,” *Journal of cryptology*, vol. 13, no. 3, pp. 361–396, 2000.
- [115] H. Morita, J. C. Schuldt, T. Matsuda, G. Hanaoka, and T. Iwata, “On the security of the schnorr signature scheme and dsa against related-key attacks,” in *Information Security and Cryptology - ICISC 2015*, Springer, 2015, pp. 20–35.
- [116] D. Boneh, M. Drijvers, and G. Neven, “Compact multi-signatures for smaller blockchains,” in *International Conference on the Theory and Application of Cryptology and Information Security*, Springer, 2018, pp. 435–464.
- [117] M. Drijvers, K. Edalatnejad, B. Ford, E. Kiltz, J. Loss, G. Neven, and I. Stepanovs, “On the security of two-round multi-signatures,” in *2019 IEEE Symposium on Security and Privacy (SP)*, San Francisco, CA, USA: IEEE, 2019, pp. 1084–1101. doi: 10.1109/SP.2019.00050.
- [118] C. P. Schnorr, “Security of blind discrete log signatures against interactive attacks,” in *International Conference on Information and Communications Security*, Springer, 2001, pp. 1–12.
- [119] D. Wagner, “A generalized birthday problem,” in *Annual International Cryptology Conference*, Springer, 2002, pp. 288–304.
- [120] F. Benhamouda, T. Lepoint, J. Loss, M. Orrù, and M. Raykova, “On the (in)security of ROS,” *J. Cryptology*, vol. 35, no. 4, p. 25, 2022.
- [121] J. Nick, T. Ruffing, Y. Seurin, and P. Wuille, “Musig-DN: Schnorr multi-signatures with verifiably deterministic nonces,” in *27th ACM CCS*, 2020, pp. 1717–1731.
- [122] F. Garillot, Y. Kondi, P. Mohassel, and V. Nikolaenko, “Threshold schnorr with stateless deterministic signing from standard

- assumptions,” in *Annual International Cryptology Conference*, Springer, 2021, pp. 127–156.
- [123] L. Zaoral, *Automatic performance profiler for security analysis of cryptographic smart cards*, Master’s thesis, Masaryk University, 2023. [Online]. Available: <https://is.muni.cz/th/v7130/> (visited on Oct. 18, 2024).
- [124] M. Bellare, C. Namprempre, D. Pointcheval, and M. Semanko, “The one-more-rsa-inversion problems and the security of chaum’s blind signature scheme.,” *Journal of Cryptology*, vol. 16, no. 3, 2003.
- [125] M. Bellare and P. Rogaway, “Random oracles are practical: A paradigm for designing efficient protocols,” in *Proceedings of the 1st ACM Conference on Computer and Communications Security*, 1993, pp. 62–73.
- [126] A. Bagherzandi, J.-H. Cheon, and S. Jarecki, “Multisignatures secure under the discrete logarithm assumption and a generalized forking lemma,” in *Proceedings of the 15th ACM conference on Computer and communications security*, 2008, pp. 449–458.
- [127] P. Dzurenda, S. Ricci, J. Hajny, and L. Malina, “Performance analysis and comparison of different elliptic curves on smart cards,” in *2017 15th Annual Conference on Privacy, Security and Trust (PST)*, IEEE, 2017, pp. 365–36509.
- [128] E. Rescorla and T. Dierks, *The Transport Layer Security (TLS) Protocol Version 1.2*, RFC 5246, Aug. 2008. doi: 10.17487/RFC5246.
- [129] O. Surý, *Use of the SHA-256 Algorithm with RSA, Digital Signature Algorithm (DSA), and Elliptic Curve DSA (ECDSA) in SSHFP Resource Records*, RFC 6594, Apr. 2012. doi: 10.17487/RFC6594.
- [130] GnuPG, *What’s new in GnuPG 2.1*, 2014. [Online]. Available: <https://gnupg.org/faq/whats-new-in-2.1.html>.
- [131] Y. Ishai, M. Prabhakaran, and A. Sahai, “Secure arithmetic computation with no honest majority,” in *Theory of Cryptography: 6th Theory of Cryptography Conference, TCC 2009, San Francisco, CA, USA, March 15-17, 2009. Proceedings 6*, Springer, 2009, pp. 294–314.
- [132] D. Beaver, “Efficient multiparty protocols using circuit randomization,” in *Advances in Cryptology—CRYPTO’91: Proceedings 11*, Springer, 1992, pp. 420–432.

- [133] I. Damgård, M. Keller, E. Larraia, V. Pastro, P. Scholl, and N. P. Smart, "Practical covertly secure MPC for dishonest majority—or: Breaking the SPDZ limits," in *Computer Security—ESORICS 2013: 18th European Symposium on Research in Computer Security*, Egham, UK, September 9–13, 2013. *Proceedings 18*, Springer, 2013, pp. 1–18.
- [134] I. Damgård, V. Pastro, N. Smart, and S. Zakarias, "Multiparty computation from somewhat homomorphic encryption," in *Annual Cryptology Conference*, Springer, 2012, pp. 643–662.
- [135] P. Paillier, "Public-key cryptosystems based on composite degree residuosity classes," in *International conference on the theory and applications of cryptographic techniques*, Springer, 1999, pp. 223–238.
- [136] V. Shoup, "Lower bounds for discrete logarithms and related problems," in *Advances in Cryptology—EUROCRYPT'97: International Conference on the Theory and Application of Cryptographic Techniques Konstanz, Germany, May 11–15, 1997 Proceedings 16*, Springer, 1997, pp. 256–266.
- [137] M. Bellare and C. Namprempre, "Authenticated encryption: Relations among notions and analysis of the generic composition paradigm," *Advances in Cryptology—ASIACRYPT 2000*, pp. 531–545, 2000.
- [138] W. F. Ehrsam, C. H. Meyer, J. L. Smith, and W. L. Tuchman, *Message verification and transmission error detection by block chaining*, US Patent 4,074,066, Feb. 1978.
- [139] M. Bellare, J. Kilian, and P. Rogaway, "The security of the cipher block chaining message authentication code," *Journal of Computer and System Sciences*, vol. 61, no. 3, pp. 362–399, 2000.
- [140] D. Chaum, "Blind signatures for untraceable payments," in *Advances in Cryptology*, D. Chaum, R. L. Rivest, and A. T. Sherman, Eds., Boston, MA: Springer US, 1983, pp. 199–203, ISBN: 978-1-4757-0602-4.
- [141] G. Fuchsbauer, A. Plouviez, and Y. Seurin, "Blind schnorr signatures and signed elgamal encryption in the algebraic group model," in *Annual International Conference on the Theory and Applications of Cryptographic Techniques (EUROCRYPT)*, Springer, 2020, pp. 63–95.

- [142] J. Kastner, J. Loss, and J. Xu, "The Abe-Okamoto partially blind signature scheme revisited," in *International Conference on the Theory and Application of Cryptology and Information Security*, Springer, 2022, pp. 279–309.
- [143] S. Tessaro and C. Zhu, "Short pairing-free blind signatures with exponential security," in *Annual International Conference on the Theory and Applications of Cryptographic Techniques (EUROCRYPT)*, Springer, 2022, pp. 782–811.
- [144] E. Crites, C. Komlo, M. Maller, S. Tessaro, and C. Zhu, "Snow-blind: A threshold blind signature in pairing-free groups," in *Annual International Cryptology Conference*, Springer, 2023, pp. 710–742.
- [145] D. Wagner. "Chaumian ecash without RSA." (1996), [Online]. Available: <https://cypherpunks.venona.com/date/1996/03/msg01848.html> (visited on Oct. 18, 2024).
- [146] A. Boldyreva, "Threshold signatures, multisignatures and blind signatures based on the gap-diffie-hellman-group signature scheme," in *International Workshop on Public Key Cryptography*, Springer, 2002, pp. 31–46.
- [147] S. Jarecki, A. Kiayias, and H. Krawczyk, "Round-optimal password-protected secret sharing and T-PAKE in the password-only model," in *Advances in Cryptology—ASIACRYPT 2014: 20th International Conference on the Theory and Application of Cryptology and Information Security, Kaoshiung, Taiwan, ROC, December 7–11, 2014, Proceedings, Part II 20*, Springer, 2014, pp. 233–253.
- [148] D. L. Vo, F. Zhang, and K. Kim, "A new threshold blind signature scheme from pairings," in *SCIS2003*, SCIS, 2003, pp. 233–238.
- [149] D. Chaum, "Security without identification: Transaction systems to make big brother obsolete," *Communications of the ACM*, vol. 28, no. 10, pp. 1030–1044, 1985.
- [150] D. Pointcheval and J. Stern, "Provably secure blind signature schemes," in *International Conference on the Theory and Application of Cryptology and Information Security*, Springer, 1996, pp. 252–265.
- [151] A. Tomescu, A. Bhat, B. Applebaum, I. Abraham, G. Gueta, B. Pinkas, and A. Yanai, "UTT: Decentralized ecash with accountable privacy," *Cryptology ePrint Archive*, 2022.

BIBLIOGRAPHY

- [152] D. Pointcheval and O. Sanders, "Short randomizable signatures," in *Topics in Cryptology-CT-RSA 2016: The Cryptographers' Track at the RSA Conference 2016, San Francisco, CA, USA, February 29-March 4, 2016, Proceedings*, Springer, 2016, pp. 111–126.
- [153] M. Baudet, A. Sonnino, M. Kelkar, and G. Danezis, "Zef: Low-latency, scalable, private payments," in *Proceedings of the 22nd Workshop on Privacy in the Electronic Society*, 2023, pp. 1–16.
- [154] A. Rial and A. M. Piotrowska, "Compact and divisible e-cash with threshold issuance," *arXiv preprint arXiv:2303.08221*, 2023.
- [155] A. J. Menezes, P. C. Van Oorschot, and S. A. Vanstone, *Handbook of applied cryptography*. CRC press, 2018.
- [156] D. Chaum and T. P. Pedersen, "Wallet databases with observers," in *Annual international cryptology conference*, Springer, 1992, pp. 89–105.
- [157] D. Chaum, A. Fiat, and M. Naor, "Untraceable electronic cash," in *Advances in Cryptology—CRYPTO'88: Proceedings 8*, Springer, 1990, pp. 319–327.
- [158] J. Camenisch and A. Lysyanskaya, "Signature schemes and anonymous credentials from bilinear maps," in *Annual international cryptology conference*, Springer, 2004, pp. 56–72.
- [159] V. Kuchta and M. Manulis, "Rerandomizable threshold blind signatures," in *Trusted Systems: 6th International Conference, INTRUST 2014, Beijing, China, December 16-17, 2014, Revised Selected Papers 6*, Springer, 2015, pp. 70–89.
- [160] G. Fuchsbauer, E. Kiltz, and J. Loss, "The algebraic group model and its applications," in *Advances in Cryptology—CRYPTO 2018: 38th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19–23, 2018, Proceedings, Part II 38*, Springer, 2018, pp. 33–62.
- [161] D. Chaum and E. Van Heyst, "Group signatures," in *Advances in Cryptology—EUROCRYPT'91: Workshop on the Theory and Application of Cryptographic Techniques Brighton, UK, April 8–11, 1991 Proceedings 10*, Springer, 1991, pp. 257–265.
- [162] A. Back, "Hashcash – a denial of service counter-measure," 2002.
- [163] NIST, "FIPS 203: Module-lattice-based key-encapsulation mechanism standard," Standard, 2024. doi: 10.6028/NIST.FIPS.203.

- [164] NIST, “FIPS 204: Module-lattice-based digital signature standard,” National Institute of Standards and Technology, Gaithersburg, Standard, 2024. doi: 10.6028/NIST.FIPS.204.
- [165] NIST, “FIPS 205: Stateless hash-based digital signature standard,” National Institute of Standards and Technology, Standard, 2024. doi: 10.6028/NIST.FIPS.205.
- [166] D. Cozzo and N. P. Smart, “Sharing the LUOV: Threshold post-quantum signatures,” in *IMA International Conference on Cryptography and Coding*, Springer, 2019, pp. 128–153.
- [167] SGS Brightsight B.V., *Samsung STRONGV4P00 of S5E9945 with specific IC dedicated software revision 1.0*, NSCIB-CC-2300085-01, 2024. [Online]. Available: <https://sec-certs.org/cc/54e6eb5c1bd57036/>.
- [168] D. Cozzo and N. P. Smart, “Sashimi: Cutting up CSI-FiSh secret keys to produce an actively secure distributed signing protocol,” in *Post-Quantum Cryptography: 11th International Conference, PQCrypto 2020, Paris, France, April 15–17, 2020, Proceedings 11*, Springer, 2020, pp. 169–186.
- [169] J. Vakarjuk, N. Snetkov, and J. Willemson, “Dilizium: A two-party lattice-based signature scheme,” *Entropy*, vol. 23, no. 8, p. 989, 2021.
- [170] N. Snetkov, J. Vakarjuk, and P. Laud, “TOPCOAT: Towards practical two-party Crystals-Dilithium,” *Discover Computing*, vol. 27, no. 1, p. 18, 2024.
- [171] L. Allen, A. O’Connell, and V. Kiermer, “How can we ensure visibility and diversity in research contributions? how the contributor role taxonomy (credit) is helping the shift from authorship to contributorship,” *Learned Publishing*, vol. 32, no. 1, pp. 71–74, 2019.